

Codebase Dump: Wolfenstein3D_Clone

54 files

Header/basicShader.h

```
#ifndef BASICSHADER_H
#define BASICSHADER_H

#include "shader.h"

class BasicShader : public Shader
{
public:
    static BasicShader* getInstance()
    {
        static BasicShader instance;
        return &instance;
    }

    virtual void updateUniforms(const Matrix4f& worldMatrix,
                               const Matrix4f& projectedMatrix,
                               Material& material);

protected:
private:
    BasicShader();
    ~BasicShader();

    BasicShader(BasicShader const&)
    {
    } // Don't Implement
    void operator=(BasicShader const&)
    {
    } // Don't implement
};

#endif
```

Header/bitmap.h

```
#ifndef BITMAP_H
#define BITMAP_H

#include <string>
#include <vector>

class Bitmap
{
public:
    Bitmap(const std::string& fileName);
    Bitmap(int width, int height);

    Bitmap& flipX();
    Bitmap& flipY();

    int getWidth();
    int getHeight();
    const std::vector<int> getPixels() const;
    int getPixel(int x, int y) const;

    void setPixel(int x, int y, int value);

    Bitmap& operator=(const Bitmap& other);
    Bitmap& operator=(Bitmap* other);

    Bitmap(const Bitmap& other);

protected:
private:
    int m_width;
    int m_height;
    std::vector<int> m_pixels;
};

#endif // !BITMAP_H
```

Header/camera.h

```
#ifndef CAMERA_H
#define CAMERA_H

#include "custom_time.h"
#include "input.h"
#include "vector3f.h"
#include "window.h"

class Camera
{
public:
    Camera(Vector3f pos = Vector3f(0, 0, 0), Vector3f forward = Vector3f(0, 0, 1),
           Vector3f up = Vector3f(0, 1, 0));
    ~Camera();

    void input();
    void move(const Vector3f& direction, float amt);
    void rotateY(float angle);
    void rotateX(float angle);

    Vector3f& getPos();
    Vector3f& getForward();
    Vector3f& getUp();

    Vector3f getLeft() const;
    Vector3f getRight() const;

    void setPos(const Vector3f& pos);
    void setForward(const Vector3f& forward);
    void setUp(const Vector3f& up);

protected:
private:
    Vector3f m_pos;
    Vector3f m_forward;
    Vector3f m_up;
};

#endif
```

Header/custom_time.h

```
#ifndef CUSTOMTIME_H
#define CUSTOMTIME_H

#include <chrono>

class Time
{
public:
    static const long SECOND = 1000000000L;

    static double getTime();
    static double getDelta();

    static void setDelta(double value);

protected:
private:
    static double m_delta;
    static std::chrono::high_resolution_clock::time_point m_epoch;
};

#endif
```

Header/door.h

```
#ifndef DOOR_H
#define DOOR_H

#include "custom_time.h"
#include "material.h"
#include "mesh.h"
#include "shader.h"
#include "transform.h"
#include "util.h"
#include "vector2f.h"
#include "vector3f.h"

class Door
{
public:
    Door()
        : m_closePosition(Vector3f(0, 0, 0)), isOpening(false), openingStartTime(0), openTime(0),
          closingStartTime(0), closeTime(0)
    {
    }
    Door(Transform transform, Material* material, Vector3f openPosition);

    void open();
    void update();
    void render();

    Vector2f getDoorSize() const;
    Transform& getTransform();

protected:
private:
    Vector3f VectorLerp(const Vector3f& startPos, const Vector3f& endPos, float lerpFactor);

    static Mesh* m_mesh;
    Transform m_transform;

    Vector3f m_openPosition;
    Vector3f m_closePosition;

    Material* m_material;

    bool isOpening;
    double openingStartTime;
    double openTime;
    double closingStartTime;
    double closeTime;
};

#endif // !DOOR_H
```

Header/game.h

```
#ifndef GAME_H
#define GAME_H

#include "level.h"

class Game
{
public:
    Game();
    void input();
    void update();
    void render();

    static Level* getLevel();
    static void loadNextLevel();
    static void setIsRunning(bool value);

protected:
private:
    static Level* m_level;
    static bool isRunning;
    static int levelNum;
};

#endif // !GAME_H
```

Header/input.h

```
#ifndef MYINPUT_H
#define MYINPUT_H

#include "sdl_requests.h"
#include "vector2f.h"

typedef enum
{
    LEFT_MOUSE = 1,
    MIDDLE_MOUSE = 2,
    RIGHT_MOUSE = 3,
    MOUSE_WHEEL_UP = 4,
    MOUSE_WHEEL_DOWN = 5
} MOUSE;

typedef enum
{
    KEY_UNKNOWN = 0,

    KEY_A = 4,
    KEY_B = 5,
    KEY_C = 6,
    KEY_D = 7,
    KEY_E = 8,
    KEY_F = 9,
    KEY_G = 10,
    KEY_H = 11,
    KEY_I = 12,
    KEY_J = 13,
    KEY_K = 14,
    KEY_L = 15,
    KEY_M = 16,
    KEY_N = 17,
    KEY_O = 18,
    KEY_P = 19,
    KEY_Q = 20,
    KEY_R = 21,
    KEY_S = 22,
    KEY_T = 23,
    KEY_U = 24,
    KEY_V = 25,
    KEY_W = 26,
    KEY_X = 27,
    KEY_Y = 28,
    KEY_Z = 29,

    KEY_1 = 30,
    KEY_2 = 31,
    KEY_3 = 32,
    KEY_4 = 33,
    KEY_5 = 34,
    KEY_6 = 35,
    KEY_7 = 36,
    KEY_8 = 37,
    KEY_9 = 38,
    KEY_0 = 39,

    KEY_RETURN = 40,
    KEY_ESCAPE = 41,
    KEY_BACKSPACE = 42,
    KEY_TAB = 43,
    KEY_SPACE = 44,

    KEY_MINUS = 45,
    KEY_EQUALS = 46,
    KEY_LEFTBRACKET = 47,
    KEY_RIGHTBRACKET = 48,
    KEY_BACKSLASH = 49,
    KEY_NONUSHASH = 50,
    KEY_SEMICOLON = 51,
    KEY_APOSTROPHE = 52,
    KEY_GRAVE = 53,
    KEY_COMMA = 54,
    KEY_PERIOD = 55,
```

```
KEY_SLASH = 56,

KEY_CAPSLOCK = 57,

KEY_F1 = 58,
KEY_F2 = 59,
KEY_F3 = 60,
KEY_F4 = 61,
KEY_F5 = 62,
KEY_F6 = 63,
KEY_F7 = 64,
KEY_F8 = 65,
KEY_F9 = 66,
KEY_F10 = 67,
KEY_F11 = 68,
KEY_F12 = 69,

KEY_PRINTSCREEN = 70,
KEY_SCROLLLOCK = 71,
KEY_PAUSE = 72,
KEY_INSERT = 73,

KEY_HOME = 74,
KEY_PAGEUP = 75,
KEY_DELETE = 76,
KEY_END = 77,
KEY_PAGEDOWN = 78,
KEY_RIGHT = 79,
KEY_LEFT = 80,
KEY_DOWN = 81,
KEY_UP = 82,

KEY_NUMLOCKCLEAR = 83,

KEY_KP_DIVIDE = 84,
KEY_KP_MULTIPLY = 85,
KEY_KP_MINUS = 86,
KEY_KP_PLUS = 87,
KEY_KP_ENTER = 88,
KEY_KP_1 = 89,
KEY_KP_2 = 90,
KEY_KP_3 = 91,
KEY_KP_4 = 92,
KEY_KP_5 = 93,
KEY_KP_6 = 94,
KEY_KP_7 = 95,
KEY_KP_8 = 96,
KEY_KP_9 = 97,
KEY_KP_0 = 98,
KEY_KP_PERIOD = 99,

KEY_NONUSBACKSLASH = 100,
KEY_APPLICATION = 101,
KEY_POWER = 102,

KEY_KP_EQUALS = 103,
KEY_F13 = 104,
KEY_F14 = 105,
KEY_F15 = 106,
KEY_F16 = 107,
KEY_F17 = 108,
KEY_F18 = 109,
KEY_F19 = 110,
KEY_F20 = 111,
KEY_F21 = 112,
KEY_F22 = 113,
KEY_F23 = 114,
KEY_F24 = 115,
KEY_EXECUTE = 116,
KEY_HELP = 117,
KEY_MENU = 118,
KEY_SELECT = 119,
KEY_STOP = 120,
KEY_AGAIN = 121,
KEY_UNDO = 122,
KEY_CUT = 123,
KEY_COPY = 124,
```



```
KEY_PASTE = 125,
KEY_FIND = 126,
KEY_MUTE = 127,
KEY_VOLUMEUP = 128,
KEY_VOLUMEDOWN = 129,
KEY_KP_COMMA = 133,
KEY_KP_EQUALSAS400 = 134,

KEY INTERNATIONAL1 = 135,
KEY INTERNATIONAL2 = 136,
KEY INTERNATIONAL3 = 137,
KEY INTERNATIONAL4 = 138,
KEY INTERNATIONAL5 = 139,
KEY INTERNATIONAL6 = 140,
KEY INTERNATIONAL7 = 141,
KEY INTERNATIONAL8 = 142,
KEY INTERNATIONAL9 = 143,
KEY_LANG1 = 144, /**< Hangul/English toggle */
KEY_LANG2 = 145, /**< Hanja conversion */
KEY_LANG3 = 146, /**< Katakana */
KEY_LANG4 = 147, /**< Hiragana */
KEY_LANG5 = 148, /**< Zenkaku/Hankaku */
KEY_LANG6 = 149, /**< reserved */
KEY_LANG7 = 150, /**< reserved */
KEY_LANG8 = 151, /**< reserved */
KEY_LANG9 = 152, /**< reserved */

KEY_ALTERASE = 153, /**< Erase-Eaze */
KEY_SYSREQ = 154,
KEY_CANCEL = 155,
KEY_CLEAR = 156,
KEY_PRIOR = 157,
KEY_RETURN2 = 158,
KEY_SEPARATOR = 159,
KEY_OUT = 160,
KEY_OPER = 161,
KEY_CLEARAGAIN = 162,
KEY_CRSEL = 163,
KEY_EXSEL = 164,

KEY_KP_00 = 176,
KEY_KP_000 = 177,
KEY_THOUSANDSSEPARATOR = 178,
KEY_DECIMALSEPARATOR = 179,
KEY_CURRENCYUNIT = 180,
KEY_CURRENCYSUBUNIT = 181,
KEY_KP_LEFTPAREN = 182,
KEY_KP_RIGHTPAREN = 183,
KEY_KP_LEFTBRACE = 184,
KEY_KP_RIGHTBRACE = 185,
KEY_KP_TAB = 186,
KEY_KP_BACKSPACE = 187,
KEY_KP_A = 188,
KEY_KP_B = 189,
KEY_KP_C = 190,
KEY_KP_D = 191,
KEY_KP_E = 192,
KEY_KP_F = 193,
KEY_KP_XOR = 194,
KEY_KP_POWER = 195,
KEY_KP_PERCENT = 196,
KEY_KP_LESS = 197,
KEY_KP_GREATER = 198,
KEY_KP_AMPERSAND = 199,
KEY_KP_DBLAMPERSAND = 200,
KEY_KP_VERTICALBAR = 201,
KEY_KP_DBLVERTICALBAR = 202,
KEY_KP_COLON = 203,
KEY_KP_HASH = 204,
KEY_KP_SPACE = 205,
KEY_KP_AT = 206,
KEY_KP_EXCLAM = 207,
KEY_KP_MEMSTORE = 208,
KEY_KP_MEMRECALL = 209,
KEY_KP_MEMCLEAR = 210,
KEY_KP_MEMADD = 211,
KEY_KP_MEMSUBTRACT = 212,
```

```

KEY_KP_MEMMULTIPLY = 213,
KEY_KP_MEMDIVIDE = 214,
KEY_KP_PLUSMINUS = 215,
KEY_KP_CLEAR = 216,
KEY_KP_CLEARENTRY = 217,
KEY_KP_BINARY = 218,
KEY_KP_OCTAL = 219,
KEY_KP_DECIMAL = 220,
KEY_KP_HEXADecimal = 221,

KEY_LCTRL = 224,
KEY_LSHIFT = 225,
KEY_LALT = 226, /**< alt, option */
KEY_LGUI = 227, /**< windows, command (apple), meta */
KEY_RCTRL = 228,
KEY_RSHIFT = 229,
KEY_RALT = 230, /**< alt gr, option */
KEY_RGUI = 231, /**< windows, command (apple), meta */

KEY_MODE = 257,
KEY_AUDIONEXT = 258,
KEY_AUDIOPREV = 259,
KEY_AUDIOSTOP = 260,
KEY_AUDIOPLAY = 261,
KEY_AUDIOMUTE = 262,
KEY_MEDIASELECT = 263,
KEY_WWW = 264,
KEY_MAIL = 265,
KEY_CALCULATOR = 266,
KEY_COMPUTER = 267,
KEY_AC_SEARCH = 268,
KEY_AC_HOME = 269,
KEY_AC_BACK = 270,
KEY_AC_FORWARD = 271,
KEY_AC_STOP = 272,
KEY_AC_REFRESH = 273,
KEY_AC_BOOKMARKS = 274,

KEY_BRIGHTNESSDOWN = 275,
KEY_BRIGHTNESSUP = 276,
KEY_DISPLAYSWITCH = 277, /**< display mirroring/dual display
                                switch, video mode switch */

KEY_KBDILLUMTOGGLE = 278,
KEY_KBDILLUMDOWN = 279,
KEY_KBDILLUMUP = 280,
KEY_EJECT = 281,
KEY_SLEEP = 282,

KEY_APP1 = 283,
KEY_APP2 = 284,
} KEY;

class Input
{
public:
    static void update();

    static bool getKey(int keyCode);
    static bool getKeyDown(int keyCode);
    static bool getKeyUp(int keyCode);

    static bool getMouse(int button);
    static bool getMouseDown(int button);
    static bool getMouseUp(int button);

    static Vector2f getMousePosition();

    static void setCursor(bool value);
    static void setMousePosition(Vector2f pos);

protected:
private:
};

#endif

```

Header/level.h

```
#ifndef LEVEL_H
#define LEVEL_H

#include "bitmap.h"
#include "door.h"
#include "material.h"
#include "medkit.h"
#include "mesh.h"
#include "monster.h"
#include "player.h"
#include "shader.h"
#include "transform.h"
#include "vector2f.h"
#include "vector3f.h"
#include "vertex.h"
#include <string>
#include <vector>

class Level
{
public:
    Level(const std::string& levelName, const std::string& textureName);

    void input();
    void update();
    void render();

    Vector3f checkCollision(Vector3f& oldPos, Vector3f& newPos, float objWidth, float objLength);
    Vector2f checkIntersection(const Vector2f& lineStart, const Vector2f& lineEnd, bool hurtMonster);
    Vector2f lineIntersectRect(const Vector2f& lineStart, const Vector2f& lineEnd,
                              const Vector2f& rectPos, const Vector2f& rectSize);

    Player* getPlayer();
    Shader* getShader();

    void openDoors(Vector3f pos, bool tryExitLevel);
    void damagePlayer(int amount);
    void removeMedkit(Medkit* medkit);

protected:
private:
    std::vector<float> calcTexCoords(int value);

    Vector2f findNearestVector2f(const Vector2f& a, const Vector2f& b,
                                const Vector2f& positionRelativeTo);
    Vector2f lineIntersect(const Vector2f& lineStart1, const Vector2f& lineEnd1,
                           const Vector2f& lineStart2, const Vector2f& lineEnd2);
    Vector2f rectCollide(Vector2f oldPos, Vector2f newPos, Vector2f size1, Vector2f pos2,
                         Vector2f size2);

    float Vector2fCross(const Vector2f& a, const Vector2f& b);

    void addVertices(std::vector<Vertex>& vertices, int i, int j, float offset, bool x, bool y,
                    bool z, std::vector<float> texCoord);
    void addFace(std::vector<int>& indices, int startLocation, bool direction);
    void addSpecial(int blueValue, int x, int y);
    void addDoor(int x, int y);
    void generateLevel();

    std::vector<Door*> m_doors;
    std::vector<Monster*> m_monsters;

    std::vector<Medkit> m_medkits;
    std::vector<Vector3f> exitPoints;
    std::vector<Vector2f> collisionPosStart;
    std::vector<Vector2f> collisionPosEnd;

    std::vector<Medkit*> m_medkitsToRemove;

    Transform m_transform;
    Player m_player;

    Mesh m_mesh;
    Bitmap m_level;
};
```

```
Texture m_texture;  
Shader* m_shader;  
Material m_material;  
};  
  
#endif // !LEVEL_H
```

Header/lighting.h

```
#ifndef LIGHTING_H
#define LIGHTING_H

#include "vector3f.h"

struct BaseLight
{
    Vector3f color;
    float intensity;

    BaseLight(Vector3f color = Vector3f(0, 0, 0), float intensity = 0)
    {
        this->color = color;
        this->intensity = intensity;
    }
};

struct DirectionalLight
{
    BaseLight base;
    Vector3f direction;

    DirectionalLight(BaseLight base = BaseLight(),
                     Vector3f direction = Vector3f(0, 0, 0))
    {
        this->base = base;
        this->direction = direction;
    }
};

struct Attenuation
{
    float constant;
    float linear;
    float exponent;

    Attenuation(float constant = 0, float linear = 0, float exponent = 1)
    {
        this->constant = constant;
        this->linear = linear;
        this->exponent = exponent;
    }
};

struct PointLight
{
    BaseLight base;
    Attenuation atten;
    Vector3f position;
    float range;

    PointLight(BaseLight base = BaseLight(), Attenuation atten = Attenuation(),
               Vector3f position = Vector3f(), float range = 0)
    {
        this->base = base;
        this->atten = atten;
        this->position = position;
        this->range = range;
    }
};

struct SpotLight
{
    PointLight pointLight;
    Vector3f direction;
    float cutoff;

    SpotLight(PointLight pointLight = PointLight(),
               Vector3f direction = Vector3f(), float cutoff = 0)
    {
        this->pointLight = pointLight;
        this->direction = direction;
        this->cutoff = cutoff;
    }
};
```

```
};  
#endif
```

Header/main.h

```
#ifndef MAINCOMPONENT_H
#define MAINCOMPONENT_H

#include "custom_time.h"
#include "game.h"
#include "input.h"
#include "renderUtil.h"
#include "util.h"
#include "window.h"

class Main
{
public:
    Main();
    ~Main();

    void start();
    void stop();

protected:
private:
    bool isRunning;
    Game game;

    void run();
};

#endif // MAINCOMPONENT_H
```

Header/material.h

```
#ifndef MATERIAL_H
#define MATERIAL_H

#include "texture.h"
#include "vector3f.h"

struct Material
{
    Texture* texture;
    Vector3f color;
    float specularIntensity;
    float specularPower;

    Material(Texture* texture = NULL, const Vector3f& color = Vector3f(1, 1, 1),
             float specularIntensity = 2, float specularPower = 32)
    {
        this->texture = texture;
        this->color = color;
        this->specularIntensity = specularIntensity;
        this->specularPower = specularPower;
    }
    Material(const Material& other)
    {
        this->texture = other.texture;
        this->color = other.color;
        this->specularIntensity = other.specularIntensity;
        this->specularPower = other.specularPower;
    }
    Texture* getTexture() const
    {
        return texture;
    }
    void setTexture(Texture* texture)
    {
        this->texture = texture;
    }

    Vector3f getColor() const
    {
        return color;
    }
    void setColor(const Vector3f& color)
    {
        this->color = color;
    }

    float getSpecularPower() const
    {
        return specularPower;
    }
    void setSpecularPower(float value)
    {
        this->specularPower = value;
    }

    float getSpecularIntensity() const
    {
        return specularIntensity;
    }
    void setSpecularIntensity(float value)
    {
        this->specularIntensity = value;
    }
};

#endif
```


Header/matrix4f.h

```
#ifndef MATRIX4F_H
#define MATRIX4F_H

#include "util.h"
#include "vector3f.h"

class Matrix4f
{
public:
    void initIdentity();
    void initScaleTransform(float scaleX, float scaleY, float scaleZ);
    void initRotateTransform(float rotateX, float rotateY, float rotateZ);
    void initTranslationTransform(float x, float y, float z);
    void initCameraTransform(const Vector3f& target, const Vector3f& up);
    void initPersProjTransform(float fov, float width, float height, float zNear, float zFar);

    Matrix4f operator*(const Matrix4f& right) const;
    const float* operator[](int index) const;
    float* operator[](int index);

    static Matrix4f createTranslationTransform(float x, float y, float z);

protected:
private:
    float m[4][4];
};

#endif
```

Header/medkit.h

```
#ifndef MEDKIT_H
#define MEDKIT_H

#include "material.h"
#include "mesh.h"
#include "player.h"
#include "shader.h"
#include "transform.h"
#include "util.h"
#include "vector3f.h"

class Medkit
{
public:
    static constexpr float PICKUP_DISTANCE = 0.75f;
    static constexpr int HEAL_AMOUNT = 25;

    static constexpr float SCALE = 0.25f;
    static constexpr float SIZEY = SCALE;
    static constexpr float SIZEX =
        static_cast<float>(SIZEY / (0.67857142857142857142857142857143 * 2.5));
    static constexpr float START = 0;

    static constexpr float OFFSET_X = 0.0f;
    static constexpr float OFFSET_Y = 0.0f;

    static constexpr float TEX_MIN_X = -OFFSET_X;
    static constexpr float TEX_MAX_X = -1.0f - OFFSET_X;
    static constexpr float TEX_MIN_Y = -OFFSET_Y;
    static constexpr float TEX_MAX_Y = 1.0f - OFFSET_Y;

    explicit Medkit(const Vector3f& position);

    void update();
    void render();

private:
    static Mesh* m_mesh;
    static Material* m_material;

    Transform transform;
};

#endif // MEDKIT_H
```

Header/mesh.h

```
#ifndef MESH_H
#define MESH_H

#include "vertex.h"

class Mesh
{
public:
    Mesh();
    ~Mesh();

    void addVertices(Vertex* vertices, int vertSize, int* indices, int indexSize,
                    bool calcNormals = true);
    void draw();

protected:
private:
    void calcNormals(Vertex* vertices, int vertSize, int* indices, int indexSize);

    unsigned int m_vao; // VAO member
    unsigned int m_vbo;
    unsigned int m_ibo;
    int m_size;
};

#endif
```

Header/monster.h

```
#ifndef MONSTER_H
#define MONSTER_H

#include "custom_time.h"
#include "material.h"
#include "matrix4f.h"
#include "mesh.h"
#include "texture.h"
#include "transform.h"
#include "util.h"
#include "vector2f.h"
#include "vector3f.h"

#include <random>
#include <vector>

class Monster
{
public:
    Monster(const Transform& transform);

    Transform& getTransform();
    Vector2f getSize() const;
    void damage(int amount);
    void update();
    void render();

protected:
private:
    void idleUpdate(const Vector3f& orientation, float distance);
    void chaseUpdate(const Vector3f& orientation, float distance);
    void attackUpdate(const Vector3f& orientation, float distance);
    void dyingUpdate(const Vector3f& orientation, float distance);
    void deadUpdate(const Vector3f& orientation, float distance);
    void alignWithGround();
    void faceCamera(const Vector3f& directionToCamera);

    static Mesh* m_mesh;
    static std::vector<Texture*> m_animations;

    Material m_material;
    Transform m_transform;
    std::mt19937 m_rand;
    int state;
    int health;
    bool canLook;
    bool canAttack;
    double deathTime;
};

#endif // !MONSTER_H
```

Header/phongShader.h

```
#ifndef PHONGSHADER_H
#define PHONGSHADER_H

#include "lighting.h"
#include "shader.h"

class PhongShader : public Shader
{
public:
    static PhongShader* getInstance()
    {
        static PhongShader instance;
        return &instance;
    }

    virtual void updateUniforms(const Matrix4f& worldMatrix,
                               const Matrix4f& projectedMatrix,
                               Material& material);

    static Vector3f& getAmbientLight();
    static DirectionalLight& getDirectionalLight();

    static void setAmbientLight(const Vector3f& ambientLight);
    static void setDirectionalLight(const DirectionalLight& directionalLight);
    static void setPointLights(PointLight* pointLights, int arraySize);
    static void setSpotLights(SpotLight* spotLights, int arraySize);

protected:
private:
    static const int MAX_POINT_LIGHTS = 4;
    static const int MAX_SPOT_LIGHTS = 4;
    static Vector3f m_ambientLight;
    static DirectionalLight m_directionalLight;
    static PointLight* m_pointLights;
    static SpotLight* m_spotLights;
    static int m_numPointLights;
    static int m_numSpotLights;

    PhongShader();
    ~PhongShader();

    PhongShader(PhongShader const&)
    {
    } // Don't Implement
    void operator=(PhongShader const&)
    {
    } // Don't implement
};

#endif
```

Header/player.h

```
#ifndef PLAYER_H
#define PLAYER_H

#include "camera.h"
#include "custom_time.h"
#include "input.h"
#include "material.h"
#include "mesh.h"
#include "transform.h"
#include "util.h"
#include "vector2f.h"
#include "vector3f.h"
#include "window.h"
#include <random>

class Player
{
public:
    static constexpr float PLAYER_SIZE = 0.2f;

    Player(const Vector3f& pos);

    void input();
    void update();
    void render();

    void damage(int amount);
    int getDamage();
    int getHealth();
    int getMaxHealth();

    Camera& getCamera();

    Player(Player&& other) noexcept;
    Player& operator=(Player&& other) noexcept;

protected:
private:
    static Mesh* m_mesh;
    static Material* gunMaterial;
    static bool mouseLocked;

    std::mt19937 rand;
    std::random_device rd;

    bool isFiring;
    float gunFireTimer;
    int m_health;
    Transform gunTransform;
    Camera m_camera;
    Vector2f centerPosition;
    Vector3f movementVector;
};

#endif // !PLAYER_H
```

Header/quaternion.h

```
#ifndef QUATERNION_H
#define QUATERNION_H

#include "vector3f.h"

class Quaternion
{
public:
    float x, y, z, w;

    Quaternion(float _x, float _y, float _z, float _w);

    void normalize();

    Quaternion conjugate();

    Quaternion operator*(const Quaternion& r);
    Quaternion operator*(const Vector3f& v);
};

#endif
```

Header/renderUtil.h

```
#ifndef RENDERUTIL_H
#define RENDERUTIL_H

class RenderUtil
{
public:
    static void clearScreen();
    static void initGraphics();

protected:
private:
};

#endif // !RENDERUTIL_H
```


Header/sdl_requests.h

```
#ifndef SDLREQUESTS_H
#define SDLREQUESTS_H

bool SDLGetIsCloseRequested();
void SDLSetIsCloseRequested(bool value);
void SDLCreateWindow(const char* title, int x, int y, int width, int height, bool fullscreen);
void SDLSetWindowFullscreen(bool value);
void SDLSwapBuffers();
void SDLDestroyWindow();
void SDLSetMousePosition(int x, int y);

#endif // !SDLREQUESTS_H
```

Header/shader.h

```
#ifndef SHADER_H
#define SHADER_H

#include <string>
#include <unordered_map>
#include <vector>

#include "material.h"
#include "matrix4f.h"
#include "util.h"
#include "vector3f.h"

class Shader
{
public:
    Shader();
    ~Shader();

    void bind();
    void addUniform(const std::string& uniform);
    void addVertexShaderFromFile(const std::string& text);
    void addGeometryShaderFromFile(const std::string& text);
    void addFragmentShaderFromFile(const std::string& text);
    void addVertexShader(const std::string& text);
    void addGeometryShader(const std::string& text);
    void addFragmentShader(const std::string& text);
    void compileShader();

    virtual void updateUniforms(const Matrix4f& worldMatrix, const Matrix4f& projectedMatrix,
                               Material& material);

    void setUniformi(const std::string& name, int value);
    void setUniformf(const std::string& name, float value);
    void setUniform(const std::string& name, const Vector3f& value);
    void setUniform(const std::string& name, const Matrix4f& value);

protected:
private:
    int m_program;
    std::unordered_map<std::string, int> m_uniforms;
    std::vector<int> m_shaders;

    void addProgram(const std::string& text, int type);
};

#endif
```

Header/stb_image.h

```
//// begin header file //////////////////////////////////////
//
// Limitations:
// - no jpeg progressive support
// - non-HDR formats support 8-bit samples only (jpeg, png)
// - no delayed line count (jpeg) -- IJG doesn't support either
// - no 1-bit BMP
// - GIF always returns *comp=4
//
// Basic usage (see HDR discussion below):
// int x,y,n;
// unsigned char *data = stbi_load(filename, &x, &y, &n, 0);
// // ... process data if not NULL ...
// // ... x = width, y = height, n = # 8-bit components per pixel ...
// // ... replace '0' with '1'..'4' to force that many components per pixel
// // ... but 'n' will always be the number that it would have been if you
// // said 0 stbi_image_free(data)
//
// Standard parameters:
// int *x -- outputs image width in pixels
// int *y -- outputs image height in pixels
// int *comp -- outputs # of image components in image file
// int req_comp -- if non-zero, # of image components requested in result
//
// The return value from an image loader is an 'unsigned char *' which points
// to the pixel data. The pixel data consists of *y scanlines of *x pixels,
// with each pixel consisting of N interleaved 8-bit components; the first
// pixel pointed to is top-left-most in the image. There is no padding between
// image scanlines or between pixels, regardless of format. The number of
// components N is 'req_comp' if req_comp is non-zero, or *comp otherwise.
// If req_comp is non-zero, *comp has the number of components that _would_
// have been output otherwise. E.g. if you set req_comp to 4, you will always
// get RGBA output, but you can check *comp to easily see if it's opaque.
//
// An output image with N components has the following components interleaved
// in this order in each pixel:
//
// N=#comp components
// 1 grey
// 2 grey, alpha
// 3 red, green, blue
// 4 red, green, blue, alpha
//
// If image loading fails for any reason, the return value will be NULL,
// and *x, *y, *comp will be unchanged. The function stbi_failure_reason()
// can be queried for an extremely brief, end-user unfriendly explanation
// of why the load failed. Define STBI_NO_FAILURE_STRINGS to avoid
// compiling these strings at all, and STBI_FAILURE_USERMSG to get slightly
// more user-friendly ones.
//
// Paletted PNG, BMP, GIF, and PIC images are automatically depalettized.
//
// =====
// iPhone PNG support:
//
// By default we convert iphone-formatted PNGs back to RGB; nominally they
// would silently load as BGR, except the existing code should have just
// failed on such iPhone PNGs. But you can disable this conversion by
// by calling stbi_convert_iphone_png_to_rgb(0), in which case
// you will always just get the native iphone "format" through.
//
// Call stbi_set_unpremultiply_on_load(1) as well to force a divide per
// pixel to remove any premultiplied alpha *only* if the image file explicitly
// says there's premultiplied data (currently only happens in iPhone images,
// and only if iPhone convert-to-rgb processing is on).
//
// =====
// HDR image support (disable by defining STBI_NO_HDR)
//
// stb_image now supports loading HDR images in general, and currently
// the Radiance .HDR file format, although the support is provided
// generically. You can still load any file through the existing interface;
```

```

// if you attempt to load an HDR file, it will be automatically remapped to
// LDR, assuming gamma 2.2 and an arbitrary scale factor defaulting to 1;
// both of these constants can be reconfigured through this interface:
//
//     stbi_hdr_to_ldr_gamma(2.2f);
//     stbi_hdr_to_ldr_scale(1.0f);
//
// (note, do not use _inverse_ constants; stbi_image will invert them
// appropriately).
//
// Additionally, there is a new, parallel interface for loading files as
// (linear) floats to preserve the full dynamic range:
//
//     float *data = stbi_loadf(filename, &x, &y, &n, 0);
//
// If you load LDR images through this interface, those images will
// be promoted to floating point values, run through the inverse of
// constants corresponding to the above:
//
//     stbi_ldr_to_hdr_scale(1.0f);
//     stbi_ldr_to_hdr_gamma(2.2f);
//
// Finally, given a filename (or an open file or memory block--see header
// file for details) containing image data, you can query for the "most
// appropriate" interface to use (that is, whether the image is HDR or
// not), using:
//
//     stbi_is_hdr(char *filename);
//
// =====
//
// I/O callbacks
//
// I/O callbacks allow you to read from arbitrary sources, like packaged
// files or some other source. Data read from callbacks are processed
// through a small internal buffer (currently 128 bytes) to try to reduce
// overhead.
//
// The three functions you must define are "read" (reads some bytes of data),
// "skip" (skips some bytes of data), "eof" (reports if the stream is at the
// end).

#ifndef STBI_NO_STDIO

#if defined(_MSC_VER) && _MSC_VER >= 0x1400
#define _CRT_SECURE_NO_WARNINGS // suppress bogus warnings about fopen()
#endif

#include <stdio.h>
#endif

#define STBI_VERSION 1

enum
{
    STBI_default = 0, // only used for req_comp

    STBI_grey = 1,
    STBI_grey_alpha = 2,
    STBI_rgb = 3,
    STBI_rgb_alpha = 4
};

typedef unsigned char stbi_uc;

#ifdef __cplusplus
extern "C"
{
#endif

    //////////////////////////////////////
    //
    // PRIMARY API - works on images of any type
    //
    //
    //
    // load image by filename, open file, or memory buffer

```

```

//

extern stbi_uc* stbi_load_from_memory(stbi_uc const* buffer, int len, int* x,
                                     int* y, int* comp, int req_comp);

#ifdef STBI_NO_STDIO
extern stbi_uc* stbi_load(char const* filename, int* x, int* y, int* comp,
                          int req_comp);
extern stbi_uc* stbi_load_from_file(FILE* f, int* x, int* y, int* comp,
                                    int req_comp);
// for stbi_load_from_file, file pointer is left pointing immediately after
// image
#endif

typedef struct
{
    int (*read)(void* user, char* data,
               int size); // fill 'data' with 'size' bytes. return number of
                          // bytes actually read
    void (*skip)(void* user, unsigned n); // skip the next 'n' bytes
    int (*eof)(void* user); // returns nonzero if we are at end of file/data
} stbi_io_callbacks;

extern stbi_uc* stbi_load_from_callbacks(stbi_io_callbacks const* clbk,
                                       void* user, int* x, int* y,
                                       int* comp, int req_comp);

#ifdef STBI_NO_HDR
extern float* stbi_loadf_from_memory(stbi_uc const* buffer, int len, int* x,
                                    int* y, int* comp, int req_comp);

#ifdef STBI_NO_STDIO
extern float* stbi_loadf(char const* filename, int* x, int* y, int* comp,
                         int req_comp);
extern float* stbi_loadf_from_file(FILE* f, int* x, int* y, int* comp,
                                   int req_comp);
#endif

extern float* stbi_loadf_from_callbacks(stbi_io_callbacks const* clbk,
                                       void* user, int* x, int* y, int* comp,
                                       int req_comp);

extern void stbi_hdr_to_ldr_gamma(float gamma);
extern void stbi_hdr_to_ldr_scale(float scale);

extern void stbi_ldr_to_hdr_gamma(float gamma);
extern void stbi_ldr_to_hdr_scale(float scale);
#endif // STBI_NO_HDR

// stbi_is_hdr is always defined
extern int stbi_is_hdr_from_callbacks(stbi_io_callbacks const* clbk,
                                     void* user);
extern int stbi_is_hdr_from_memory(stbi_uc const* buffer, int len);
#ifdef STBI_NO_STDIO
extern int stbi_is_hdr(char const* filename);
extern int stbi_is_hdr_from_file(FILE* f);
#endif // STBI_NO_STDIO

// get a VERY brief reason for failure
// NOT THREADSAFE
extern const char* stbi_failure_reason(void);

// free the loaded image -- this is just free()
extern void stbi_image_free(void* retval_from_stbi_load);

// get image dimensions & components without fully decoding
extern int stbi_info_from_memory(stbi_uc const* buffer, int len, int* x,
                                int* y, int* comp);
extern int stbi_info_from_callbacks(stbi_io_callbacks const* clbk, void* user,
                                   int* x, int* y, int* comp);

#ifdef STBI_NO_STDIO
extern int stbi_info(char const* filename, int* x, int* y, int* comp);
extern int stbi_info_from_file(FILE* f, int* x, int* y, int* comp);
#endif

#endif
// for image formats that explicitly notate that they have premultiplied

```

```

// alpha, we just return the colors as stored in the file. set this flag to
// force unpremultiplication. results are undefined if the unpremultiply
// overflow.
extern void
stbi_set_unpremultiply_on_load(int flag_true_if_should_unpremultiply);

// indicate whether we should process iphone images back to canonical format,
// or just pass them through "as-is"
extern void stbi_convert_iphone_png_to_rgb(int flag_true_if_should_convert);

// ZLIB client - used by PNG, available for other purposes
extern char* stbi_zlib_decode_malloc_guesssize(const char* buffer, int len,
                                              int initial_size, int* outlen);
extern char* stbi_zlib_decode_malloc(const char* buffer, int len,
                                      int* outlen);
extern int stbi_zlib_decode_buffer(char* obuffer, int olen,
                                   const char* ibuffer, int ilen);

extern char* stbi_zlib_decode_noheader_malloc(const char* buffer, int len,
                                              int* outlen);
extern int stbi_zlib_decode_noheader_buffer(char* obuffer, int olen,
                                             const char* ibuffer, int ilen);

// define faster low-level operations (typically SIMD support)
#ifdef STBI_SIMD
typedef void (*stbi_idct_8x8)(stbi_uc* out, int out_stride, short data[64],
                             unsigned short* dequantize);
// compute an integer IDCT on "input"
//   input[x] = data[x] * dequantize[x]
//   write results to 'out': 64 samples, each run of 8 spaced by
//   'out_stride'
//
//   CLAMP results to 0..255
typedef void (*stbi_YCbCr_to_RGB_run)(stbi_uc* output, stbi_uc const* y,
                                      stbi_uc const* cb, stbi_uc const* cr,
                                      int count, int step);

// compute a conversion from YCbCr to RGB
//   'count' pixels
//   write pixels to 'output'; each pixel is 'step' bytes (either 3 or 4; if
//   4, write '255' as 4th), order R,G,B y: Y input channel cb: Cb input
//   channel; scale/biased to be 0..255 cr: Cr input channel; scale/biased
//   to be 0..255

extern void stbi_install_idct(stbi_idct_8x8 func);
extern void stbi_install_YCbCr_to_RGB(stbi_YCbCr_to_RGB_run func);
#endif // STBI_SIMD

#ifdef __cplusplus
}
#endif

//
//
//// end header file //////////////////////////////////////

```

Header/texture.h

```
#ifndef TEXTURE_H
#define TEXTURE_H

#include "GL/glew.h"
#include <string>

class Texture
{
public:
    Texture(const std::string& fileName, GLenum textureTarget = GL_TEXTURE_2D,
            GLfloat filter = GL_NEAREST);
    Texture(int width = 0, int height = 0, unsigned char* data = 0,
            GLenum textureTarget = GL_TEXTURE_2D, GLfloat filter = GL_NEAREST);
    ~Texture();

    void bind(GLenum textureUnit = GL_TEXTURE0);
    Texture(Texture& texture);

    GLuint getID();

    Texture& operator=(Texture&& other);
    void operator=(Texture& texture);

protected:
private:
    static Texture* m_lastBind;

    GLenum m_textureTarget;
    GLuint m_textureID;
    bool m_freeTexture;

    void initTexture(int width, int height, unsigned char* data, GLenum textureTarget,
                    GLfloat filter);
};

#endif
```

Header/transform.h

```
#ifndef TRANSFORM_H
#define TRANSFORM_H

#include "camera.h"
#include "matrix4f.h"
#include "vector3f.h"

class Transform
{
public:
    static Camera& getCamera();
    static void setCamera(Camera& camera);
    static void setProjection(float fov, float width, float height, float zNear, float zFar);

    Transform(Vector3f pos = Vector3f(0, 0, 0), Vector3f rot = Vector3f(0, 0, 0),
              Vector3f scale = Vector3f(1, 1, 1));
    ~Transform();

    Matrix4f getTransformation();
    Matrix4f getProjectedTransformation();

    const Vector3f& getPos() const;
    const Vector3f& getRot() const;
    const Vector3f& getScale() const;

    Vector3f& getPos();
    Vector3f& getRot();
    Vector3f& getScale();

    void setPos(const Vector3f& pos);
    void setRot(const Vector3f& rot);
    void setScale(const Vector3f& scale);

protected:
private:
    static Camera* m_camera;

    static float m_fov;
    static float m_width;
    static float m_height;
    static float m_zNear;
    static float m_zFar;

    Vector3f m_pos;
    Vector3f m_rot;
    Vector3f m_scale;
};

#endif
```


Header/util.h

```
#ifndef UTIL_H
#define UTIL_H

#ifdef WIN32
#include <unistd.h>
#endif

#ifdef WIN32
#define SNPRINTF _snprintf_s
#else
#define SNPRINTF snprintf
#endif

#define ZERO_MEM(a) memset(a, 0, sizeof(a))
#define ARRAY_SIZE(a) (sizeof(a) / sizeof(a[0]))
#define SAFE_DELETE(p)
    if (p)
    {
        delete p;
        p = NULL;
    }
#define INVALID_VALUE 0xFFFFFFFF

#define ToRadian(x) (float)(((x) * 3.1415926536f / 180.0f))
#define ToDegree(x) (float)(((x) * 180.0f / 3.1415926536f))

#include <SDL.h>

namespace Util
{
    inline void sleep(int milliseconds)
    {
        SDL_Delay(milliseconds);
    }
}; // namespace Util

#endif
```

Header/vector2f.h

```
#ifndef VECTOR2F_H
#define VECTOR2F_H

class Vector2f
{
public:
    float x;
    float y;

    Vector2f(float x = 0.0f, float y = 0.0f)
    {
        this->x = x;
        this->y = y;
    }

    float getX() const;
    float getY() const;

    float length() const;

    Vector2f rotate(float angle);

    void setX(float value);
    void setY(float value);

    Vector2f& normalize();

    Vector2f& operator+=(const Vector2f& r);
    Vector2f& operator-=(const Vector2f& r);
    Vector2f& operator*=(float f);
    Vector2f& operator/=(float f);

    Vector2f operator+(const Vector2f& r) const;
    Vector2f operator-(const Vector2f& r) const;
    Vector2f operator*(float f) const;
    Vector2f operator/(float f) const;

    Vector2f mult(const Vector2f& other);

    bool operator==(const Vector2f& r) const;
    bool operator!=(const Vector2f& r) const;

protected:
private:
};

#endif
```

Header/vector3f.h

```
#ifndef VECTOR3F_H
#define VECTOR3F_H

class Vector3f
{
public:
    static const Vector3f UP;
    static const Vector3f DOWN;
    static const Vector3f LEFT;
    static const Vector3f RIGHT;
    static const Vector3f FORWARD;
    static const Vector3f BACK;
    static const Vector3f ONE;
    static const Vector3f ZERO;

    float x, y, z;

    Vector3f(float x = 0.0f, float y = 0.0f, float z = 0.0f);

    float length() const;
    float dot(const Vector3f& v) const;

    Vector3f cross(const Vector3f& v) const;
    Vector3f rotate(float angle, const Vector3f& axis) const;

    Vector3f& normalize();

    void setX(float value);
    void setY(float value);
    void setZ(float value);

    float getX() const;
    float getY() const;
    float getZ() const;

    Vector3f& operator+=(const Vector3f& r);
    Vector3f& operator-=(const Vector3f& r);
    Vector3f& operator*=(float f);
    Vector3f& operator/=(float f);

    Vector3f operator+(const Vector3f& r) const;
    Vector3f operator-(const Vector3f& r) const;
    Vector3f operator*(float f) const;
    Vector3f operator/(float f) const;

    Vector3f mult(const Vector3f& r);

    inline bool operator==(const Vector3f& r) const;
    inline bool operator!=(const Vector3f& r) const;
};

#endif
```

Header/vertex.h

```
#ifndef VERTEX_H
#define VERTEX_H

#include "vector2f.h"
#include "vector3f.h"

struct Vertex
{
    Vector3f pos;
    Vector2f texCoord;
    Vector3f normal;

    Vertex()
    {
    }

    Vertex(Vector3f pos, Vector2f texCoord = Vector2f(0, 0),
           Vector3f normal = Vector3f(0, 0, 0))
    {
        this->pos = pos;
        this->texCoord = texCoord;
        this->normal = normal;
    }
};

#endif
```

Header/window.h

```
#ifndef GAMEWINDOW_H
#define GAMEWINDOW_H

#include <string>

class Window
{
public:
    static constexpr int PLAYER_FIRE_CHANNEL = 0;
    static constexpr int PLAYER_WALK_CHANNEL = 1;
    static constexpr int PLAYER_HEALTH_CHANNEL = 2;
    static constexpr int PLAYER_DEATH_CHANNEL = 3;
    static constexpr int MONSTER_FIRE_CHANNEL = 4;
    static constexpr int MONSTER_HEALTH_CHANNEL = 5;
    static constexpr int DOOR_OPEN_CHANNEL = 6;
    static constexpr int DOOR_CLOSE_CHANNEL = 7;

    static void create(int width, int height, const std::string& title);
    static void render();
    static void dispose();

    static bool isCloseRequested();
    static int getWidth();
    static int getHeight();
    static const std::string& getTitle();
    static void setFullScreen(bool value);

    static void playSound(const std::string& fileName, int channel = -1, float volume = 1.0f);

protected:
private:
    static int s_width;
    static int s_height;
    static std::string s_title;
};

#endif
```

Sources/basicShader.cpp

```
#include "basicShader.h"

BasicShader::BasicShader()
{
    addVertexShaderFromFile("basicVertex.vs");
    addFragmentShaderFromFile("basicFragment.fs");
    compileShader();

    addUniform("transform");
    addUniform("color");
}

BasicShader::~BasicShader()
{
}

static unsigned char whitePixel[] = {0xFF, 0xFF, 0xFF, 0xFF};

void BasicShader::updateUniforms(const Matrix4f& worldMatrix,
                                const Matrix4f& projectedMatrix,
                                Material& material)
{
    static Texture WHITE = Texture(1, 1, whitePixel);

    if (material.texture != NULL)
        material.texture->bind();
    else
        WHITE.bind();

    setUniform("transform", projectedMatrix);
    setUniform("color", material.color);
}
```

Sources/bitmap.cpp

```
#include "bitmap.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_image.h>
#include <SDL_surface.h>
#include <iostream>
#include <sys/types.h>

Bitmap::Bitmap(int width, int height)
{
    m_width = width;
    m_height = height;
    m_pixels.resize(m_width * m_height);
}

Bitmap::Bitmap(const std::string& fileName)
{
    SDL_Surface* image =
        IMG_Load((std::string(RESOURCE_PATH) + "/bitmaps/" + fileName).c_str());
    if (!image)
    {
        std::cerr << "Failed to load image : " << fileName << "\n"
                    << IMG_GetError() << std::endl;
    }
    m_width = image->w;
    m_height = image->h;
    m_pixels.resize(m_width * m_height);

    int* pixels = static_cast<int*>(image->pixels);
    for (int i = 0; i < m_width * m_height; i++)
    {
        m_pixels[i] = pixels[i];
    }
    SDL_FreeSurface(image);
}

Bitmap::Bitmap(const Bitmap& other)
{
    m_width = other.m_width;
    m_height = other.m_height;
    m_pixels = other.m_pixels;
}

Bitmap& Bitmap::flipX()
{
    std::vector<int> temp = m_pixels;

    for (int i = 0; i < m_width; ++i)
    {
        for (int j = 0; j < m_height; ++j)
        {
            temp[i + j * m_width] = m_pixels[(m_width - i - 1) + j * m_width];
        }
    }

    m_pixels = temp;
    return *this;
}

Bitmap& Bitmap::flipY()
{
    std::vector<int> temp = m_pixels;

    for (int i = 0; i < m_width; ++i)
    {
        for (int j = 0; j < m_height; ++j)
        {
            temp[i + j * m_width] = m_pixels[i + (m_height - j - 1) * m_width];
        }
    }

    m_pixels = temp;
    return *this;
}
```

```

Bitmap& Bitmap::operator=(const Bitmap& other)
{
    if (this == &other)
        return *this;

    m_width = other.m_width;
    m_height = other.m_height;
    m_pixels = other.m_pixels;

    return *this;
}

Bitmap& Bitmap::operator=(Bitmap* other)
{
    if (this == other)
        return *this;
    if (other)
    {
        m_width = other->m_width;
        m_height = other->m_height;
        m_pixels = other->m_pixels;
    }

    return *this;
}

int Bitmap::getWidth()
{
    return m_width;
}

int Bitmap::getHeight()
{
    return m_height;
}

const std::vector<int> Bitmap::getPixels() const
{
    return m_pixels;
}

int Bitmap::getPixel(int x, int y) const
{
    return m_pixels[x + y * m_width];
}

void Bitmap::setPixel(int x, int y, int value)
{
    m_pixels[x + y * m_width] = value;
}

```


Sources/camera.cpp

```
#include "camera.h"

Camera::Camera(Vector3f pos, Vector3f forward, Vector3f up)
{
    m_pos = pos;
    m_forward = forward;
    m_up = up;
}

Camera::~Camera()
{
}

static bool mouseLocked = false;

void Camera::input()
{
    float sensitivity = 0.5f;
    float movAmt = (float)(10 * Time::getDelta());
    float rotAmt = (float)(100 * Time::getDelta());

    if (Input::getKey(KEY::KEY_ESCAPE))
    {
        Input::setCursor(true);
        mouseLocked = false;
    }

    if (mouseLocked)
    {
        Vector2f centerPosition = Vector2f((float)Window::getWidth() / 2.0f,
                                            (float)Window::getHeight() / 2.0f);
        Vector2f deltaPos = Input::getMousePosition() - centerPosition;

        bool rotY = deltaPos.x != 0;
        bool rotX = deltaPos.y != 0;

        if (rotY)
            rotateY(deltaPos.x * sensitivity);
        if (rotX)
            rotateX(deltaPos.y * sensitivity);

        if (rotY || rotX)
            Input::setMousePosition(centerPosition);
    }

    if (Input::getMouseDown(MOUSE::LEFT_MOUSE))
    {
        Vector2f centerPosition = Vector2f((float)Window::getWidth() / 2.0f,
                                            (float)Window::getHeight() / 2.0f);
        Input::setMousePosition(centerPosition);
        Input::setCursor(false);
        mouseLocked = true;
    }

    if (Input::getKey(KEY::KEY_W))
        move(getForward(), movAmt);
    if (Input::getKey(KEY::KEY_S))
        move(getForward(), -movAmt);
    if (Input::getKey(KEY::KEY_A))
        move(getLeft(), movAmt);
    if (Input::getKey(KEY::KEY_D))
        move(getRight(), movAmt);
}

void Camera::move(const Vector3f& direction, float amt)
{
    m_pos += (direction * amt);
}

void Camera::rotateY(float angle)
{
    Vector3f hAxis = Vector3f::UP.cross(m_forward).normalize();

    m_forward = m_forward.rotate(angle, Vector3f::UP).normalize();
}
```

```

    m_up = m_forward.cross(hAxis).normalize();
}

void Camera::rotateX(float angle)
{
    Vector3f hAxis = Vector3f::UP.cross(m_forward).normalize();

    m_forward = m_forward.rotate(angle, hAxis).normalize();
    m_up = m_forward.cross(hAxis).normalize();
}

Vector3f& Camera::getPos()
{
    return m_pos;
}

Vector3f& Camera::getForward()
{
    return m_forward;
}

Vector3f& Camera::getUp()
{
    return m_up;
}

Vector3f Camera::getLeft() const
{
    return m_forward.cross(m_up).normalize();
}

Vector3f Camera::getRight() const
{
    return m_up.cross(m_forward).normalize();
}

void Camera::setPos(const Vector3f& pos)
{
    m_pos = pos;
}

void Camera::setForward(const Vector3f& forward)
{
    m_forward = forward;
}

void Camera::setUp(const Vector3f& up)
{
    m_up = up;
}

```

Sources/custom_time.cpp

```
#include "custom_time.h"

#include "time.h"
#include <chrono>

std::chrono::high_resolution_clock::time_point Time::m_epoch =
    std::chrono::high_resolution_clock::now();
double Time::m_delta = 0.0;

double Time::getTime()
{
    return std::chrono::duration_cast<std::chrono::nanoseconds>(
        std::chrono::high_resolution_clock::now() - m_epoch)
        .count() /
        1000000000.0;
}

double Time::getDelta()
{
    return m_delta;
}

void Time::setDelta(double value)
{
    Time::m_delta = value;
}
```

Sources/door.cpp

```
#include "door.h"
#include "game.h"

static const float LENGTH = 1;
static const float HEIGHT = 1;
static const float WIDTH = 0.125f;
static const float START = 0;
static const double TIME_TO_OPEN = 2.5f;
static const double CLOSE_DELAY = 2.0f;

Mesh* Door::m_mesh = nullptr;

Door::Door(Transform transform, Material* material, Vector3f openPosition)
    : m_transform(transform), m_material(material), isOpening(false),
      m_closePosition(m_transform.getPos()), m_openPosition(openPosition)
{
    if (!m_mesh)
    {
        Vertex vertices[] = {
            Vertex(Vector3f(START, START, START), Vector2f(0.5f, 1.0f)),
            Vertex(Vector3f(START, HEIGHT, START), Vector2f(0.5f, 0.75f)),
            Vertex(Vector3f(LENGTH, HEIGHT, START), Vector2f(0.75f, 0.75f)),
            Vertex(Vector3f(LENGTH, START, START), Vector2f(0.75f, 1.0f)),

            Vertex(Vector3f(START, START, START), Vector2f(0.73f, 1.0f)),
            Vertex(Vector3f(START, HEIGHT, START), Vector2f(0.73f, 0.75f)),
            Vertex(Vector3f(START, HEIGHT, WIDTH), Vector2f(0.75f, 0.75f)),
            Vertex(Vector3f(START, START, WIDTH), Vector2f(0.75f, 1.0f)),

            Vertex(Vector3f(START, START, WIDTH), Vector2f(0.5f, 1.0f)),
            Vertex(Vector3f(START, HEIGHT, WIDTH), Vector2f(0.5f, 0.75f)),
            Vertex(Vector3f(LENGTH, HEIGHT, WIDTH), Vector2f(0.75f, 0.75f)),
            Vertex(Vector3f(LENGTH, START, WIDTH), Vector2f(0.75f, 1.0f)),

            Vertex(Vector3f(LENGTH, START, START), Vector2f(0.73f, 1.0f)),
            Vertex(Vector3f(LENGTH, HEIGHT, START), Vector2f(0.73f, 0.75f)),
            Vertex(Vector3f(LENGTH, HEIGHT, WIDTH), Vector2f(0.75f, 0.75f)),
            Vertex(Vector3f(LENGTH, START, WIDTH), Vector2f(0.75f, 1.0f)),
        };

        int indices[] = {0, 1, 2, 0, 2, 3, 6, 5, 4, 7, 6, 4,
                        10, 9, 8, 11, 10, 8, 12, 13, 14, 12, 14, 15};

        m_mesh = new Mesh();
        m_mesh->addVertices(vertices, ARRAY_SIZE(vertices), indices,
                           ARRAY_SIZE(indices));
    }
}

void Door::open()
{
    if (isOpening)
        return;

    openingStartTime = (double)Time::getTime();

    openTime = openingStartTime + TIME_TO_OPEN;
    closingStartTime = openTime + CLOSE_DELAY;
    closeTime = closingStartTime + TIME_TO_OPEN;

    isOpening = true;
}

void Door::update()
{
    if (isOpening)
    {
        double time = (double)Time::getTime();
        if (time < openTime)
        {
            Window::playSound("Door.wav", Window::DOOR_OPEN_CHANNEL, 0.45f);
            getTransform().setPos(
                VectorLerp(m_closePosition, m_openPosition,
                           (float)((time - openingStartTime) / TIME_TO_OPEN)));
        }
    }
}
```

```

    }
    else if (time < closingStartTime)
    {
        getTransform().setPos(m_openPosition);
    }
    else if (time < closeTime)
    {
        getTransform().setPos(
            VectorLerp(m_openPosition, m_closePosition,
                (float)((time - closingStartTime) / TIME_TO_OPEN));
        Window::playSound("Door.wav", Window::DOOR_CLOSE_CHANNEL, 0.45f);
    }
    else
    {
        getTransform().setPos(m_closePosition);
        isOpening = false;
    }
}

void Door::render()
{
    Shader* m_shader = Game::getLevel()->getShader();
    m_shader->updateUniforms(m_transform.getTransformation(),
        m_transform.getProjectedTransformation(),
        *m_material);

    m_mesh->draw();
}

Vector2f Door::getDoorSize() const
{
    if (m_transform.getRot().getY() == 90)
        return Vector2f(WIDTH, LENGTH);
    else
        return Vector2f(LENGTH, WIDTH);
}

Transform& Door::getTransform()
{
    return m_transform;
}

Vector3f Door::VectorLerp(const Vector3f& startPos, const Vector3f& endPos,
    float lerpFactor)
{
    return startPos + ((endPos - startPos) * lerpFactor);
}

```

Sources/game.cpp

```
#include "game.h"
#include "window.h"

#include <math.h>
double temp = 0.0f;

Level* Game::m_level = nullptr;
bool Game::isRunning = false;
int Game::levelNum = 0;

Game::Game()
{
    loadNextLevel();
}

void Game::input()
{
    if (m_level)
        m_level->input();
}

void Game::update()
{
    if (isRunning && m_level)
        m_level->update();
}

void Game::render()
{
    if (isRunning && m_level)
        m_level->render();
}

void Game::loadNextLevel()
{
    levelNum++;
    std::string levelFile = "level" + std::to_string(levelNum) + ".png";
    std::string collectionFile = "WolfCollection.png";

    delete m_level;
    m_level = new Level(levelFile, collectionFile);

    Transform::setProjection(70.0f, (float)Window::getWidth(),
                             (float)Window::getHeight(), 0.1f, 100.0f);
    Transform::setCamera(m_level->getPlayer()->getCamera());

    isRunning = true;
}

Level* Game::getLevel()
{
    return m_level;
}

void Game::setIsRunning(bool value)
{
    isRunning = value;
}
```

Sources/input.cpp

```
#include "input.h"
#include <SDL.h>
#include <SDL_events.h>
#include <SDL_mouse.h>

const static int NUM_KEYS = 512;
const static int NUM_MOUSEBUTTONS = 256;

static SDL_Event e;

static int mouseX = 0;
static int mouseY = 0;

static bool inputs[NUM_KEYS];
static bool downKeys[NUM_KEYS];
static bool upKeys[NUM_KEYS];

static bool mouseInput[NUM_MOUSEBUTTONS];
static bool downMouse[NUM_MOUSEBUTTONS];
static bool upMouse[NUM_MOUSEBUTTONS];

void Input::update()
{
    for (int i = 0; i < NUM_MOUSEBUTTONS; i++)
    {
        downMouse[i] = false;
        upMouse[i] = false;
    }

    for (int i = 0; i < NUM_KEYS; i++)
    {
        downKeys[i] = false;
        upKeys[i] = false;
    }

    while (SDL_PollEvent(&e))
    {
        if (e.type == SDL_QUIT)
            SDLSetIsCloseRequested(true);

        if (e.type == SDL_MOUSEMOTION)
        {
            mouseX = e.motion.x;
            mouseY = e.motion.y;
        }

        if (e.type == SDL_KEYDOWN)
        {
            int value = e.key.keysym.scancode;

            inputs[value] = true;
            downKeys[value] = true;
        }
        if (e.type == SDL_KEYUP)
        {
            int value = e.key.keysym.scancode;

            inputs[value] = false;
            upKeys[value] = true;
        }
        if (e.type == SDL_MOUSEBUTTONDOWN)
        {
            int value = e.button.button;

            mouseInput[value] = true;
            downMouse[value] = true;
        }
        if (e.type == SDL_MOUSEBUTTONUP)
        {
            int value = e.button.button;

            mouseInput[value] = false;
            upMouse[value] = true;
        }
    }
}
```

```

    }
}

bool Input::getKey(int keyCode)
{
    return inputs[keyCode];
}

bool Input::getKeyDown(int keyCode)
{
    return downKeys[keyCode];
}

bool Input::getKeyUp(int keyCode)
{
    return upKeys[keyCode];
}

bool Input::getMouse(int button)
{
    return mouseInput[button];
}

bool Input::getMouseDown(int button)
{
    return downMouse[button];
}

bool Input::getMouseUp(int button)
{
    return upMouse[button];
}

Vector2f Input::getMousePosition()
{
    Vector2f res((float)mouseX, (float)mouseY);
    return res;
}

void Input::setCursor(bool visible)
{
    if (visible)
        SDL_ShowCursor(1);
    else
        SDL_ShowCursor(0);
}

void Input::setMousePosition(Vector2f pos)
{
    SDLSetMousePosition((int)pos.x, (int)pos.y);
}

```


Sources/level.cpp

```
#include "basicShader.h"
#include "game.h"
#include "level.h"
#include "window.h"
#include <cstdlib>
#include <iostream>
#include <ostream>
#include <vector>

const static float SPOT_WIDTH = 1.0f;
const static float SPOT_LENGTH = 1.0f;
const static float SPOT_HEIGHT = 1.0f;
const static float OPEN_DISTANCE = 1.0f;
const static float DOOR_OPEN_MOVEMENT_AMOUNT = 0.9f;
const static int NUM_TEXTURE_EXP = 4;
const static int NUM_TEXTURE = (int)pow(2, NUM_TEXTURE_EXP);

Level::Level(const std::string& levelName, const std::string& textureName)
    : m_transform(), m_player(Player(Vector3f(7, 0.435, 7))), m_level(levelName)
{
    m_level.flipY();
    m_texture = Texture(textureName);
    m_material = Material(&m_texture);
    m_shader = BasicShader::getInstance();
    generateLevel();
    Window::playSound("Guten_Tag.wav");
}

void Level::openDoors(Vector3f pos, bool tryExitLevel)
{
    for (Door* door : m_doors)
    {
        if ((door->getTransform().getPos() - pos).length() < OPEN_DISTANCE)
            door->open();
    }
    if (tryExitLevel)
    {
        for (Vector3f exitPoint : exitPoints)
        {
            if ((exitPoint - pos).length() < OPEN_DISTANCE)
            {
                Game::loadNextLevel();
            }
        }
    }
}

void Level::damagePlayer(int amount)
{
    m_player.damage(amount);
}

void Level::input()
{
    m_player.input();
}

void Level::update()
{
    for (Door* door : m_doors)
        door->update();

    for (Monster* monster : m_monsters)
        monster->update();
    for (Medkit& medkit : m_medkits)
        medkit.update();
    m_medkits.erase(std::remove_if(m_medkits.begin(), m_medkits.end(),
                                   [this](const Medkit& medkit)
                                   {
                                       return std::find(m_medkitsToRemove.begin(),
                                                           m_medkitsToRemove.end(),
                                                           &medkit) !=
                                                           m_medkitsToRemove.end();
                                   })),
    },
```

```

        m_medkits.end());
    m_player.update();
}

void Level::render()
{
    m_shader->bind();
    m_shader->updateUniforms(m_transform.getTransformation(),
                           m_transform.getProjectedTransformation(),
                           m_material);

    m_mesh.draw();

    for (Door* door : m_doors)
    {
        door->render();
    }
    for (Monster* monster : m_monsters)
    {
        monster->render();
    }
    for (Medkit& medkit : m_medkits)
    {
        medkit.render();
    }
    m_player.render();
}

Player* Level::getPlayer()
{
    return &m_player;
}

Shader* Level::getShader()
{
    return m_shader;
}

void Level::addSpecial(int blueValue, int x, int y)
{
    if (blueValue == 16)
        addDoor(x, y);
    if (blueValue == 1)
        m_player = Player(
            Vector3f((x + 0.5f) * SPOT_WIDTH, 0.4375f, (y + 0.5f) * SPOT_LENGTH));
    if (blueValue == 128)
    {
        Transform monsterTransform = Transform();
        monsterTransform.setPos(
            Vector3f((x + 0.5f) * SPOT_WIDTH, 0, (y + 0.5f) * SPOT_LENGTH));
        m_monsters.push_back(new Monster(monsterTransform));
    }
    if (blueValue == 192)
        m_medkits.push_back(
            Medkit(Vector3f((x + 0.5f) * SPOT_WIDTH, 0, (y + 0.5f) * SPOT_LENGTH)));
    if (blueValue == 97)
        exitPoints.push_back(
            Vector3f((x + 0.5f) * SPOT_WIDTH, 0, (y + 0.5f) * SPOT_LENGTH));
}

void Level::generateLevel()
{
    std::vector<int> indices;
    std::vector<Vertex> vertices;

    for (int i = 0; i < m_level.getWidth(); i++)
    {
        for (int j = 0; j < m_level.getHeight(); j++)
        {
            if ((m_level.getPixel(i, j) & 0xFFFFFF) == 0)
                continue;

            addSpecial(((m_level.getPixel(i, j) & 0xFF0000) >> 16), i, j);

            std::vector<float> texCoord =
                calcTexCoords((m_level.getPixel(i, j) & 0x00FF00) >> 8);

            // Generate Floor

```

[illegible]

```

Vector2f collisionVector(1, 1);
Vector3f movementVector = newPos - oldPos;

if (movementVector.length() > 0)
{
    Vector2f blockSize(SPOT_WIDTH, SPOT_LENGTH);
    Vector2f objectSize(objWidth, objLength);

    Vector2f oldPos2(oldPos.getX(), oldPos.getZ());
    Vector2f newPos2(newPos.getX(), newPos.getZ());

    for (int i = 0; i < m_level.getWidth(); i++)
    {
        for (int j = 0; j < m_level.getHeight(); j++)
        {
            if ((m_level.getPixel(i, j) & 0xFFFFF) == 0)
            {
                collisionVector = collisionVector.mult(
                    rectCollide(oldPos2, newPos2, objectSize,
                        blockSize.mult(Vector2f(i, j)), blockSize));
            }
        }
    }
    for (Door* door : m_doors)
    {
        Vector2f doorSize = door->getDoorSize();
        Vector3f doorPos3f = door->getTransform().getPos();
        Vector2f doorPos2f = Vector2f(doorPos3f.getX(), doorPos3f.getZ());
        collisionVector = collisionVector.mult(
            rectCollide(oldPos2, newPos2, objectSize, doorPos2f, doorSize));
    }
}

return Vector3f(collisionVector.getX(), 0, collisionVector.getY());
}

Vector2f Level::checkIntersection(const Vector2f& lineStart,
                                const Vector2f& lineEnd, bool hurtMonster)
{
    Vector2f nearestIntersection = NULL;
    for (int i = 0; i < collisionPosStart.size(); i++)
    {
        Vector2f collisionVector = lineIntersect(
            lineStart, lineEnd, collisionPosStart[i], collisionPosEnd[i]);
        nearestIntersection =
            findNearestVector2f(nearestIntersection, collisionVector, lineStart);
    }
    for (Door* door : m_doors)
    {
        Vector2f doorSize = door->getDoorSize();
        Vector3f doorPos3f = door->getTransform().getPos();
        Vector2f doorPos2f = Vector2f(doorPos3f.getX(), doorPos3f.getZ());
        Vector2f collisionVector =
            lineIntersectRect(lineStart, lineEnd, doorPos2f, doorSize);

        nearestIntersection =
            findNearestVector2f(nearestIntersection, collisionVector, lineStart);
    }
    if (hurtMonster)
    {
        Vector2f nearestMonsterIntersect = NULL;
        Monster* nearestMonster = nullptr;

        for (Monster* monster : m_monsters)
        {
            Vector2f monsterSize = monster->getSize();
            Vector3f monsterPos3f = monster->getTransform().getPos();
            Vector2f monsterPos2f =
                Vector2f(monsterPos3f.getX(), monsterPos3f.getZ());
            Vector2f collisionVector =
                lineIntersectRect(lineStart, lineEnd, monsterPos2f, monsterSize);

            nearestMonsterIntersect = findNearestVector2f(nearestMonsterIntersect,
                collisionVector, lineStart);

            if (nearestMonsterIntersect == collisionVector)
                nearestMonster = monster;
        }
    }
}

```

```

    }

    if (nearestMonsterIntersect != NULL &&
        (nearestIntersection == NULL ||
         (nearestMonsterIntersect - lineStart).length() <
          (nearestIntersection - lineStart).length()))
    {
        if (nearestMonster != nullptr)
            nearestMonster->damage(m_player.getDamage());
    }
}
return nearestIntersection;
}

void Level::addFace(std::vector<int>& indices, int startLocation,
                   bool direction)
{
    if (direction)
    {
        indices.push_back(startLocation + 2);
        indices.push_back(startLocation + 1);
        indices.push_back(startLocation + 0);
        indices.push_back(startLocation + 3);
        indices.push_back(startLocation + 2);
        indices.push_back(startLocation + 0);
    }
    else
    {
        indices.push_back(startLocation + 0);
        indices.push_back(startLocation + 1);
        indices.push_back(startLocation + 2);
        indices.push_back(startLocation + 0);
        indices.push_back(startLocation + 2);
        indices.push_back(startLocation + 3);
    }
}

std::vector<float> Level::calcTexCoords(int value)
{
    int texX = value / NUM_TEXTURE;
    int texY = texX % NUM_TEXTURE_EXP;
    texX /= NUM_TEXTURE_EXP;

    std::vector<float> result(4);

    result[0] = 1.0f - (float)texX / (float)NUM_TEXTURE_EXP;
    result[1] = result[0] - 1.0f / (float)NUM_TEXTURE_EXP;
    result[3] = 1.0f - (float)texY / (float)NUM_TEXTURE_EXP;
    result[2] = result[3] - 1.0f / (float)NUM_TEXTURE_EXP;

    return result;
}

void Level::addVertices(std::vector<Vertex>& vertices, int i, int j,
                      float offset, bool x, bool y, bool z,
                      std::vector<float> texCoord)
{
    if (x && z)
    {
        vertices.push_back(
            Vertex(Vector3f(i * SPOT_WIDTH, offset * SPOT_HEIGHT, j * SPOT_LENGTH),
                  Vector2f(texCoord[1], texCoord[3])));
        vertices.push_back(Vertex(
            Vector3f((i + 1) * SPOT_WIDTH, offset * SPOT_HEIGHT, j * SPOT_LENGTH),
            Vector2f(texCoord[0], texCoord[3])));
        vertices.push_back(
            Vertex(Vector3f((i + 1) * SPOT_WIDTH, offset * SPOT_HEIGHT,
                          (j + 1) * SPOT_LENGTH),
                  Vector2f(texCoord[0], texCoord[2])));
        vertices.push_back(Vertex(
            Vector3f(i * SPOT_WIDTH, offset * SPOT_HEIGHT, (j + 1) * SPOT_LENGTH),
            Vector2f(texCoord[1], texCoord[2])));
    }
    else if (x && y)
    {
        vertices.push_back(
            Vertex(Vector3f(i * SPOT_WIDTH, j * SPOT_HEIGHT, offset * SPOT_LENGTH),

```

```

        Vector2f(texCoord[1], texCoord[3]));
vertices.push_back(Vertex(
    Vector3f((i + 1) * SPOT_WIDTH, j * SPOT_HEIGHT, offset * SPOT_LENGTH),
    Vector2f(texCoord[0], texCoord[3])));
vertices.push_back(
    Vertex(Vector3f((i + 1) * SPOT_WIDTH, (j + 1) * SPOT_HEIGHT,
        offset * SPOT_LENGTH),
        Vector2f(texCoord[0], texCoord[2])));
vertices.push_back(Vertex(
    Vector3f(i * SPOT_WIDTH, (j + 1) * SPOT_HEIGHT, offset * SPOT_LENGTH),
    Vector2f(texCoord[1], texCoord[2])));
}
else if (y && z)
{
    vertices.push_back(
        Vertex(Vector3f(offset * SPOT_WIDTH, i * SPOT_HEIGHT, j * SPOT_LENGTH),
            Vector2f(texCoord[1], texCoord[3])));
vertices.push_back(Vertex(
    Vector3f(offset * SPOT_WIDTH, i * SPOT_HEIGHT, (j + 1) * SPOT_LENGTH),
    Vector2f(texCoord[0], texCoord[3])));
vertices.push_back(
    Vertex(Vector3f(offset * SPOT_WIDTH, (i + 1) * SPOT_HEIGHT,
        (j + 1) * SPOT_LENGTH),
        Vector2f(texCoord[0], texCoord[2])));
vertices.push_back(Vertex(
    Vector3f(offset * SPOT_WIDTH, (i + 1) * SPOT_HEIGHT, j * SPOT_LENGTH),
    Vector2f(texCoord[1], texCoord[2])));
}
else
{
    std::cerr << "Invalid plane generation" << std::endl;
    exit(1);
}
}

```

```

void Level::addDoor(int x, int y)
{
    Transform doorTransform = Transform();

    bool xDoor = (m_level.getPixel(x, y - 1) & 0xFFFFFF) == 0 &&
        (m_level.getPixel(x, y + 1) & 0xFFFFFF) == 0;
    bool yDoor = (m_level.getPixel(x - 1, y) & 0xFFFFFF) == 0 &&
        (m_level.getPixel(x + 1, y) & 0xFFFFFF) == 0;
    if (!(xDoor ^ yDoor))
    {
        std::cerr << "Level Generation failed : Placed doors at invalid position : "
            << x << ", " << y << std::endl;
        exit(1);
    }
    Vector3f openPosition = NULL;

    if (yDoor)
    {
        doorTransform.setPos(Vector3f(x, 0, y + SPOT_LENGTH / 2));
        openPosition = doorTransform.getPos() -
            Vector3f(DOOR_OPEN_MOVEMENT_AMOUNT, 0.0f, 0.0f);
    }
    if (xDoor)
    {
        doorTransform.setPos(Vector3f(x + SPOT_WIDTH / 2, 0, y));
        doorTransform.setRot(Vector3f(0, 90, 0));
        openPosition = doorTransform.getPos() -
            Vector3f(0.0f, 0.0f, DOOR_OPEN_MOVEMENT_AMOUNT);
    }
    m_doors.push_back(new Door(doorTransform, &m_material, openPosition));
}

```

```

Vector2f Level::findNearestVector2f(const Vector2f& a, const Vector2f& b,
    const Vector2f& positionRelativeTo)
{
    if (b != NULL && (a == NULL || ((a - positionRelativeTo).length() >
        (b - positionRelativeTo).length())))
        return b;

    return a;
}
Vector2f Level::lineIntersectRect(const Vector2f& lineStart,

```

```

        const Vector2f& lineEnd,
        const Vector2f& rectPos,
        const Vector2f& rectSize)
{
    Vector2f result = NULL;

    Vector2f collisionVector =
        lineIntersect(lineStart, lineEnd, rectPos,
            Vector2f(rectPos.getX() + rectSize.getX(), rectPos.getY()));
    result = findNearestVector2f(result, collisionVector, lineStart);

    collisionVector =
        lineIntersect(lineStart, lineEnd, rectPos,
            Vector2f(rectPos.getX(), rectPos.getY() + rectSize.getY()));
    result = findNearestVector2f(result, collisionVector, lineStart);

    collisionVector =
        lineIntersect(lineStart, lineEnd,
            Vector2f(rectPos.getX(), rectPos.getY() + rectSize.getY()),
            rectPos + rectSize);
    result = findNearestVector2f(result, collisionVector, lineStart);

    collisionVector =
        lineIntersect(lineStart, lineEnd,
            Vector2f(rectPos.getX() + rectSize.getX(), rectPos.getY()),
            rectPos + rectSize);
    result = findNearestVector2f(result, collisionVector, lineStart);

    return result;
}

Vector2f Level::lineIntersect(const Vector2f& lineStart1,
    const Vector2f& lineEnd1,
    const Vector2f& lineStart2,
    const Vector2f& lineEnd2)
{
    Vector2f line1 = lineEnd1 - lineStart1;
    Vector2f line2 = lineEnd2 - lineStart2;

    float cross = Vector2fCross(line1, line2);

    if (cross == 0)
        return NULL;

    Vector2f distanceBetweenLineStarts = lineStart2 - lineStart1;

    float a = Vector2fCross(distanceBetweenLineStarts, line2) / cross;
    float b = Vector2fCross(distanceBetweenLineStarts, line1) / cross;

    if (0.0f < a && a < 1.0f && 0.0f < b && b < 1.0f)
        return lineStart1 + line1.mult(a);

    return NULL;
}

float Level::Vector2fCross(const Vector2f& a, const Vector2f& b)
{
    return a.getX() * b.getY() - a.getY() * b.getX();
}

void Level::removeMedkit(Medkit* medkit)
{
    m_medkitsToRemove.emplace_back(medkit);
}

```

Sources/matrix4f.cpp

```
#include "matrix4f.h"
#include <math.h>

void Matrix4f::initIdentity()
{
    m[0][0] = 1.0f;
    m[0][1] = 0.0f;
    m[0][2] = 0.0f;
    m[0][3] = 0.0f;

    m[1][0] = 0.0f;
    m[1][1] = 1.0f;
    m[1][2] = 0.0f;
    m[1][3] = 0.0f;

    m[2][0] = 0.0f;
    m[2][1] = 0.0f;
    m[2][2] = 1.0f;
    m[2][3] = 0.0f;

    m[3][0] = 0.0f;
    m[3][1] = 0.0f;
    m[3][2] = 0.0f;
    m[3][3] = 1.0f;
}

void Matrix4f::initScaleTransform(float scaleX, float scaleY, float scaleZ)
{
    m[0][0] = scaleX;
    m[0][1] = 0.0f;
    m[0][2] = 0.0f;
    m[0][3] = 0.0f;

    m[1][0] = 0.0f;
    m[1][1] = scaleY;
    m[1][2] = 0.0f;
    m[1][3] = 0.0f;

    m[2][0] = 0.0f;
    m[2][1] = 0.0f;
    m[2][2] = scaleZ;
    m[2][3] = 0.0f;

    m[3][0] = 0.0f;
    m[3][1] = 0.0f;
    m[3][2] = 0.0f;
    m[3][3] = 1.0f;
}

void Matrix4f::initRotateTransform(float rotateX, float rotateY, float rotateZ)
{
    Matrix4f rx, ry, rz;

    const float x = ToRadian(rotateX);
    const float y = ToRadian(rotateY);
    const float z = ToRadian(rotateZ);

    rx.m[0][0] = 1.0f;
    rx.m[0][1] = 0.0f;
    rx.m[0][2] = 0.0f;
    rx.m[0][3] = 0.0f;

    rx.m[1][0] = 0.0f;
    rx.m[1][1] = cosf(x);
    rx.m[1][2] = -sinf(x);
    rx.m[1][3] = 0.0f;

    rx.m[2][0] = 0.0f;
    rx.m[2][1] = sinf(x);
    rx.m[2][2] = cosf(x);
    rx.m[2][3] = 0.0f;

    rx.m[3][0] = 0.0f;
    rx.m[3][1] = 0.0f;
```



```

    rx.m[3][2] = 0.0f;
    rx.m[3][3] = 1.0f;

    ry.m[0][0] = cosf(y);
    ry.m[0][1] = 0.0f;
    ry.m[0][2] = -sinf(y);
    ry.m[0][3] = 0.0f;

    ry.m[1][0] = 0.0f;
    ry.m[1][1] = 1.0f;
    ry.m[1][2] = 0.0f;
    ry.m[1][3] = 0.0f;

    ry.m[2][0] = sinf(y);
    ry.m[2][1] = 0.0f;
    ry.m[2][2] = cosf(y);
    ry.m[2][3] = 0.0f;

    ry.m[3][0] = 0.0f;
    ry.m[3][1] = 0.0f;
    ry.m[3][2] = 0.0f;
    ry.m[3][3] = 1.0f;

    rz.m[0][0] = cosf(z);
    rz.m[0][1] = -sinf(z);
    rz.m[0][2] = 0.0f;
    rz.m[0][3] = 0.0f;

    rz.m[1][0] = sinf(z);
    rz.m[1][1] = cosf(z);
    rz.m[1][2] = 0.0f;
    rz.m[1][3] = 0.0f;

    rz.m[2][0] = 0.0f;
    rz.m[2][1] = 0.0f;
    rz.m[2][2] = 1.0f;
    rz.m[2][3] = 0.0f;

    rz.m[3][0] = 0.0f;
    rz.m[3][1] = 0.0f;
    rz.m[3][2] = 0.0f;
    rz.m[3][3] = 1.0f;

    *this = rz * ry * rx;
}

void Matrix4f::initTranslationTransform(float x, float y, float z)
{
    m[0][0] = 1.0f;
    m[0][1] = 0.0f;
    m[0][2] = 0.0f;
    m[0][3] = x;

    m[1][0] = 0.0f;
    m[1][1] = 1.0f;
    m[1][2] = 0.0f;
    m[1][3] = y;

    m[2][0] = 0.0f;
    m[2][1] = 0.0f;
    m[2][2] = 1.0f;
    m[2][3] = z;

    m[3][0] = 0.0f;
    m[3][1] = 0.0f;
    m[3][2] = 0.0f;
    m[3][3] = 1.0f;
}

void Matrix4f::initCameraTransform(const Vector3f& target, const Vector3f& up)
{
    Vector3f n = target;
    n.normalize();

    Vector3f u = up;
    u.normalize();
    u = u.cross(n);

```

```

Vector3f v = n.cross(u);

m[0][0] = u.x;
m[0][1] = u.y;
m[0][2] = u.z;
m[0][3] = 0.0f;

m[1][0] = v.x;
m[1][1] = v.y;
m[1][2] = v.z;
m[1][3] = 0.0f;

m[2][0] = n.x;
m[2][1] = n.y;
m[2][2] = n.z;
m[2][3] = 0.0f;

m[3][0] = 0.0f;
m[3][1] = 0.0f;
m[3][2] = 0.0f;
m[3][3] = 1.0f;
}

void Matrix4f::initPersProjTransform(float fov, float width, float height,
                                     float zNear, float zFar)
{
    const float ar = width / height;
    const float zRange = zNear - zFar;
    const float tanHalfFOV = tanf(ToRadian(fov / 2.0f));

    m[0][0] = 1.0f / (tanHalfFOV * ar);
    m[0][1] = 0.0f;
    m[0][2] = 0.0f;
    m[0][3] = 0.0;

    m[1][0] = 0.0f;
    m[1][1] = 1.0f / tanHalfFOV;
    m[1][2] = 0.0f;
    m[1][3] = 0.0;

    m[2][0] = 0.0f;
    m[2][1] = 0.0f;
    m[2][2] = (-zNear - zFar) / zRange;
    m[2][3] = 2.0f * zFar * zNear / zRange;

    m[3][0] = 0.0f;
    m[3][1] = 0.0f;
    m[3][2] = 1.0f;
    m[3][3] = 0.0;
}

Matrix4f Matrix4f::operator*(const Matrix4f& right) const
{
    Matrix4f ret;
    for (unsigned int i = 0; i < 4; i++)
    {
        for (unsigned int j = 0; j < 4; j++)
        {
            ret.m[i][j] = m[i][0] * right.m[0][j] + m[i][1] * right.m[1][j] +
                m[i][2] * right.m[2][j] + m[i][3] * right.m[3][j];
        }
    }
    return ret;
}

const float* Matrix4f::operator[](int index) const
{
    return m[index];
}

float* Matrix4f::operator[](int index)
{
    return m[index];
}

Matrix4f Matrix4f::createTranslationTransform(float x, float y, float z)
{

```

```
Matrix4f mat;  
mat.initTranslationTransform(x, y, z);  
return mat;  
}
```

Sources/medkit.cpp

```
#include "game.h"
#include "medkit.h"

Mesh* Medkit::m_mesh = nullptr;
Material* Medkit::m_material = nullptr;

Medkit::Medkit(const Vector3f& position)
{
    if (!m_mesh)
    {
        Vertex vertices[] = {
            Vertex(Vector3f(-SIZE_X, START, START), Vector2f(TEXT_MAX_X, TEXT_MAX_Y)),
            Vertex(Vector3f(-SIZE_X, SIZE_Y, START), Vector2f(TEXT_MAX_X, TEXT_MIN_Y)),
            Vertex(Vector3f(SIZE_X, SIZE_Y, START), Vector2f(TEXT_MIN_X, TEXT_MIN_Y)),
            Vertex(Vector3f(SIZE_X, START, START), Vector2f(TEXT_MIN_X, TEXT_MAX_Y));

        int indices[] = {0, 1, 2, 0, 2, 3};

        m_mesh = new Mesh();
        m_mesh->addVertices(vertices, ARRAY_SIZE(vertices), indices,
            ARRAY_SIZE(indices));
    }

    if (!m_material)
    {
        m_material = new Material(new Texture("MEDIA0.png"));
    }

    transform.setPos(position);
}

void Medkit::update()
{
    Vector3f directionToCamera =
        Transform::getCamera().getPos() - transform.getPos();

    float angleToFaceTheCamera = static_cast<float>{
        std::atan2(directionToCamera.getZ(), directionToCamera.getX()) * 180.0 /
        M_PI};

    if (directionToCamera.getX() < 0)
        angleToFaceTheCamera += 360;

    transform.getRot().setY(angleToFaceTheCamera + 90);

    if (directionToCamera.length() < PICKUP_DISTANCE)
    {
        Player* player = Game::getLevel()->getPlayer();

        if (player->getHealth() < player->getMaxHealth())
        {
            Game::getLevel()->removeMedkit(this);
            player->damage(-HEAL_AMOUNT);
        }
    }
}

void Medkit::render()
{
    Shader* shader = Game::getLevel()->getShader();
    shader->updateUniforms(transform.getTransformation(),
        transform.getProjectedTransformation(), *m_material);
    m_mesh->draw();
}
```

Sources/mesh.cpp

```
#include "mesh.h"
#include <GL/glew.h>

Mesh::Mesh()
{
    glGenVertexArrays(1, &m_vao); // Generate a VAO
    glGenBuffers(1, &m_vbo);
    glGenBuffers(1, &m_ibo);
    m_size = 0;
}

Mesh::~Mesh()
{
    glDeleteVertexArrays(1, &m_vao); // Delete VAO
    glDeleteBuffers(1, &m_vbo);
    glDeleteBuffers(1, &m_ibo);
}

void Mesh::addVertices(Vertex* vertices, int vertSize, int* indices,
                      int indexSize, bool calcNormals)
{
    m_size = indexSize;

    if (calcNormals)
        this->calcNormals(vertices, vertSize, indices, indexSize);

    // Bind VAO
    glBindVertexArray(m_vao);

    glBindBuffer(GL_ARRAY_BUFFER, m_vbo);
    glBufferData(GL_ARRAY_BUFFER, vertSize * sizeof(Vertex), vertices,
                 GL_STATIC_DRAW);

    glEnableVertexAttribArray(0);
    glEnableVertexAttribArray(1);
    glEnableVertexAttribArray(2);

    glBindBuffer(GL_ARRAY_BUFFER, m_vbo);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), 0);
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, sizeof(Vertex),
                          (GLvoid*)sizeof(Vector3f));
    glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex),
                          (GLvoid*)(sizeof(Vector3f) + sizeof(Vector2f)));

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, m_ibo);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, indexSize * sizeof(int), indices,
                 GL_STATIC_DRAW);

    glBindVertexArray(0); // Unbind VAO
}

void Mesh::draw()
{
    glBindVertexArray(m_vao);
    glDrawElements(GL_TRIANGLES, m_size, GL_UNSIGNED_INT, 0);
    glBindVertexArray(0);
}

void Mesh::calcNormals(Vertex* vertices, int vertSize, int* indices,
                      int indexSize)
{
    for (int i = 0; i < indexSize; i += 3)
    {
        int i0 = indices[i];
        int i1 = indices[i + 1];
        int i2 = indices[i + 2];

        Vector3f v1 = vertices[i1].pos - vertices[i0].pos;
        Vector3f v2 = vertices[i2].pos - vertices[i0].pos;

        Vector3f normal = v1.cross(v2).normalize();

        vertices[i0].normal += normal;
        vertices[i1].normal += normal;
    }
}
```

```
    vertices[i2].normal += normal;
}

for (int i = 0; i < vertSize; i++)
    vertices[i].normal.normalize();
}
```

Sources/monster.cpp

```
#include "game.h"
#include "monster.h"
#include "transform.h"
#include "window.h"
#include <vector>

static const float SCALE = 0.7f;
static const float SIZEY = SCALE;
static const float SIZEX =
    (float)((double)SIZEY / (1.9310344827586206896551724137931 * 2.0));
static const float START = 0.0;

static const float OFFSET_X = 0.00f;
static const float OFFSET_Y = 0.0f;
static const float OFFSET_FROM_GROUND = 0.0f;

static const float TEX_MIN_X = -OFFSET_X;
static const float TEX_MAX_X = -1 - OFFSET_X;
static const float TEX_MIN_Y = -OFFSET_Y;
static const float TEX_MAX_Y = 1 - OFFSET_Y;

static const int STATE_IDLE = 0;
static const int STATE_CHASE = 1;
static const int STATE_ATTACK = 2;
static const int STATE_DYING = 3;
static const int STATE_DEAD = 4;

static const float MOVE_SPEED = 1.0f;
static const float MOVEMENT_STOP_DISTANCE = 1.5f;
static const float MONSTER_WIDTH = 0.2f;
static const float MONSTER_LENGTH = 0.2f;

static const float SHOOT_DISTANCE = 1000.0f;
static const float SHOT_ANGLE = 10.0f;
static const float ATTACK_CHANCE = 0.75f;
static const int MAX_HEALTH = 100;
static const int DAMAGE_MIN = 5;
static const int DAMAGE_MAX = 25;

Mesh* Monster::m_mesh = nullptr;
std::vector<Texture*> Monster::m_animations;

Monster::Monster(const Transform& transform)
    : m_transform(transform), state(STATE_IDLE), health(MAX_HEALTH),
      canLook(false), canAttack(false), deathTime(0),
      m_rand(std::random_device{}())
{
    if (m_animations.empty())
    {
        m_animations.push_back(new Texture("SSWVA1.png"));
        m_animations.push_back(new Texture("SSWVB1.png"));
        m_animations.push_back(new Texture("SSWVC1.png"));
        m_animations.push_back(new Texture("SSWVD1.png"));

        m_animations.push_back(new Texture("SSWVE0.png"));
        m_animations.push_back(new Texture("SSWVF0.png"));
        m_animations.push_back(new Texture("SSWVG0.png"));

        m_animations.push_back(new Texture("SSWVH0.png"));

        m_animations.push_back(new Texture("SSWVI0.png"));
        m_animations.push_back(new Texture("SSWVJ0.png"));
        m_animations.push_back(new Texture("SSWVK0.png"));
        m_animations.push_back(new Texture("SSWVL0.png"));

        m_animations.push_back(new Texture("SSWVM0.png"));
    }
}

if (!m_mesh)
{
    Vertex vertices[] = {
        Vertex(Vector3f(-SIZEX, START, START), Vector2f(TEX_MAX_X, TEX_MAX_Y)),
        Vertex(Vector3f(-SIZEX, SIZEY, START), Vector2f(TEX_MAX_X, TEX_MIN_Y)),
        Vertex(Vector3f(SIZEX, SIZEY, START), Vector2f(TEX_MIN_X, TEX_MIN_Y)),
```

```

        Vertex(Vector3f(SIZE_X, START, START), Vector2f(TEX_MIN_X, TEX_MAX_Y));

int indices[] = {0, 1, 2, 0, 2, 3};

m_mesh = new Mesh();
m_mesh->addVertices(vertices, ARRAY_SIZE(vertices), indices,
                    ARRAY_SIZE(indices));
}

this->m_transform = transform;
}

Transform& Monster::getTransform()
{
    return m_transform;
}

Vector2f Monster::getSize() const
{
    return Vector2f(MONSTER_WIDTH, MONSTER_LENGTH);
}

void Monster::damage(int amount)
{
    if (state == STATE_IDLE)
    {
        state = STATE_CHASE;
    }
    if (state != STATE_DEAD)
        Window::playSound("EnemyPain_2.wav", Window::MONSTER_HEALTH_CHANNEL);
    health -= amount;
    if (health <= 0)
    {
        state = STATE_DYING;
    }
}

void Monster::idleUpdate(const Vector3f& orientation, float direction)
{
    double time = (double)Time::getTime();
    double timeDecimals = time - (int)time;

    if (timeDecimals < 0.5)
    {
        canLook = true;
        m_material.setTexture(m_animations[0]);
    }
    else
    {
        m_material.setTexture(m_animations[1]);
        if (direction < SHOOT_DISTANCE)
        {
            Vector2f lineStart(m_transform.getPos().getX(),
                               m_transform.getPos().getZ());
            Vector2f castDirection(orientation.getX(), orientation.getZ());
            Vector2f lineEnd = lineStart + castDirection * SHOOT_DISTANCE;

            Vector2f collisionVector =
                Game::getLevel()->checkIntersection(lineStart, lineEnd, false);
            Vector2f playerIntersectVector(Transform::getCamera().getPos().getX(),
                                           Transform::getCamera().getPos().getZ());

            if (collisionVector.length() == 0 ||
                (playerIntersectVector - lineStart).length() <
                 (collisionVector - lineStart).length())
            {
                state = STATE_CHASE;
            }
            canLook = false;
        }
    }
}

void Monster::chaseUpdate(const Vector3f& orientation, float distance)
{
    double time = (double)Time::getTime();
    double timeDecimals = time - (int)time;

```



```

if (timeDecimals < 0.25)
{
    m_material.setTexture(m_animations[0]);
}
else if (timeDecimals < 0.5)
{
    m_material.setTexture(m_animations[1]);
}
else if (timeDecimals < 0.75)
{
    m_material.setTexture(m_animations[2]);
}
else
{
    m_material.setTexture(m_animations[3]);
}

if (std::generate_canonical<double, 1000000000>(m_rand) <
    ATTACK_CHANCE * Time::getDelta())
    state = STATE_ATTACK;

if (distance > MOVEMENT_STOP_DISTANCE)
{
    float moveAmount = MOVE_SPEED * Time::getDelta();

    Vector3f oldPos = m_transform.getPos();
    Vector3f newPos = m_transform.getPos() + orientation * moveAmount;

    Vector3f collisionVector = Game::getLevel()->checkCollision(
        oldPos, newPos, MONSTER_WIDTH, MONSTER_LENGTH);
    Vector3f movementVector = collisionVector.mult(orientation);

    if (movementVector.length() > 0)
    {
        m_transform.setPos(m_transform.getPos() + movementVector * moveAmount);
    }

    if ((movementVector - orientation).length() != 0)
    {
        Game::getLevel()->openDoors(m_transform.getPos(), false);
    }
}
else
{
    state = STATE_ATTACK;
}
}

void Monster::attackUpdate(const Vector3f& orientation, float distance)
{
    double time = (double)Time::getTime();
    double timeDecimals = time - static_cast<int>(time);

    if (timeDecimals < 0.25)
    {
        m_material.setTexture(m_animations[4]);
    }
    else if (timeDecimals < 0.5)
    {
        m_material.setTexture(m_animations[5]);
    }
    else if (timeDecimals < 0.75)
    {
        m_material.setTexture(m_animations[6]);
    }

    if (canAttack)
    {
        Window::playSound("Machine_Gun.wav", Window::MONSTER_FIRE_CHANNEL, 0.4f);
        Vector2f lineStart(m_transform.getPos().getX(),
            m_transform.getPos().getZ());
        Vector2f castDirection(orientation.getX(), orientation.getZ());
        castDirection = castDirection.rotate(
            (std::generate_canonical<float, 10>(m_rand) - 0.5f) * SHOT_ANGLE);
        Vector2f lineEnd = lineStart + castDirection * SHOOT_DISTANCE;

        Vector2f collisionVector =
            Game::getLevel()->checkIntersection(lineStart, lineEnd, false);
    }
}

```

```

Vector2f playerIntersectVector = Game::getLevel()->lineIntersectRect(
    lineStart, lineEnd,
    Vector2f(Transform::getCamera().getPos().getX(),
        Transform::getCamera().getPos().getZ()),
    Vector2f(Player::PLAYER_SIZE, Player::PLAYER_SIZE));

if ((playerIntersectVector - lineStart).length() == 0 ||
    (collisionVector.length() == 0 ||
    (playerIntersectVector - lineStart).length() <
    (collisionVector - lineStart).length()))
{
    Game::getLevel()->damagePlayer(
        m_rand() % (DAMAGE_MAX - DAMAGE_MIN + 1) + DAMAGE_MIN);
}

canAttack = false;
}
}
else
{
    m_material.setTexture(m_animations[5]);
    canAttack = true;
    state = STATE_CHASE;
}
}

void Monster::dyingUpdate(const Vector3f& orientation, float distance)
{
    double time = (double)Time::getTime();

    if (deathTime == 0)
    {
        deathTime = time;
    }

    float time1 = 0.1f;
    float time2 = 0.3f;
    float time3 = 0.45f;
    float time4 = 0.6f;

    if (time < deathTime + time1)
    {
        m_material.setTexture(m_animations[8]);
        m_transform.setScale(Vector3f(1, 0.9642857142857142f, 1));
    }
    else if (time < deathTime + time2)
    {
        Window::playSound("Enemy_Dies.wav", Window::MONSTER_HEALTH_CHANNEL);
        m_material.setTexture(m_animations[9]);
        m_transform.setScale(Vector3f(1.7f, 0.9f, 1));
    }
    else if (time < deathTime + time3)
    {
        m_material.setTexture(m_animations[10]);
        m_transform.setScale(Vector3f(1.7f, 0.9f, 1));
    }
    else if (time < deathTime + time4)
    {
        m_material.setTexture(m_animations[11]);
        m_transform.setScale(Vector3f(1.7f, 0.5f, 1));
    }
    else
    {
        state = STATE_DEAD;
    }
}

void Monster::deadUpdate(const Vector3f& orientation, float distance)
{
    m_material.setTexture(m_animations[12]);
    m_transform.setScale(Vector3f(1.7586206896551724f, 0.2857142857142857f, 1));
}

void Monster::alignWithGround()
{
    m_transform.getPos().setY(OFFSET_FROM_GROUND);
}

```

```

void Monster::faceCamera(const Vector3f& directionToCamera)
{
    float angleToFaceTheCamera =
        std::atan2(directionToCamera.getZ(), directionToCamera.getX()) * 180.0f /
        M_PI;

    if (directionToCamera.getX() < 0)
    {
        angleToFaceTheCamera += 360;
    }

    m_transform.getRot().setY(angleToFaceTheCamera + 90);
}

void Monster::update()
{
    Vector3f directionToCamera =
        Transform::getCamera().getPos() - m_transform.getPos();

    float distance = directionToCamera.length();
    Vector3f orientation = directionToCamera / distance;

    alignWithGround();
    faceCamera(orientation);

    switch (state)
    {
    case STATE_IDLE:
        idleUpdate(orientation, distance);
        break;
    case STATE_CHASE:
        chaseUpdate(orientation, distance);
        break;
    case STATE_ATTACK:
        attackUpdate(orientation, distance);
        break;
    case STATE_DYING:
        dyingUpdate(orientation, distance);
        break;
    case STATE_DEAD:
        deadUpdate(orientation, distance);
        break;
    }
}

void Monster::render()
{
    Shader* shader = Game::getLevel()->getShader();
    shader->updateUniforms(m_transform.getTransformation(),
                        m_transform.getProjectedTransformation(), m_material);
    m_mesh->draw();
}

```

Sources/phongShader.cpp

```
#include "phongShader.h"
#include "transform.h"

#include "util.h"
#include <iostream>

Vector3f PhongShader::m_ambientLight = Vector3f(0.1f, 0.1f, 0.1f);
DirectionalLight PhongShader::m_directionalLight =
    DirectionalLight(BaseLight(Vector3f(0, 0, 0), 0), Vector3f(0, 0, 0));
PointLight* PhongShader::m_pointLights = NULL;
SpotLight* PhongShader::m_spotLights = NULL;
int PhongShader::m_numPointLights = 0;
int PhongShader::m_numSpotLights = 0;

PhongShader::PhongShader()
{
    addVertexShaderFromFile("phongVertex.vs");
    addFragmentShaderFromFile("phongFragment.fs");
    compileShader();

    addUniform("transform");
    addUniform("transformProjected");
    addUniform("baseColor");
    addUniform("ambientLight");

    addUniform("specularIntensity");
    addUniform("specularPower");
    addUniform("eyePos");

    addUniform("directionalLight.base.color");
    addUniform("directionalLight.base.intensity");
    addUniform("directionalLight.direction");

    for (int i = 0; i < MAX_POINT_LIGHTS; i++)
    {
        std::string pointLightName = "pointLights[" + std::to_string(i) + "]";

        addUniform(pointLightName + ".base.color");
        addUniform(pointLightName + ".base.intensity");
        addUniform(pointLightName + ".atten.constant");
        addUniform(pointLightName + ".atten.linear");
        addUniform(pointLightName + ".atten.exponent");
        addUniform(pointLightName + ".position");
        addUniform(pointLightName + ".range");
    }

    for (int i = 0; i < MAX_SPOT_LIGHTS; i++)
    {
        std::string spotLightName = "spotLights[" + std::to_string(i) + "]";

        addUniform(spotLightName + ".pointLight.base.color");
        addUniform(spotLightName + ".pointLight.base.intensity");
        addUniform(spotLightName + ".pointLight.atten.constant");
        addUniform(spotLightName + ".pointLight.atten.linear");
        addUniform(spotLightName + ".pointLight.atten.exponent");
        addUniform(spotLightName + ".pointLight.position");
        addUniform(spotLightName + ".pointLight.range");
        addUniform(spotLightName + ".direction");
        addUniform(spotLightName + ".cutoff");
    }
}

PhongShader::~PhongShader()
{
}

void PhongShader::updateUniforms(const Matrix4f& worldMatrix,
                                const Matrix4f& projectedMatrix,
                                Material& material)
{
    material.texture->bind();
    setUniform("transformProjected", projectedMatrix);
    setUniform("transform", worldMatrix);
    setUniform("baseColor", material.color);
```

```

setUniform("ambientLight", m_ambientLight);
setUniform("directionalLight.direction", m_directionalLight.direction);
setUniform("directionalLight.base.color", m_directionalLight.base.color);
setUniformf("directionalLight.base.intensity",
            m_directionalLight.base.intensity);

for (int i = 0; i < m_numPointLights; i++)
{
    std::string pointLightName = "pointLights[" + std::to_string(i) + "]";

    setUniform(pointLightName + ".base.color", m_pointLights[i].base.color);
    setUniformf(pointLightName + ".base.intensity",
                m_pointLights[i].base.intensity);
    setUniformf(pointLightName + ".atten.constant",
                m_pointLights[i].atten.constant);
    setUniformf(pointLightName + ".atten.linear",
                m_pointLights[i].atten.linear);
    setUniformf(pointLightName + ".atten.exponent",
                m_pointLights[i].atten.exponent);
    setUniform(pointLightName + ".position", m_pointLights[i].position);
    setUniformf(pointLightName + ".range", m_pointLights[i].range);
}

for (int i = 0; i < m_numSpotLights; i++)
{
    std::string spotLightName = "spotLights[" + std::to_string(i) + "]";

    setUniform(spotLightName + ".pointLight.base.color",
                m_spotLights[i].pointLight.base.color);
    setUniformf(spotLightName + ".pointLight.base.intensity",
                m_spotLights[i].pointLight.base.intensity);
    setUniformf(spotLightName + ".pointLight.atten.constant",
                m_spotLights[i].pointLight.atten.constant);
    setUniformf(spotLightName + ".pointLight.atten.linear",
                m_spotLights[i].pointLight.atten.linear);
    setUniformf(spotLightName + ".pointLight.atten.exponent",
                m_spotLights[i].pointLight.atten.exponent);
    setUniform(spotLightName + ".pointLight.position",
                m_spotLights[i].pointLight.position);
    setUniformf(spotLightName + ".pointLight.range",
                m_spotLights[i].pointLight.range);
    setUniform(spotLightName + ".direction", m_spotLights[i].direction);
    setUniformf(spotLightName + ".cutoff", m_spotLights[i].cutoff);
}

setUniformf("specularIntensity", material.specularIntensity);
setUniformf("specularPower", material.specularPower);

setUniform("eyePos", Transform::getCamera().getPos());
}

Vector3f& PhongShader::getAmbientLight()
{
    return m_ambientLight;
}

DirectionalLight& PhongShader::getDirectionalLight()
{
    return m_directionalLight;
}

void PhongShader::setAmbientLight(const Vector3f& ambientLight)
{
    PhongShader::m_ambientLight = ambientLight;
}

void PhongShader::setDirectionalLight(const DirectionalLight& directionalLight)
{
    PhongShader::m_directionalLight = directionalLight;
}

void PhongShader::setPointLights(PointLight* pointLights, int arraySize)
{
    if (arraySize > MAX_POINT_LIGHTS)
    {
        std::cerr << "Error: Too many point lights" << std::endl;
    }
}

```

```
    else
    {
        PhongShader::m_numPointLights = arraySize;
        PhongShader::m_pointLights = pointLights;
    }
}

void PhongShader::setSpotLights(SpotLight* spotLights, int arraySize)
{
    if (arraySize > MAX_SPOT_LIGHTS)
    {
        std::cerr << "Error: Too many spot lights" << std::endl;
    }
    else
    {
        PhongShader::m_numSpotLights = arraySize;
        PhongShader::m_spotLights = spotLights;
    }
}
```

Sources/player.cpp

```
#include "game.h"
#include "level.h"
#include "player.h"
#include "window.h"
#include <iostream>
#include <ostream>
#include <random>
#include <stdlib.h>
#include <string>

static const float GUN_OFFSET = -0.0875f;

static const float SCALE = 0.0625f;
static const float SIZEY = SCALE;
static const float SIZEX =
    (float)((double)SIZEY / (1.0379746835443037974683544303797 * 2.0));
static const float START = 0;

static const float OFFSET_X = 0.0f; // 0.05f
static const float OFFSET_Y = 0.0f; // 0.01f

static const float GUN_FIRE_DURATION = 0.2f;

static const float TEX_MIN_X = -OFFSET_X;
static const float TEX_MAX_X = -1 - OFFSET_X;
static const float TEX_MIN_Y = -OFFSET_Y;
static const float TEX_MAX_Y = 1 - OFFSET_Y;

static const float MOUSE_SENSITIVITY = 0.25f;
static const float MOVE_SPEED = 3.5f;

static const float SHOOT_DISTANCE = 1000.0f;
static const int DAMAGE_MIN = 20;
static const int DAMAGE_MAX = 60;
static const int MAX_HEALTH = 100;

static const Vector3f zeroVector = Vector3f(0, 0, 0);

Mesh* Player::m_mesh = nullptr;
Material* Player::gunMaterial = nullptr;
bool Player::mouseLocked = false;

Player::Player(const Vector3f& pos)
    : m_camera(pos), rd(std::random_device()), rand(rd()), m_health(MAX_HEALTH),
      centerPosition((float)window::getWidth() / 2,
                     (float)window::getHeight() / 2),
      movementVector(Vector3f(0, 0, 0))
{
    // system("clear");
    std::cout << "Health : " << m_health << std::endl;

    bool isFiring = false;
    float gunFireTimer = 0.0f;
    if (!m_mesh)
    {
        Vertex vertices[] = {
            Vertex(Vector3f(-SIZEX, START, START), Vector2f(TEX_MAX_X, TEX_MAX_Y)),
            Vertex(Vector3f(-SIZEX, SIZEY, START), Vector2f(TEX_MAX_X, TEX_MIN_Y)),
            Vertex(Vector3f(SIZEX, SIZEY, START), Vector2f(TEX_MIN_X, TEX_MIN_Y)),
            Vertex(Vector3f(SIZEX, START, START), Vector2f(TEX_MIN_X, TEX_MAX_Y));

        int indices[] = {0, 1, 2, 0, 2, 3};
        m_mesh = new Mesh();
        m_mesh->addVertices(vertices, ARRAY_SIZE(vertices), indices,
                           ARRAY_SIZE(indices));
    }
    if (!gunMaterial)
    {
        gunMaterial = new Material(new Texture("PISGB0.png"));
    }

    gunTransform.setPos(Vector3f(7, 0, 7));
}

void Player::damage(int amount)
```

```

{
    if (amount > 0)
    {
        Window::playSound("PlayerPain.wav", Window::PLAYER_HEALTH_CHANNEL, 0.7f);
    }
    else
    {
        Window::playSound("Health.wav", Window::PLAYER_HEALTH_CHANNEL, 0.5f);
    }
    m_health -= amount;
    if (m_health > MAX_HEALTH)
    {
        m_health = MAX_HEALTH;
    }
    system("clear");

    std::cout << "Health : " << m_health << std::endl;

    if (m_health <= 0)
    {
        Window::playSound("Player_Dies.wav", Window::PLAYER_DEATH_CHANNEL);
        Game::setIsRunning(false);
        std::cout << "You Just Died ! GAME OVER!" << std::endl;
    }
}

int Player::getDamage()
{
    return std::uniform_int_distribution<>(DAMAGE_MIN, DAMAGE_MAX)(rand);
}

int Player::getHealth()
{
    return m_health;
}

int Player::getMaxHealth()
{
    return MAX_HEALTH;
}

void Player::input()
{
    if (Input::getKey(KEY::KEY_ESCAPE))
    {
        Input::setCursor(true);
        mouseLocked = false;
    }

    if (Input::getKeyDown(KEY::KEY_E))
    {
        Game::getLevel()->openDoors(m_camera.getPos(), true);
    }

    if (Input::getMouseDown(MOUSE::LEFT_MOUSE))
    {
        if (!mouseLocked)
        {
            Input::setMousePosition(centerPosition);
            Input::setCursor(false);
            mouseLocked = true;
        }
        else
        {
            isFiring = true;
            gunFireTimer = GUN_FIRE_DURATION;
            gunMaterial = new Material(new Texture("PISFA0.png"));

            Window::playSound("Pistol.wav", Window::PLAYER_FIRE_CHANNEL, 0.65f);

            Vector2f lineStart(m_camera.getPos().getX(), m_camera.getPos().getZ());
            Vector2f castDirection(m_camera.getForward().getX(),
                                   m_camera.getForward().getZ());
            castDirection = castDirection.normalize();
            Vector2f lineEnd = lineStart + castDirection * SHOOT_DISTANCE;

            Game::getLevel()->checkIntersection(lineStart, lineEnd, true);
        }
    }
}

```



```

    }
}

movementVector = zeroVector;

if (Input::getKey(KEY::KEY_W))
    movementVector += m_camera.getForward();
if (Input::getKey(KEY::KEY_S))
    movementVector -= m_camera.getForward();
if (Input::getKey(KEY::KEY_A))
    movementVector += m_camera.getLeft();
if (Input::getKey(KEY::KEY_D))
    movementVector += m_camera.getRight();

if (mouseLocked)
{
    Vector2f deltaPos = Input::getMousePosition() - centerPosition;

    bool rotY = deltaPos.x != 0;
    bool rotX = deltaPos.y != 0;

    if (rotY)
        m_camera.rotateY(deltaPos.x * MOUSE_SENSITIVITY);
    if (rotX)
        m_camera.rotateX(deltaPos.y * MOUSE_SENSITIVITY * 0);

    if (rotY || rotX)
        Input::setMousePosition(centerPosition);
}
}

void Player::update()
{
    float movAmt = (float)(MOVE_SPEED * Time::getDelta());

    if (isFiring)
    {
        gunFireTimer -= Time::getDelta();
        if (gunFireTimer <= 0.0f)
        {
            isFiring = false;
            gunMaterial = new Material(new Texture("PISGB0.png"));
        }
    }

    movementVector.setY(0);
    if (movementVector.length() > 0)
        movementVector = movementVector.normalize();

    Vector3f oldPos = m_camera.getPos();
    Vector3f newPos = oldPos + (movementVector * movAmt);

    Vector3f collisionVector = Game::getLevel()->checkCollision(
        oldPos, newPos, PLAYER_SIZE, PLAYER_SIZE);

    if (collisionVector.getX() != 0 || collisionVector.getZ() != 0)
        movementVector = movementVector.mult(collisionVector);
    if (collisionVector.getX() == 0 && collisionVector.getZ() == 0)
    {
        movementVector.setX(0);
        movementVector.setZ(0);
    }
    if (movementVector.length() > 0)
    {
        Window::playSound("PlayerWalk.wav", Window::PLAYER_WALK_CHANNEL);
        m_camera.move(movementVector, movAmt);
    }
    gunTransform.setPos(m_camera.getPos() +
        m_camera.getForward().normalize() * 0.105f);
    gunTransform.getPos().setY(gunTransform.getPos().getY() + GUN_OFFSET);

    Vector3f directionToCamera =
        Transform::getCamera().getPos() - gunTransform.getPos();
    float angleToFaceCamera =
        std::atan2(directionToCamera.getZ(), directionToCamera.getX()) *
        (180.0f / M_PI);
    if (directionToCamera.getX() < 0)

```

```

    angleToFaceCamera += 360;

    gunTransform.getRot().setY(angleToFaceCamera + 90);
}

void Player::render()
{
    Shader* shader = Game::getLevel()->getShader();

    shader->updateUniforms(gunTransform.getTransformation(),
                          gunTransform.getProjectedTransformation(),
                          *gunMaterial);

    m_mesh->draw();
}

Camera& Player::getCamera()
{
    return m_camera;
}

Player::Player(Player&& other) noexcept
    : m_camera(std::move(other.m_camera)), rd(), rand(std::move(other.rand)),
      m_health(other.m_health), centerPosition(std::move(other.centerPosition)),
      movementVector(std::move(other.movementVector))
{
}

Player& Player::operator=(Player&& other) noexcept
{
    if (this != &other)
    {
        m_camera = std::move(other.m_camera);
        m_health = other.m_health;
        centerPosition = std::move(other.centerPosition);
        movementVector = std::move(other.movementVector);

        rd();
        rand = std::move(other.rand);
    }
    return *this;
}

```

Sources/quaternion.cpp

```
#include "quaternion.h"
#include <math.h>

Quaternion::Quaternion(float _x, float _y, float _z, float _w)
{
    x = _x;
    y = _y;
    z = _z;
    w = _w;
}

void Quaternion::normalize()
{
    float length = sqrtf(x * x + y * y + z * z + w * w);

    x /= length;
    y /= length;
    z /= length;
    w /= length;
}

Quaternion Quaternion::conjugate()
{
    Quaternion ret(-x, -y, -z, w);
    return ret;
}

Quaternion Quaternion::operator*(const Quaternion& r)
{
    const float _w = (w * r.w) - (x * r.x) - (y * r.y) - (z * r.z);
    const float _x = (x * r.w) + (w * r.x) + (y * r.z) - (z * r.y);
    const float _y = (y * r.w) + (w * r.y) + (z * r.x) - (x * r.z);
    const float _z = (z * r.w) + (w * r.z) + (x * r.y) - (y * r.x);

    Quaternion ret(_x, _y, _z, _w);

    return ret;
}

Quaternion Quaternion::operator*(const Vector3f& v)
{
    const float _w = -(x * v.x) - (y * v.y) - (z * v.z);
    const float _x = (w * v.x) + (y * v.z) - (z * v.y);
    const float _y = (w * v.y) + (z * v.x) - (x * v.z);
    const float _z = (w * v.z) + (x * v.y) - (y * v.x);

    Quaternion ret(_x, _y, _z, _w);

    return ret;
}
```

Sources/renderUtil.cpp

```
#include "renderUtil.h"
#include <GL/glew.h>

void RenderUtil::clearScreen()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
}

void RenderUtil::initGraphics()
{
    glClearColor(0.0f, 0.0f, 0.0f, 0.0f);

    glFrontFace(GL_CW);
    glCullFace(GL_BACK);

    glEnable(GL_CULL_FACE);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_DEPTH_CLAMP);

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
}
```

Sources/sdl_requests.cpp

```
#include "sdl_requests.h"
#include <SDL2/SDL.h>

static SDL_Window* window;
static SDL_GLContext glContext;
static bool isCloseRequested = false;

bool SDLGetIsCloseRequested()
{
    return isCloseRequested;
}

void SDLSetIsCloseRequested(bool value)
{
    isCloseRequested = value;
}

void SDLCreateWindow(const char* title, int x, int y, int width, int height,
                    bool fullscreen)
{
    int mode = 0;

    if (fullscreen)
        mode = SDL_WINDOW_FULLSCREEN;

    window =
        SDL_CreateWindow(title, SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
                        width, height, SDL_WINDOW_OPENGL | mode);
    glContext = SDL_GL_CreateContext(window);
}

void SDLSetWindowFullscreen(bool value)
{
    int mode = 0;
    if (value)
        mode = SDL_WINDOW_FULLSCREEN;
    else
        mode = 0;

    SDL_SetWindowFullscreen(window, mode);
}

void SDLSwapBuffers()
{
    SDL_GL_SwapWindow(window);
}

void SDLDestroyWindow()
{
    SDL_GL_DeleteContext(glContext);
    SDL_DestroyWindow(window);
}

void SDLSetMousePosition(int x, int y)
{
    SDL_WarpMouseInWindow(window, x, y);
}
```

Sources/shader.cpp

```
#include "shader.h"
#include <GL/glew.h>
#include <cassert>
#include <cstdio>
#include <fstream>
#include <iostream>
#include <memory>
#include <vector>

static void checkShaderError(int shader, int flag, bool isProgram,
                             const std::string& errorMessage);
static std::shared_ptr<std::string> loadShader(const std::string& fileName);

Shader::Shader()
{
    m_program = glCreateProgram();

    if (m_program == 0)
    {
        fprintf(stderr, "Error creating shader program\n");
        exit(1);
    }
}

Shader::~Shader()
{
    for (std::vector<int>::iterator it = m_shaders.begin(); it != m_shaders.end();
        ++it)
    {
        glDetachShader(m_program, *it);
        glDeleteShader(*it);
    }
    glDeleteProgram(m_program);
}

void Shader::bind()
{
    glUseProgram(m_program);
}

void Shader::addUniform(const std::string& uniform)
{
    int location = glGetUniformLocation(m_program, uniform.c_str());

    assert(location != INVALID_VALUE);

    m_uniforms.emplace(uniform, location);
}

void Shader::addVertexShaderFromFile(const std::string& text)
{
    addVertexShader(*loadShader(text));
}

void Shader::addGeometryShaderFromFile(const std::string& text)
{
    addGeometryShader(*loadShader(text));
}

void Shader::addFragmentShaderFromFile(const std::string& text)
{
    addFragmentShader(*loadShader(text));
}

void Shader::addVertexShader(const std::string& text)
{
    addProgram(text, GL_VERTEX_SHADER);
}

void Shader::addGeometryShader(const std::string& text)
{
    addProgram(text, GL_GEOMETRY_SHADER);
}

void Shader::addFragmentShader(const std::string& text)
```

```

{
    addProgram(text, GL_FRAGMENT_SHADER);
}

void Shader::addProgram(const std::string& text, int type)
{
    int shader = glCreateShader(type);

    if (shader == 0)
    {
        fprintf(stderr, "Error creating shader type %d\n", type);
        exit(1);
    }

    const GLchar* p[1];
    p[0] = text.c_str();
    GLint lengths[1];
    lengths[0] = text.length(); // strlen(text.c_str());

    glShaderSource(shader, 1, p, lengths);
    glCompileShader(shader);

    GLint success;
    glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
    if (!success)
    {
        GLchar InfoLog[1024];

        glGetShaderInfoLog(shader, 1024, NULL, InfoLog);
        fprintf(stderr, "Error compiling shader type %d: '%s'\n", shader, InfoLog);

        exit(1);
    }

    glAttachShader(m_program, shader);
    m_shaders.push_back(shader);
}

void Shader::compileShader()
{
    glLinkProgram(m_program);
    checkShaderError(m_program, GL_LINK_STATUS, true,
        "Error linking shader program");

    glValidateProgram(m_program);
    checkShaderError(m_program, GL_VALIDATE_STATUS, true,
        "Invalid shader program");
}

void Shader::updateUniforms(const Matrix4f& worldMatrix,
    const Matrix4f& projectedMatrix, Material& material)
{
}

void Shader::setUniformi(const std::string& name, int value)
{
    glUniform1i(m_uniforms.at(name), value);
}

void Shader::setUniformf(const std::string& name, float value)
{
    glUniform1f(m_uniforms.at(name), value);
}

void Shader::setUniform(const std::string& name, const Vector3f& value)
{
    glUniform3f(m_uniforms.at(name), value.x, value.y, value.z);
}

void Shader::setUniform(const std::string& name, const Matrix4f& value)
{
    glUniformMatrix4fv(m_uniforms.at(name), 1, GL_TRUE, &(value[0][0]));
}

static void checkShaderError(int shader, int flag, bool isProgram,
    const std::string& errorMessage)
{

```

```

GLint success = 0;
GLchar error[1024] = {0};

if (isProgram)
    glGetProgramiv(shader, flag, &success);
else
    glGetShaderiv(shader, flag, &success);

if (!success)
{
    if (isProgram)
        glGetProgramInfoLog(shader, sizeof(error), NULL, error);
    else
        glGetShaderInfoLog(shader, sizeof(error), NULL, error);

    fprintf(stderr, "%s: '%s'\n", errorMessage.c_str(), error);
}
}

static std::shared_ptr<std::string> loadShader(const std::string& fileName)
{
    std::ifstream file;
    file.open(std::string(RESOURCE_PATH) + "/shader/" + fileName);

    std::shared_ptr<std::string> output(new std::string());
    std::string line;

    if (file.is_open())
    {
        while (file.good())
        {
            getline(file, line);
            output->append(line + "\n");
        }
    }
    else
    {
        std::cerr << "Unable to load shader: " << fileName << std::endl;
    }

    return output;
};

```


Sources/stb_image.cpp

```
/* stbi-1.33 - public domain JPEG/PNG reader - http://nothings.org/stb_image.c
   when you control the images you're loading
                                   no warranty implied; use at your own risk
```

QUICK NOTES:

Primarily of interest to game developers and other people who can
avoid problematic images and only need the trivial interface

JPEG baseline (no JPEG progressive)
PNG 8-bit-per-channel only

TGA (not sure what subset, if a subset)
BMP non-1bpp, non-RLE
PSD (composited view only, no extra channels)

GIF (*comp always reports as 4-channel)
HDR (radiance rgbE format)
PIC (Softimage PIC)

- decode from memory or through FILE (define STBI_NO_STDIO to remove code)
- decode from arbitrary I/O callbacks
- overridable dequantizing-IDCT, YCbCr-to-RGB conversion (define

STBI_SIMD)

Latest revisions:

1.33 (2011-07-14) minor fixes suggested by Dave Moore
1.32 (2011-07-13) info support for all filetypes (SpartanJ)
1.31 (2011-06-19) a few more leak fixes, bug in PNG handling (SpartanJ)
1.30 (2011-06-11) added ability to load files via io callbacks (Ben
Wenger) 1.29 (2010-08-16) various warning fixes from Aurelien Pocheville 1.28
(2010-08-01) fix bug in GIF palette transparency (SpartanJ) 1.27 (2010-08-01)
cast-to-uint8 to fix warnings (Laurent Gomila) allow trailing 0s at end of
image data (Laurent Gomila) 1.26 (2010-07-24) fix bug in file buffering for PNG
reported by SpartanJ

See end of file for full revision history.

TODO:

stbi_info support for BMP,PSD,HDR,PIC

```
===== Contributors =====

Image formats                                Optimizations & bugfixes
Sean Barrett (jpeg, png, bmp)              Fabian "ryg" Giesen
Nicolas Schulz (hdr, psd)
Jonathan Dummer (tga)                      Bug fixes & warning fixes
Jean-Marc Lienher (gif)                   Marc LeBlanc
Tom Seddon (pic)                           Christopher Lloyd
Thatcher Ulrich (psd)                     Dave Moore
                                           Won Chun
                                           the Horde3D community
Extensions, features                       Janez Zemva
Jetro Lauha (stbi_info)                   Jonathan Blow
James "moose2000" Brown (iPhone PNG)      Laurent Gomila
Ben "Disch" Wenger (io callbacks)         Aruelien Pocheville
Martin "SpartanJ" Golini                  Ryamond Barbiero
                                           David Woo
```

If your name should be here but isn't, let Sean know.

*/

```
#ifndef STBI_INCLUDE_STB_IMAGE_H
#define STBI_INCLUDE_STB_IMAGE_H
```

```
// To get a header file for this, either cut and paste the header,
// or create stb_image.h, #define STBI_HEADER_FILE_ONLY, and
// then include stb_image.c from it.
```

```
#include "stb_image.h"
```

```
#endif // STBI_INCLUDE_STB_IMAGE_H
```

```

#ifndef STBI_HEADER_FILE_ONLY

#ifndef STBI_NO_HDR
#include <math.h> // ldexp
#include <string.h> // strcmp, strtok
#endif

#ifndef STBI_NO_STDIO
#include <stdio.h>
#endif
#include <assert.h>
#include <memory.h>
#include <stdarg.h>
#include <stdlib.h>

#ifndef _MSC_VER
#ifdef __cplusplus
#define stbi_inline inline
#else
#define stbi_inline
#endif
#else
#define stbi_inline __forceinline
#endif

// implementation:
typedef unsigned char uint8;
typedef unsigned short uint16;
typedef signed short int16;
typedef unsigned int uint32;
typedef signed int int32;
typedef unsigned int uint;

// should produce compiler error if size is wrong
typedef unsigned char validate_uint32[sizeof(uint32) == 4 ? 1 : -1];

#if defined(STBI_NO_STDIO) && !defined(STBI_NO_WRITE)
#define STBI_NO_WRITE
#endif

#define STBI_NOTUSED(v) (void)sizeof(v)

#ifdef _MSC_VER
#define STBI_HAS_LR0TL
#endif

#ifdef STBI_HAS_LR0TL
#define stbi_lrot(x, y) _lrotl(x, y)
#else
#define stbi_lrot(x, y) (((x) << (y)) | ((x) >> (32 - (y))))
#endif

////////////////////

//
// stbi struct and start_xxx functions

// stbi structure is our basic context used by all images, so it
// contains all the IO context, plus some basic image information
typedef struct
{
    uint32 img_x, img_y;
    int img_n, img_out_n;

    stbi_io_callbacks io;
    void* io_user_data;

    int read_from_callbacks;
    int buflen;
    uint8 buffer_start[128];

    uint8 *img_buffer, *img_buffer_end;
    uint8* img_buffer_original;
} stbi;

static void refill_buffer(stbi* s);

// initialize a memory-decode context

```

```

static void start_mem(stbi* s, uint8 const* buffer, int len)
{
    s->io.read = NULL;
    s->read_from_callbacks = 0;
    s->img_buffer = s->img_buffer_original = (uint8*)buffer;
    s->img_buffer_end = (uint8*)buffer + len;
}

// initialize a callback-based context
static void start_callbacks(stbi* s, stbi_io_callbacks* c, void* user)
{
    s->io = *c;
    s->io_user_data = user;
    s->buflen = sizeof(s->buffer_start);
    s->read_from_callbacks = 1;
    s->img_buffer_original = s->buffer_start;
    refill_buffer(s);
}

#ifdef STBI_NO_STDIO

static int stdio_read(void* user, char* data, int size)
{
    return (int)fread(data, 1, size, (FILE*)user);
}

static void stdio_skip(void* user, unsigned n)
{
    fseek((FILE*)user, n, SEEK_CUR);
}

static int stdio_eof(void* user)
{
    return feof((FILE*)user);
}

static stbi_io_callbacks stbi_stdio_callbacks = {
    stdio_read,
    stdio_skip,
    stdio_eof,
};

static void start_file(stbi* s, FILE* f)
{
    start_callbacks(s, &stbi_stdio_callbacks, (void*)f);
}

// static void stop_file(stbi *s) { }

#endif // !STBI_NO_STDIO

static void stbi_rewind(stbi* s)
{
    // conceptually rewind SHOULD rewind to the beginning of the stream,
    // but we just rewind to the beginning of the initial buffer, because
    // we only use it after doing 'test', which only ever looks at at most 92
    // bytes
    s->img_buffer = s->img_buffer_original;
}

static int stbi_jpeg_test(stbi* s);
static stbi_uc* stbi_jpeg_load(stbi* s, int* x, int* y, int* comp,
                               int req_comp);
static int stbi_jpeg_info(stbi* s, int* x, int* y, int* comp);
static int stbi_png_test(stbi* s);
static stbi_uc* stbi_png_load(stbi* s, int* x, int* y, int* comp, int req_comp);
static int stbi_png_info(stbi* s, int* x, int* y, int* comp);
static int stbi_bmp_test(stbi* s);
static stbi_uc* stbi_bmp_load(stbi* s, int* x, int* y, int* comp, int req_comp);
static int stbi_tga_test(stbi* s);
static stbi_uc* stbi_tga_load(stbi* s, int* x, int* y, int* comp, int req_comp);
static int stbi_tga_info(stbi* s, int* x, int* y, int* comp);
static int stbi_psd_test(stbi* s);
static stbi_uc* stbi_psd_load(stbi* s, int* x, int* y, int* comp, int req_comp);
static int stbi_hdr_test(stbi* s);
static float* stbi_hdr_load(stbi* s, int* x, int* y, int* comp, int req_comp);
static int stbi_pic_test(stbi* s);

```

```

static stbi_uc* stbi_pic_load(stbi* s, int* x, int* y, int* comp, int req_comp);
static int stbi_gif_test(stbi* s);
static stbi_uc* stbi_gif_load(stbi* s, int* x, int* y, int* comp, int req_comp);
static int stbi_gif_info(stbi* s, int* x, int* y, int* comp);

// this is not threadsafe
static const char* failure_reason;

const char* stbi_failure_reason(void)
{
    return failure_reason;
}

static int e(const char* str)
{
    failure_reason = str;
    return 0;
}

// e - error
// epf - error returning pointer to float
// epuc - error returning pointer to unsigned char

#ifdef STBI_NO_FAILURE_STRINGS
#define e(x, y) 0
#elif defined(STBI_FAILURE_USERMSG)
#define e(x, y) e(y)
#else
#define e(x, y) e(x)
#endif

#define epf(x, y) ((float*)(e(x, y) ? NULL : NULL))
#define epuc(x, y) ((unsigned char*)(e(x, y) ? NULL : NULL))

void stbi_image_free(void* retval_from_stbi_load)
{
    free(retval_from_stbi_load);
}

#ifdef STBI_NO_HDR
static float* ldr_to_hdr(stbi_uc* data, int x, int y, int comp);
static stbi_uc* hdr_to_ldr(float* data, int x, int y, int comp);
#endif

static unsigned char* stbi_load_main(stbi* s, int* x, int* y, int* comp,
                                     int req_comp)
{
    {
        if (stbi_jpeg_test(s))
            return stbi_jpeg_load(s, x, y, comp, req_comp);
        if (stbi_png_test(s))
            return stbi_png_load(s, x, y, comp, req_comp);
        if (stbi_bmp_test(s))
            return stbi_bmp_load(s, x, y, comp, req_comp);
        if (stbi_gif_test(s))
            return stbi_gif_load(s, x, y, comp, req_comp);
        if (stbi_psd_test(s))
            return stbi_psd_load(s, x, y, comp, req_comp);
        if (stbi_pic_test(s))
            return stbi_pic_load(s, x, y, comp, req_comp);
    }

#ifdef STBI_NO_HDR
    if (stbi_hdr_test(s))
    {
        float* hdr = stbi_hdr_load(s, x, y, comp, req_comp);
        return hdr_to_ldr(hdr, *x, *y, req_comp ? req_comp : *comp);
    }
#endif

    // test tga last because it's a crappy test!
    if (stbi_tga_test(s))
        return stbi_tga_load(s, x, y, comp, req_comp);
    return epuc("unknown image type", "Image not of any known type, or corrupt");
}

#ifdef STBI_NO_STDIO
unsigned char* stbi_load(char const* filename, int* x, int* y, int* comp,
                        int req_comp)

```

```

{
    FILE* f = fopen(filename, "rb");
    unsigned char* result;
    if (!f)
        return epuc("can't fopen", "Unable to open file");
    result = stbi_load_from_file(f, x, y, comp, req_comp);
    fclose(f);
    return result;
}

unsigned char* stbi_load_from_file(FILE* f, int* x, int* y, int* comp,
                                   int req_comp)
{
    stbi s;
    start_file(&s, f);
    return stbi_load_main(&s, x, y, comp, req_comp);
}
#endif //! STBI_NO_STDIO

unsigned char* stbi_load_from_memory(stbi_uc const* buffer, int len, int* x,
                                     int* y, int* comp, int req_comp)
{
    stbi s;
    start_mem(&s, buffer, len);
    return stbi_load_main(&s, x, y, comp, req_comp);
}

unsigned char* stbi_load_from_callbacks(stbi_io_callbacks const* clbk,
                                       void* user, int* x, int* y, int* comp,
                                       int req_comp)
{
    stbi s;
    start_callbacks(&s, (stbi_io_callbacks*)clbk, user);
    return stbi_load_main(&s, x, y, comp, req_comp);
}

#ifdef STBI_NO_HDR

float* stbi_loadf_main(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    unsigned char* data;
#ifdef STBI_NO_HDR
    if (stbi_hdr_test(s))
        return stbi_hdr_load(s, x, y, comp, req_comp);
#endif
    data = stbi_load_main(s, x, y, comp, req_comp);
    if (data)
        return ldr_to_hdr(data, *x, *y, req_comp ? req_comp : *comp);
    return epf("unknown image type", "Image not of any known type, or corrupt");
}

float* stbi_loadf_from_memory(stbi_uc const* buffer, int len, int* x, int* y,
                             int* comp, int req_comp)
{
    stbi s;
    start_mem(&s, buffer, len);
    return stbi_loadf_main(&s, x, y, comp, req_comp);
}

float* stbi_loadf_from_callbacks(stbi_io_callbacks const* clbk, void* user,
                                int* x, int* y, int* comp, int req_comp)
{
    stbi s;
    start_callbacks(&s, (stbi_io_callbacks*)clbk, user);
    return stbi_loadf_main(&s, x, y, comp, req_comp);
}

#ifdef STBI_NO_STDIO

float* stbi_loadf(char const* filename, int* x, int* y, int* comp, int req_comp)
{
    FILE* f = fopen(filename, "rb");
    float* result;
    if (!f)
        return epf("can't fopen", "Unable to open file");
    result = stbi_loadf_from_file(f, x, y, comp, req_comp);
    fclose(f);
    return result;
}

```

```

}

float* stbi_loadf_from_file(FILE* f, int* x, int* y, int* comp, int req_comp)
{
    stbi s;
    start_file(&s, f);
    return stbi_loadf_main(&s, x, y, comp, req_comp);
}
#endif // !STBI_NO_STDIO

#endif // !STBI_NO_HDR

// these is-hdr-or-not is defined independent of whether STBI_NO_HDR is
// defined, for API simplicity; if STBI_NO_HDR is defined, it always
// reports false!

int stbi_is_hdr_from_memory(stbi_uc const* buffer, int len)
{
    #ifndef STBI_NO_HDR
        stbi s;
        start_mem(&s, buffer, len);
        return stbi_hdr_test(&s);
    #else
        STBI_NOTUSED(buffer);
        STBI_NOTUSED(len);
        return 0;
    #endif
}

#ifndef STBI_NO_STDIO
extern int stbi_is_hdr(char const* filename)
{
    FILE* f = fopen(filename, "rb");
    int result = 0;
    if (f)
    {
        result = stbi_is_hdr_from_file(f);
        fclose(f);
    }
    return result;
}

extern int stbi_is_hdr_from_file(FILE* f)
{
    #ifndef STBI_NO_HDR
        stbi s;
        start_file(&s, f);
        return stbi_hdr_test(&s);
    #else
        return 0;
    #endif
}
#endif // !STBI_NO_STDIO

extern int stbi_is_hdr_from_callbacks(stbi_io_callbacks const* clbk, void* user)
{
    #ifndef STBI_NO_HDR
        stbi s;
        start_callbacks(&s, (stbi_io_callbacks*)clbk, user);
        return stbi_hdr_test(&s);
    #else
        return 0;
    #endif
}

#ifndef STBI_NO_HDR
static float h2l_gamma_i = 1.0f / 2.2f, h2l_scale_i = 1.0f;
static float l2h_gamma = 2.2f, l2h_scale = 1.0f;

void stbi_hdr_to_ldr_gamma(float gamma)
{
    h2l_gamma_i = 1 / gamma;
}
void stbi_hdr_to_ldr_scale(float scale)
{
    h2l_scale_i = 1 / scale;
}

```

```

void stbi_ldr_to_hdr_gamma(float gamma)
{
    l2h_gamma = gamma;
}
void stbi_ldr_to_hdr_scale(float scale)
{
    l2h_scale = scale;
}
#endif

////////////////////////////////////
//
// Common code used by all image loaders
//

enum
{
    SCAN_load = 0,
    SCAN_type,
    SCAN_header
};

static void refill_buffer(stbi* s)
{
    int n = (s->io.read)(s->io_user_data, (char*)s->buffer_start, s->buflen);
    if (n == 0)
    {
        // at end of file, treat same as if from memory
        s->read_from_callbacks = 0;
        s->img_buffer = s->img_buffer_end - 1;
        *s->img_buffer = 0;
    }
    else
    {
        s->img_buffer = s->buffer_start;
        s->img_buffer_end = s->buffer_start + n;
    }
}

stbi_inline static int get8(stbi* s)
{
    if (s->img_buffer < s->img_buffer_end)
        return *s->img_buffer++;
    if (s->read_from_callbacks)
    {
        refill_buffer(s);
        return *s->img_buffer++;
    }
    return 0;
}

stbi_inline static int at_eof(stbi* s)
{
    if (s->io.read)
    {
        if (!(s->io.eof)(s->io_user_data))
            return 0;
        // if feof() is true, check if buffer = end
        // special case: we've only got the special 0 character at the end
        if (s->read_from_callbacks == 0)
            return 1;
    }

    return s->img_buffer >= s->img_buffer_end;
}

stbi_inline static uint8 get8u(stbi* s)
{
    return (uint8)get8(s);
}

static void skip(stbi* s, int n)
{
    if (s->io.read)
    {
        int blen = s->img_buffer_end - s->img_buffer;
        if (blen < n)

```

```

    {
        s->img_buffer = s->img_buffer_end;
        (s->io.skip)(s->io_user_data, n - blen);
        return;
    }
}
s->img_buffer += n;
}

static int getn(stbi* s, stbi_uc* buffer, int n)
{
    if (s->io.read)
    {
        int blen = s->img_buffer_end - s->img_buffer;
        if (blen < n)
        {
            int res, count;

            memcpy(buffer, s->img_buffer, blen);

            count = (s->io.read)(s->io_user_data, (char*)buffer + blen, n - blen);
            res = (count == (n - blen));
            s->img_buffer = s->img_buffer_end;
            return res;
        }
    }

    if (s->img_buffer + n <= s->img_buffer_end)
    {
        memcpy(buffer, s->img_buffer, n);
        s->img_buffer += n;
        return 1;
    }
    else
        return 0;
}

static int get16(stbi* s)
{
    int z = get8(s);
    return (z << 8) + get8(s);
}

static uint32 get32(stbi* s)
{
    uint32 z = get16(s);
    return (z << 16) + get16(s);
}

static int get16le(stbi* s)
{
    int z = get8(s);
    return z + (get8(s) << 8);
}

static uint32 get32le(stbi* s)
{
    uint32 z = get16le(s);
    return z + (get16le(s) << 16);
}

////////////////////////////////////
//
// generic converter from built-in img_n to req_comp
// individual types do this automatically as much as possible (e.g. jpeg
// does all cases internally since it needs to colorspace convert anyway,
// and it never has alpha, so very few cases ). png can automatically
// interleave an alpha=255 channel, but falls back to this for other cases
//
// assume data buffer is malloced, so malloc a new one and free that one
// only failure mode is malloc failing

static uint8 compute_y(int r, int g, int b)
{
    return (uint8)(((r * 77) + (g * 150) + (29 * b)) >> 8);
}

static unsigned char* convert_format(unsigned char* data, int img_n,

```



```

        int req_comp, uint x, uint y)
{
    int i, j;
    unsigned char* good;

    if (req_comp == img_n)
        return data;
    assert(req_comp >= 1 && req_comp <= 4);

    good = (unsigned char*)malloc(req_comp * x * y);
    if (good == NULL)
    {
        free(data);
        return epuc("outofmem", "Out of memory");
    }

    for (j = 0; j < (int)y; ++j)
    {
        unsigned char* src = data + j * x * img_n;
        unsigned char* dest = good + j * x * req_comp;

#define COMBO(a, b) ((a) * 8 + (b))
#define CASE(a, b) \
        case COMBO(a, b): \
            for (i = x - 1; i >= 0; --i, src += a, dest += b) \
                // convert source image with img_n components to one with req_comp \
                // components; avoid switch per pixel, so use switch per scanline and \
                // massive macros \
                switch (COMBO(img_n, req_comp)) \
                { \
                    CASE(1, 2) dest[0] = src[0], dest[1] = 255; \
                    break; \
                    CASE(1, 3) dest[0] = dest[1] = dest[2] = src[0]; \
                    break; \
                    CASE(1, 4) dest[0] = dest[1] = dest[2] = src[0], dest[3] = 255; \
                    break; \
                    CASE(2, 1) dest[0] = src[0]; \
                    break; \
                    CASE(2, 3) dest[0] = dest[1] = dest[2] = src[0]; \
                    break; \
                    CASE(2, 4) dest[0] = dest[1] = dest[2] = src[0], dest[3] = src[1]; \
                    break; \
                    CASE(3, 4) \
                    dest[0] = src[0], \
                    dest[1] = src[1], dest[2] = src[2], dest[3] = 255; \
                    break; \
                    CASE(3, 1) dest[0] = compute_y(src[0], src[1], src[2]); \
                    break; \
                    CASE(3, 2) dest[0] = compute_y(src[0], src[1], src[2]), dest[1] = 255; \
                    break; \
                    CASE(4, 1) dest[0] = compute_y(src[0], src[1], src[2]); \
                    break; \
                    CASE(4, 2) dest[0] = compute_y(src[0], src[1], src[2]), dest[1] = src[3]; \
                    break; \
                    CASE(4, 3) dest[0] = src[0], dest[1] = src[1], dest[2] = src[2]; \
                    break; \
                    default: \
                        assert(0); \
                } \
            #undef CASE
        }

    free(data);
    return good;
}

#ifdef STBI_NO_HDR
static float* ldr_to_hdr(stbi_uc* data, int x, int y, int comp)
{
    int i, k, n;
    float* output = (float*)malloc(x * y * comp * sizeof(float));
    if (output == NULL)
    {
        free(data);
        return epf("outofmem", "Out of memory");
    }
    // compute number of non-alpha components

```

```

    if (comp & 1)
        n = comp;
    else
        n = comp - 1;
    for (i = 0; i < x * y; ++i)
    {
        for (k = 0; k < n; ++k)
        {
            output[i * comp + k] =
                (float)pow(data[i * comp + k] / 255.0f, l2h_gamma) * l2h_scale;
        }
        if (k < comp)
            output[i * comp + k] = data[i * comp + k] / 255.0f;
    }
    free(data);
    return output;
}

#define float2int(x) ((int)(x))
static stbi_uc* hdr_to_ldr(float* data, int x, int y, int comp)
{
    int i, k, n;
    stbi_uc* output = (stbi_uc*)malloc(x * y * comp);
    if (output == NULL)
    {
        free(data);
        return epuc("outofmem", "Out of memory");
    }
    // compute number of non-alpha components
    if (comp & 1)
        n = comp;
    else
        n = comp - 1;
    for (i = 0; i < x * y; ++i)
    {
        for (k = 0; k < n; ++k)
        {
            float z =
                (float)pow(data[i * comp + k] * h2l_scale_i, h2l_gamma_i) * 255 +
                0.5f;
            if (z < 0)
                z = 0;
            if (z > 255)
                z = 255;
            output[i * comp + k] = (uint8)float2int(z);
        }
        if (k < comp)
        {
            float z = data[i * comp + k] * 255 + 0.5f;
            if (z < 0)
                z = 0;
            if (z > 255)
                z = 255;
            output[i * comp + k] = (uint8)float2int(z);
        }
    }
    free(data);
    return output;
}
#endif

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//
// "baseline" JPEG/JFIF decoder (not actually fully baseline implementation)
//
// simple implementation
//   - channel subsampling of at most 2 in each dimension
//   - doesn't support delayed output of y-dimension
//   - simple interface (only one output format: 8-bit interleaved RGB)
//   - doesn't try to recover corrupt jpegs
//   - doesn't allow partial loading, loading multiple at once
//   - still fast on x86 (copying globals into locals doesn't help x86)
//   - allocates lots of intermediate memory (full size of all components)
//     - non-interleaved case requires this anyway
//     - allows good upsampling (see next)
// high-quality
//   - upsampled channels are bilinearly interpolated, even across blocks

```

```

//      - quality integer IDCT derived from IJG's 'slow'
//      performance
//      - fast huffman; reasonable integer IDCT
//      - uses a lot of intermediate memory, could cache poorly
//      - load http://nothings.org/remote/anemones.jpg 3 times on 2.8Ghz P4
//          stb_jpeg: 1.34 seconds (MSVC6, default release build)
//          stb_jpeg: 1.06 seconds (MSVC6, processor = Pentium Pro)
//          IJL11.dll: 1.08 seconds (compiled by intel)
//          IJG 1998: 0.98 seconds (MSVC6, makefile provided by IJG)
//          IJG 1998: 0.95 seconds (MSVC6, makefile + proc=PPro)

```

```

// huffman decoding acceleration
#define FAST_BITS 9 // larger handles more cases; smaller stomps less cache

```

```

typedef struct
{
    uint8 fast[1 << FAST_BITS];
    // weirdly, repacking this into AoS is a 10% speed loss, instead of a win
    uint16 code[256];
    uint8 values[256];
    uint8 size[257];
    unsigned int maxcode[18];
    int delta[17]; // old 'firstsymbol' - old 'firstcode'
} huffman;

```

```

typedef struct
{
#ifdef STBI_SIMD
    unsigned short dequant2[4][64];
#endif
    stbi* s;
    huffman huff_dc[4];
    huffman huff_ac[4];
    uint8 dequant[4][64];

    // sizes for components, interleaved MCUs
    int img_h_max, img_v_max;
    int img_mcu_x, img_mcu_y;
    int img_mcu_w, img_mcu_h;

    // definition of jpeg image component
    struct
    {
        int id;
        int h, v;
        int tq;
        int hd, ha;
        int dc_pred;

        int x, y, w2, h2;
        uint8* data;
        void* raw_data;
        uint8* linebuf;
    } img_comp[4];

    uint32 code_buffer; // jpeg entropy-coded buffer
    int code_bits; // number of valid bits
    unsigned char marker; // marker seen while filling entropy buffer
    int nomore; // flag if we saw a marker so must stop

    int scan_n, order[4];
    int restart_interval, todo;
} jpeg;

```

```

static int build_huffman(huffman* h, int* count)
{
    int i, j, k = 0, code;
    // build size list for each symbol (from JPEG spec)
    for (i = 0; i < 16; ++i)
        for (j = 0; j < count[i]; ++j)
            h->size[k++] = (uint8)(i + 1);
    h->size[k] = 0;

    // compute actual symbols (from jpeg spec)
    code = 0;
    k = 0;
    for (j = 1; j <= 16; ++j)

```

```

{
    // compute delta to add to code to compute symbol id
    h->delta[j] = k - code;
    if (h->size[k] == j)
    {
        while (h->size[k] == j)
            h->code[k++] = (uint16)(code++);
        if (code - 1 >= (1 << j))
            return e("bad code lengths", "Corrupt JPEG");
    }
    // compute largest code + 1 for this size, preshifted as needed later
    h->maxcode[j] = code << (16 - j);
    code <= 1;
}
h->maxcode[j] = 0xffffffff;

// build non-spec acceleration table; 255 is flag for not-accelerated
memset(h->fast, 255, 1 << FAST_BITS);
for (i = 0; i < k; ++i)
{
    int s = h->size[i];
    if (s <= FAST_BITS)
    {
        int c = h->code[i] << (FAST_BITS - s);
        int m = 1 << (FAST_BITS - s);
        for (j = 0; j < m; ++j)
        {
            h->fast[c + j] = (uint8)i;
        }
    }
}
return 1;
}

static void grow_buffer_unsafe(jpeg* j)
{
    do
    {
        int b = j->nomore ? 0 : get8(j->s);
        if (b == 0xff)
        {
            int c = get8(j->s);
            if (c != 0)
            {
                j->marker = (unsigned char)c;
                j->nomore = 1;
                return;
            }
        }
        j->code_buffer |= b << (24 - j->code_bits);
        j->code_bits += 8;
    } while (j->code_bits <= 24);
}

// (1 << n) - 1
static uint32 bmask[17] = {0, 1, 3, 7, 15, 31, 63, 127, 255,
                           511, 1023, 2047, 4095, 8191, 16383, 32767, 65535};

// decode a jpeg huffman value from the bitstream
stbi_inline static int decode(jpeg* j, huffman* h)
{
    unsigned int temp;
    int c, k;

    if (j->code_bits < 16)
        grow_buffer_unsafe(j);

    // look at the top FAST_BITS and determine what symbol ID it is,
    // if the code is <= FAST_BITS
    c = (j->code_buffer >> (32 - FAST_BITS)) & ((1 << FAST_BITS) - 1);
    k = h->fast[c];
    if (k < 255)
    {
        int s = h->size[k];
        if (s > j->code_bits)
            return -1;
        j->code_buffer <= s;
    }

```

```

    j->code_bits -= s;
    return h->values[k];
}

// naive test is to shift the code_buffer down so k bits are
// valid, then test against maxcode. To speed this up, we've
// preshifted maxcode left so that it has (16-k) 0s at the
// end; in other words, regardless of the number of bits, it
// wants to be compared against something shifted to have 16;
// that way we don't need to shift inside the loop.
temp = j->code_buffer >> 16;
for (k = FAST_BITS + 1;; ++k)
    if (temp < h->maxcode[k])
        break;
if (k == 17)
{
    // error! code not found
    j->code_bits -= 16;
    return -1;
}

if (k > j->code_bits)
    return -1;

// convert the huffman code to the symbol id
c = ((j->code_buffer >> (32 - k)) & bmask[k]) + h->delta[k];
assert((((j->code_buffer) >> (32 - h->size[c])) & bmask[h->size[c]]) ==
        h->code[c]);

// convert the id to a symbol
j->code_bits -= k;
j->code_buffer <<= k;
return h->values[c];
}

// combined JPEG 'receive' and JPEG 'extend', since baseline
// always extends everything it receives.
stbi_inline static int extend_receive(jpeg* j, int n)
{
    unsigned int m = 1 << (n - 1);
    unsigned int k;
    if (j->code_bits < n)
        grow_buffer_unsafe(j);

#ifdef 1
    k = stbi_lrot(j->code_buffer, n);
    j->code_buffer = k & ~bmask[n];
    k &= bmask[n];
    j->code_bits -= n;
#else
    k = (j->code_buffer >> (32 - n)) & bmask[n];
    j->code_bits -= n;
    j->code_buffer <<= n;
#endif
    // the following test is probably a random branch that won't
    // predict well. I tried to table accelerate it but failed.
    // maybe it's compiling as a conditional move?
    if (k < m)
        return (-1 << n) + k + 1;
    else
        return k;
}

// given a value that's at position X in the zigzag stream,
// where does it appear in the 8x8 matrix coded as row-major?
static uint8 dezigzag[64 + 15] = {
    0, 1, 8, 16, 9, 2, 3, 10, 17, 24, 32, 25, 18, 11, 4, 5, 12, 19, 26, 33, 40,
    48, 41, 34, 27, 20, 13, 6, 7, 14, 21, 28, 35, 42, 49, 56, 57, 50, 43, 36,
    29, 22, 15, 23, 30, 37, 44, 51, 58, 59, 52, 45, 38, 31, 39, 46, 53, 60, 61,
    54, 47, 55, 62, 63,
    // let corrupt input sample past end
    63, 63, 63, 63, 63, 63, 63, 63, 63, 63, 63, 63, 63, 63, 63};

// decode one 64-entry block--
static int decode_block(jpeg* j, short data[64], huffman* hdc, huffman* hac,
    int b)
{

```

```

int diff, dc, k;
int t = decode(j, hdc);
if (t < 0)
    return e("bad huffman code", "Corrupt JPEG");

// 0 all the ac values now so we can do it 32-bits at a time
memset(data, 0, 64 * sizeof(data[0]));

diff = t ? extend_receive(j, t) : 0;
dc = j->img_comp[b].dc_pred + diff;
j->img_comp[b].dc_pred = dc;
data[0] = (short)dc;

// decode AC components, see JPEG spec
k = 1;
do
{
    int r, s;
    int rs = decode(j, hac);
    if (rs < 0)
        return e("bad huffman code", "Corrupt JPEG");
    s = rs & 15;
    r = rs >> 4;
    if (s == 0)
    {
        if (rs != 0xf0)
            break; // end block
        k += 16;
    }
    else
    {
        k += r;
        // decode into unzigzag'd location
        data[dezigzag[k++]] = (short)extend_receive(j, s);
    }
} while (k < 64);
return 1;
}

// take a -128..127 value and clamp it and convert to 0..255
stbi_inline static uint8 clamp(int x)
{
    // trick to use a single test to catch both cases
    if ((unsigned int)x > 255)
    {
        if (x < 0)
            return 0;
        if (x > 255)
            return 255;
    }
    return (uint8)x;
}

#define f2f(x) ((int)(((x) * 4096 + 0.5))
#define fsh(x) ((x) << 12)

// derived from jidctint -- DCT_ISLOW
#define IDCT_1D(s0, s1, s2, s3, s4, s5, s6, s7) \
int t0, t1, t2, t3, p1, p2, p3, p4, p5, x0, x1, x2, x3; \
p2 = s2; \
p3 = s6; \
p1 = (p2 + p3) * f2f(0.5411961f); \
t2 = p1 + p3 * f2f(-1.847759065f); \
t3 = p1 + p2 * f2f(0.765366865f); \
p2 = s0; \
p3 = s4; \
t0 = fsh(p2 + p3); \
t1 = fsh(p2 - p3); \
x0 = t0 + t3; \
x3 = t0 - t3; \
x1 = t1 + t2; \
x2 = t1 - t2; \
t0 = s7; \
t1 = s5; \
t2 = s3; \
t3 = s1; \
p3 = t0 + t2; \

```

```

p4 = t1 + t3; \
p1 = t0 + t3; \
p2 = t1 + t2; \
p5 = (p3 + p4) * f2f(1.175875602f); \
t0 = t0 * f2f(0.298631336f); \
t1 = t1 * f2f(2.053119869f); \
t2 = t2 * f2f(3.072711026f); \
t3 = t3 * f2f(1.501321110f); \
p1 = p5 + p1 * f2f(-0.899976223f); \
p2 = p5 + p2 * f2f(-2.562915447f); \
p3 = p3 * f2f(-1.961570560f); \
p4 = p4 * f2f(-0.390180644f); \
t3 += p1 + p4; \
t2 += p2 + p3; \
t1 += p2 + p4; \
t0 += p1 + p3; \

#ifdef STBI_SIMD
typedef unsigned short stbi_dequantize_t;
#else
typedef uint8 stbi_dequantize_t;
#endif

// .344 seconds on 3*anemones.jpg
static void idct_block(uint8* out, int out_stride, short data[64],
    stbi_dequantize_t* dequantize)
{
    int i, val[64], *v = val;
    stbi_dequantize_t* dq = dequantize;
    uint8* o;
    short* d = data;

    // columns
    for (i = 0; i < 8; ++i, ++d, ++dq, ++v)
    {
        // if all zeroes, shortcut -- this avoids dequantizing 0s and IDCTing
        if (d[8] == 0 && d[16] == 0 && d[24] == 0 && d[32] == 0 && d[40] == 0 &&
            d[48] == 0 && d[56] == 0)
        {
            // no shortcut          0 seconds
            // (1|2|3|4|5|6|7)==0    0 seconds
            // all separate          -0.047 seconds
            // 1 && 2|3 && 4|5 && 6|7: -0.047 seconds
            int dcterms = d[0] * dq[0] << 2;
            v[0] = v[8] = v[16] = v[24] = v[32] = v[40] = v[48] = v[56] = dcterms;
        }
        else
        {
            IDCT_1D(d[0] * dq[0], d[8] * dq[8], d[16] * dq[16], d[24] * dq[24],
                d[32] * dq[32], d[40] * dq[40], d[48] * dq[48], d[56] * dq[56])
            // constants scaled things up by 1<<12; let's bring them back
            // down, but keep 2 extra bits of precision
            x0 += 512;
            x1 += 512;
            x2 += 512;
            x3 += 512;
            v[0] = (x0 + t3) >> 10;
            v[56] = (x0 - t3) >> 10;
            v[8] = (x1 + t2) >> 10;
            v[48] = (x1 - t2) >> 10;
            v[16] = (x2 + t1) >> 10;
            v[40] = (x2 - t1) >> 10;
            v[24] = (x3 + t0) >> 10;
            v[32] = (x3 - t0) >> 10;
        }
    }
}

for (i = 0, v = val, o = out; i < 8; ++i, v += 8, o += out_stride)
{
    // no fast case since the first 1D IDCT spread components out
    IDCT_1D(v[0], v[1], v[2], v[3], v[4], v[5], v[6], v[7])
    // constants scaled things up by 1<<12, plus we had 1<<2 from first
    // loop, plus horizontal and vertical each scale by sqrt(8) so together
    // we've got an extra 1<<3, so 1<<17 total we need to remove.
    // so we want to round that, which means adding 0.5 * 1<<17,
    // aka 65536. Also, we'll end up with -128 to 127 that we want
    // to encode as 0..255 by adding 128, so we'll add that before the shift

```

```

    x0 += 65536 + (128 << 17);
    x1 += 65536 + (128 << 17);
    x2 += 65536 + (128 << 17);
    x3 += 65536 + (128 << 17);
    // tried computing the shifts into temps, or'ing the temps to see
    // if any were out of range, but that was slower
    o[0] = clamp((x0 + t3) >> 17);
    o[7] = clamp((x0 - t3) >> 17);
    o[1] = clamp((x1 + t2) >> 17);
    o[6] = clamp((x1 - t2) >> 17);
    o[2] = clamp((x2 + t1) >> 17);
    o[5] = clamp((x2 - t1) >> 17);
    o[3] = clamp((x3 + t0) >> 17);
    o[4] = clamp((x3 - t0) >> 17);
}
}

#ifdef STBI_SIMD
static stbi_idct_8x8 stbi_idct_installed = idct_block;

void stbi_install_idct(stbi_idct_8x8 func)
{
    stbi_idct_installed = func;
}
#endif

#define MARKER_none 0xff
// if there's a pending marker from the entropy stream, return that
// otherwise, fetch from the stream and get a marker. if there's no
// marker, return 0xff, which is never a valid marker value
static uint8 get_marker(jpeg* j)
{
    uint8 x;
    if (j->marker != MARKER_none)
    {
        x = j->marker;
        j->marker = MARKER_none;
        return x;
    }
    x = get8u(j->s);
    if (x != 0xff)
        return MARKER_none;
    while (x == 0xff)
        x = get8u(j->s);
    return x;
}

// in each scan, we'll have scan_n components, and the order
// of the components is specified by order[]
#define RESTART(x) ((x) >= 0xd0 && (x) <= 0xd7)

// after a restart interval, reset the entropy decoder and
// the dc prediction
static void reset(jpeg* j)
{
    j->code_bits = 0;
    j->code_buffer = 0;
    j->nomore = 0;
    j->img_comp[0].dc_pred = j->img_comp[1].dc_pred = j->img_comp[2].dc_pred = 0;
    j->marker = MARKER_none;
    j->todo = j->restart_interval ? j->restart_interval : 0xffffffff;
    // no more than 1<<31 MCUs if no restart_interval? that's plenty safe,
    // since we don't even allow 1<<30 pixels
}

static int parse_entropy_coded_data(jpeg* z)
{
    reset(z);
    if (z->scan_n == 1)
    {
        int i, j;
#ifdef STBI_SIMD
        __declspec(align(16))
#endif
        short data[64];
        int n = z->order[0];
        // non-interleaved data, we just need to process one block at a time,

```



```

// in trivial scanline order
// number of blocks to do just depends on how many actual "pixels" this
// component has, independent of interleaved MCU blocking and such
int w = (z->img_comp[n].x + 7) >> 3;
int h = (z->img_comp[n].y + 7) >> 3;
for (j = 0; j < h; ++j)
{
    for (i = 0; i < w; ++i)
    {
        if (!decode_block(z, data, z->huff_dc + z->img_comp[n].hd,
                          z->huff_ac + z->img_comp[n].ha, n))
            return 0;
#ifdef STBI_SIMD
        stbi_idct_installed(
            z->img_comp[n].data + z->img_comp[n].w2 * j * 8 + i * 8,
            z->img_comp[n].w2, data, z->dequant2[z->img_comp[n].tq]);
#else
        idct_block(z->img_comp[n].data + z->img_comp[n].w2 * j * 8 + i * 8,
                    z->img_comp[n].w2, data, z->dequant[z->img_comp[n].tq]);
#endif
        // every data block is an MCU, so countdown the restart interval
        if (--z->todo <= 0)
        {
            if (z->code_bits < 24)
                grow_buffer_unsafe(z);
            // if it's NOT a restart, then just bail, so we get corrupt data
            // rather than no data
            if (!RESTART(z->marker))
                return 1;
            reset(z);
        }
    }
}
}
else
{ // interleaved!
    int i, j, k, x, y;
    short data[64];
    for (j = 0; j < z->img_mcu_y; ++j)
    {
        for (i = 0; i < z->img_mcu_x; ++i)
        {
            // scan an interleaved mcu... process scan_n components in order
            for (k = 0; k < z->scan_n; ++k)
            {
                int n = z->order[k];
                // scan out an mcu's worth of this component; that's just determined
                // by the basic H and V specified for the component
                for (y = 0; y < z->img_comp[n].v; ++y)
                {
                    for (x = 0; x < z->img_comp[n].h; ++x)
                    {
                        int x2 = (i * z->img_comp[n].h + x) * 8;
                        int y2 = (j * z->img_comp[n].v + y) * 8;
                        if (!decode_block(z, data, z->huff_dc + z->img_comp[n].hd,
                                          z->huff_ac + z->img_comp[n].ha, n))
                            return 0;
#ifdef STBI_SIMD
                        stbi_idct_installed(
                            z->img_comp[n].data + z->img_comp[n].w2 * y2 + x2,
                            z->img_comp[n].w2, data, z->dequant2[z->img_comp[n].tq]);
#else
                        idct_block(z->img_comp[n].data + z->img_comp[n].w2 * y2 + x2,
                                    z->img_comp[n].w2, data,
                                    z->dequant[z->img_comp[n].tq]);
#endif
                    }
                }
            }
        }
    }
    // after all interleaved components, that's an interleaved MCU,
    // so now count down the restart interval
    if (--z->todo <= 0)
    {
        if (z->code_bits < 24)
            grow_buffer_unsafe(z);
        // if it's NOT a restart, then just bail, so we get corrupt data
        // rather than no data
    }
}

```

```

        if (!RESTART(z->marker))
            return 1;
        reset(z);
    }
}
}
return 1;
}

static int process_marker(jpeg* z, int m)
{
    int L;
    switch (m)
    {
    case MARKER_none: // no marker found
        return e("expected marker", "Corrupt JPEG");

    case 0xC2: // SOF - progressive
        return e("progressive jpeg", "JPEG format not supported (progressive)");

    case 0xDD: // DRI - specify restart interval
        if (get16(z->s) != 4)
            return e("bad DRI len", "Corrupt JPEG");
        z->restart_interval = get16(z->s);
        return 1;

    case 0xDB: // DQT - define quantization table
        L = get16(z->s) - 2;
        while (L > 0)
        {
            int q = get8(z->s);
            int p = q >> 4;
            int t = q & 15, i;
            if (p != 0)
                return e("bad DQT type", "Corrupt JPEG");
            if (t > 3)
                return e("bad DQT table", "Corrupt JPEG");
            for (i = 0; i < 64; ++i)
                z->dequant[t][dezigzag[i]] = get8u(z->s);
#ifdef STBI_SIMD
            for (i = 0; i < 64; ++i)
                z->dequant2[t][i] = z->dequant[t][i];
#endif
            L -= 65;
        }
        return L == 0;

    case 0xC4: // DHT - define huffman table
        L = get16(z->s) - 2;
        while (L > 0)
        {
            uint8* v;
            int sizes[16], i, m = 0;
            int q = get8(z->s);
            int tc = q >> 4;
            int th = q & 15;
            if (tc > 1 || th > 3)
                return e("bad DHT header", "Corrupt JPEG");
            for (i = 0; i < 16; ++i)
            {
                sizes[i] = get8(z->s);
                m += sizes[i];
            }
            L -= 17;
            if (tc == 0)
            {
                if (!build_huffman(z->huff_dc + th, sizes))
                    return 0;
                v = z->huff_dc[th].values;
            }
            else
            {
                if (!build_huffman(z->huff_ac + th, sizes))
                    return 0;
                v = z->huff_ac[th].values;
            }
        }
    }
}

```

```

        for (i = 0; i < m; ++i)
            v[i] = get8u(z->s);
        L -= m;
    }
    return L == 0;
}
// check for comment block or APP blocks
if ((m >= 0xE0 && m <= 0xEF) || m == 0xFE)
{
    skip(z->s, get16(z->s) - 2);
    return 1;
}
return 0;
}

```

// after we see SOS

```

static int process_scan_header(jpeg* z)
{
    int i;
    int Ls = get16(z->s);
    z->scan_n = get8(z->s);
    if (z->scan_n < 1 || z->scan_n > 4 || z->scan_n > (int)z->s->img_n)
        return e("bad SOS component count", "Corrupt JPEG");
    if (Ls != 6 + 2 * z->scan_n)
        return e("bad SOS len", "Corrupt JPEG");
    for (i = 0; i < z->scan_n; ++i)
    {
        int id = get8(z->s), which;
        int q = get8(z->s);
        for (which = 0; which < z->s->img_n; ++which)
            if (z->img_comp[which].id == id)
                break;
        if (which == z->s->img_n)
            return 0;
        z->img_comp[which].hd = q >> 4;
        if (z->img_comp[which].hd > 3)
            return e("bad DC huff", "Corrupt JPEG");
        z->img_comp[which].ha = q & 15;
        if (z->img_comp[which].ha > 3)
            return e("bad AC huff", "Corrupt JPEG");
        z->order[i] = which;
    }
    if (get8(z->s) != 0)
        return e("bad SOS", "Corrupt JPEG");
    get8(z->s); // should be 63, but might be 0
    if (get8(z->s) != 0)
        return e("bad SOS", "Corrupt JPEG");

    return 1;
}

```

```

static int process_frame_header(jpeg* z, int scan)
{
    stbi* s = z->s;
    int Lf, p, i, q, h_max = 1, v_max = 1, c;
    Lf = get16(s);
    if (Lf < 11)
        return e("bad SOF len", "Corrupt JPEG"); // JPEG
    p = get8(s);
    if (p != 8)
        return e("only 8-bit",
            "JPEG format not supported: 8-bit only"); // JPEG baseline
    s->img_y = get16(s);
    if (s->img_y == 0)
        return e(
            "no header height",
            "JPEG format not supported: delayed height"); // Legal, but we don't
                                                    // handle it--but neither
                                                    // does IJG

    s->img_x = get16(s);
    if (s->img_x == 0)
        return e("0 width", "Corrupt JPEG"); // JPEG requires
    c = get8(s);
    if (c != 3 && c != 1)
        return e("bad component count", "Corrupt JPEG"); // JFIF requires
    s->img_n = c;
    for (i = 0; i < c; ++i)

```

```

{
    z->img_comp[i].data = NULL;
    z->img_comp[i].linebuf = NULL;
}

if (Lf != 8 + 3 * s->img_n)
    return e("bad SOF len", "Corrupt JPEG");

for (i = 0; i < s->img_n; ++i)
{
    z->img_comp[i].id = get8(s);
    if (z->img_comp[i].id != i + 1) // JFIF requires
        if (z->img_comp[i].id !=
            i) // some version of jpegtran outputs non-JFIF-compliant files!
            return e("bad component ID", "Corrupt JPEG");
    q = get8(s);
    z->img_comp[i].h = (q >> 4);
    if (!z->img_comp[i].h || z->img_comp[i].h > 4)
        return e("bad H", "Corrupt JPEG");
    z->img_comp[i].v = q & 15;
    if (!z->img_comp[i].v || z->img_comp[i].v > 4)
        return e("bad V", "Corrupt JPEG");
    z->img_comp[i].tq = get8(s);
    if (z->img_comp[i].tq > 3)
        return e("bad TQ", "Corrupt JPEG");
}

if (scan != SCAN_load)
    return 1;

if ((1 << 30) / s->img_x / s->img_n < s->img_y)
    return e("too large", "Image too large to decode");

for (i = 0; i < s->img_n; ++i)
{
    if (z->img_comp[i].h > h_max)
        h_max = z->img_comp[i].h;
    if (z->img_comp[i].v > v_max)
        v_max = z->img_comp[i].v;
}

// compute interleaved mcu info
z->img_h_max = h_max;
z->img_v_max = v_max;
z->img_mcu_w = h_max * 8;
z->img_mcu_h = v_max * 8;
z->img_mcu_x = (s->img_x + z->img_mcu_w - 1) / z->img_mcu_w;
z->img_mcu_y = (s->img_y + z->img_mcu_h - 1) / z->img_mcu_h;

for (i = 0; i < s->img_n; ++i)
{
    // number of effective pixels (e.g. for non-interleaved MCU)
    z->img_comp[i].x = (s->img_x * z->img_comp[i].h + h_max - 1) / h_max;
    z->img_comp[i].y = (s->img_y * z->img_comp[i].v + v_max - 1) / v_max;
    // to simplify generation, we'll allocate enough memory to decode
    // the bogus oversized data from using interleaved MCUs and their
    // big blocks (e.g. a 16x16 iMCU on an image of width 33); we won't
    // discard the extra data until colorspace conversion
    z->img_comp[i].w2 = z->img_mcu_x * z->img_comp[i].h * 8;
    z->img_comp[i].h2 = z->img_mcu_y * z->img_comp[i].v * 8;
    z->img_comp[i].raw_data =
        malloc(z->img_comp[i].w2 * z->img_comp[i].h2 + 15);
    if (z->img_comp[i].raw_data == NULL)
    {
        for (--i; i >= 0; --i)
        {
            free(z->img_comp[i].raw_data);
            z->img_comp[i].data = NULL;
        }
        return e("outofmem", "Out of memory");
    }
    // align blocks for installable-idct using mmx/sse
    z->img_comp[i].data =
        (uint8*)(((size_t)z->img_comp[i].raw_data + 15) & ~15);
    z->img_comp[i].linebuf = NULL;
}
return 1;

```

```

}

// use comparisons since in some cases we handle more than one case (e.g. SOF)
#define DNL(x) ((x) == 0xdc)
#define SOI(x) ((x) == 0xd8)
#define EOI(x) ((x) == 0xd9)
#define SOF(x) ((x) == 0xc0 || (x) == 0xc1)
#define SOS(x) ((x) == 0xda)

static int decode_jpeg_header(jpeg* z, int scan)
{
    int m;
    z->marker = MARKER_none; // initialize cached marker to empty
    m = get_marker(z);
    if (!SOI(m))
        return e("no SOI", "Corrupt JPEG");
    if (scan == SCAN_type)
        return 1;
    m = get_marker(z);
    while (!SOF(m))
    {
        if (!process_marker(z, m))
            return 0;
        m = get_marker(z);
        while (m == MARKER_none)
        {
            // some files have extra padding after their blocks, so ok, we'll scan
            if (at_eof(z->s))
                return e("no SOF", "Corrupt JPEG");
            m = get_marker(z);
        }
    }
    if (!process_frame_header(z, scan))
        return 0;
    return 1;
}

static int decode_jpeg_image(jpeg* j)
{
    int m;
    j->restart_interval = 0;
    if (!decode_jpeg_header(j, SCAN_load))
        return 0;
    m = get_marker(j);
    while (!EOI(m))
    {
        if (SOS(m))
        {
            if (!process_scan_header(j))
                return 0;
            if (!parse_entropy_coded_data(j))
                return 0;
            if (j->marker == MARKER_none)
            {
                // handle 0s at the end of image data from IP Kamera 9060
                while (!at_eof(j->s))
                {
                    int x = get8(j->s);
                    if (x == 255)
                    {
                        j->marker = get8u(j->s);
                        break;
                    }
                    else if (x != 0)
                    {
                        return 0;
                    }
                }
                // if we reach eof without hitting a marker, get_marker() below will
                // fail and we'll eventually return 0
            }
        }
        else
        {
            if (!process_marker(j, m))
                return 0;
        }
    }
}

```

```

    m = get_marker(j);
}
return 1;
}

// static jfif-centered resampling (across block boundaries)

typedef uint8* (*resample_row_func)(uint8* out, uint8* in0, uint8* in1, int w,
                                     int hs);

#define div4(x) ((uint8)((x) >> 2))

static uint8* resample_row_1(uint8* out, uint8* in_near, uint8* in_far, int w,
                             int hs)
{
    STBI_NOTUSED(out);
    STBI_NOTUSED(in_far);
    STBI_NOTUSED(w);
    STBI_NOTUSED(hs);
    return in_near;
}

static uint8* resample_row_v_2(uint8* out, uint8* in_near, uint8* in_far, int w,
                               int hs)
{
    // need to generate two samples vertically for every one in input
    int i;
    STBI_NOTUSED(hs);
    for (i = 0; i < w; ++i)
        out[i] = div4(3 * in_near[i] + in_far[i] + 2);
    return out;
}

static uint8* resample_row_h_2(uint8* out, uint8* in_near, uint8* in_far, int w,
                               int hs)
{
    // need to generate two samples horizontally for every one in input
    int i;
    uint8* input = in_near;

    if (w == 1)
    {
        // if only one sample, can't do any interpolation
        out[0] = out[1] = input[0];
        return out;
    }

    out[0] = input[0];
    out[1] = div4(input[0] * 3 + input[1] + 2);
    for (i = 1; i < w - 1; ++i)
    {
        int n = 3 * input[i] + 2;
        out[i * 2 + 0] = div4(n + input[i - 1]);
        out[i * 2 + 1] = div4(n + input[i + 1]);
    }
    out[i * 2 + 0] = div4(input[w - 2] * 3 + input[w - 1] + 2);
    out[i * 2 + 1] = input[w - 1];

    STBI_NOTUSED(in_far);
    STBI_NOTUSED(hs);

    return out;
}

#define div16(x) ((uint8)((x) >> 4))

static uint8* resample_row_hv_2(uint8* out, uint8* in_near, uint8* in_far,
                                 int w, int hs)
{
    // need to generate 2x2 samples for every one in input
    int i, t0, t1;
    if (w == 1)
    {
        out[0] = out[1] = div4(3 * in_near[0] + in_far[0] + 2);
        return out;
    }
    t1 = 3 * in_near[0] + in_far[0];

```

```

out[0] = div4(t1 + 2);
for (i = 1; i < w; ++i)
{
    t0 = t1;
    t1 = 3 * in_near[i] + in_far[i];
    out[i * 2 - 1] = div16(3 * t0 + t1 + 8);
    out[i * 2] = div16(3 * t1 + t0 + 8);
}
out[w * 2 - 1] = div4(t1 + 2);

STBI_NOTUSED(hs);

return out;
}

static uint8* resample_row_generic(uint8* out, uint8* in_near, uint8* in_far,
                                  int w, int hs)
{
    // resample with nearest-neighbor
    int i, j;
    in_far = in_near;
    for (i = 0; i < w; ++i)
        for (j = 0; j < hs; ++j)
            out[i * hs + j] = in_near[i];
    return out;
}

#define float2fixed(x) ((int)((x) * 65536 + 0.5))

// 0.38 seconds on 3*anemones.jpg (0.25 with processor = Pro)
// VC6 without processor=Pro is generating multiple LEAs per multiply!
static void YCbCr_to_RGB_row(uint8* out, const uint8* y, const uint8* pcb,
                             const uint8* pcr, int count, int step)
{
    int i;
    for (i = 0; i < count; ++i)
    {
        int y_fixed = (y[i] << 16) + 32768; // rounding
        int r, g, b;
        int cr = pcr[i] - 128;
        int cb = pcb[i] - 128;
        r = y_fixed + cr * float2fixed(1.40200f);
        g = y_fixed - cr * float2fixed(0.71414f) - cb * float2fixed(0.34414f);
        b = y_fixed + cb * float2fixed(1.77200f);
        r >>= 16;
        g >>= 16;
        b >>= 16;
        if ((unsigned)r > 255)
        {
            if (r < 0)
                r = 0;
            else
                r = 255;
        }
        if ((unsigned)g > 255)
        {
            if (g < 0)
                g = 0;
            else
                g = 255;
        }
        if ((unsigned)b > 255)
        {
            if (b < 0)
                b = 0;
            else
                b = 255;
        }
        out[0] = (uint8)r;
        out[1] = (uint8)g;
        out[2] = (uint8)b;
        out[3] = 255;
        out += step;
    }
}

#ifdef STBI_SIMD

```

```

static stbi_YCbCr_to_RGB_run stbi_YCbCr_installed = YCbCr_to_RGB_row;

void stbi_install_YCbCr_to_RGB(stbi_YCbCr_to_RGB_run func)
{
    stbi_YCbCr_installed = func;
}
#endif

// clean up the temporary component buffers
static void cleanup_jpeg(jpeg* j)
{
    int i;
    for (i = 0; i < j->s->img_n; ++i)
    {
        if (j->img_comp[i].data)
        {
            free(j->img_comp[i].raw_data);
            j->img_comp[i].data = NULL;
        }
        if (j->img_comp[i].linebuf)
        {
            free(j->img_comp[i].linebuf);
            j->img_comp[i].linebuf = NULL;
        }
    }
}

typedef struct
{
    resample_row_func resample;
    uint8 *line0, *line1;
    int hs, vs; // expansion factor in each axis
    int w_lores; // horizontal pixels pre-expansion
    int ystep; // how far through vertical expansion we are
    int ypos; // which pre-expansion row we're on
} stbi_resample;

static uint8* load_jpeg_image(jpeg* z, int* out_x, int* out_y, int* comp,
                             int req_comp)
{
    int n, decode_n;
    // validate req_comp
    if (req_comp < 0 || req_comp > 4)
        return epuc("bad req_comp", "Internal error");
    z->s->img_n = 0;

    // load a jpeg image from whichever source
    if (!decode_jpeg_image(z))
    {
        cleanup_jpeg(z);
        return NULL;
    }

    // determine actual number of components to generate
    n = req_comp ? req_comp : z->s->img_n;

    if (z->s->img_n == 3 && n < 3)
        decode_n = 1;
    else
        decode_n = z->s->img_n;

    // resample and color-convert
    {
        int k;
        uint i, j;
        uint8* output;
        uint8* coutput[4];

        stbi_resample res_comp[4];

        for (k = 0; k < decode_n; ++k)
        {
            stbi_resample* r = &res_comp[k];

            // allocate line buffer big enough for upsampling off the edges
            // with upsample factor of 4
            z->img_comp[k].linebuf = (uint8*)malloc(z->s->img_x + 3);
        }
    }
}

```



```

if (!z->img_comp[k].linebuf)
{
    cleanup_jpeg(z);
    return epuc("outofmem", "Out of memory");
}

r->hs = z->img_h_max / z->img_comp[k].h;
r->vs = z->img_v_max / z->img_comp[k].v;
r->ystep = r->vs >> 1;
r->w_lores = (z->s->img_x + r->hs - 1) / r->hs;
r->ypos = 0;
r->line0 = r->line1 = z->img_comp[k].data;

if (r->hs == 1 && r->vs == 1)
    r->resample = resample_row_1;
else if (r->hs == 1 && r->vs == 2)
    r->resample = resample_row_v_2;
else if (r->hs == 2 && r->vs == 1)
    r->resample = resample_row_h_2;
else if (r->hs == 2 && r->vs == 2)
    r->resample = resample_row_hv_2;
else
    r->resample = resample_row_generic;
}

// can't error after this so, this is safe
output = (uint8*)malloc(n * z->s->img_x * z->s->img_y + 1);
if (!output)
{
    cleanup_jpeg(z);
    return epuc("outofmem", "Out of memory");
}

// now go ahead and resample
for (j = 0; j < z->s->img_y; ++j)
{
    uint8* out = output + n * z->s->img_x * j;
    for (k = 0; k < decode_n; ++k)
    {
        stbi_resample* r = &res_comp[k];
        int y_bot = r->ystep >= (r->vs >> 1);
        coutput[k] =
            r->resample(z->img_comp[k].linebuf, y_bot ? r->line1 : r->line0,
                y_bot ? r->line0 : r->line1, r->w_lores, r->hs);
        if (++r->ystep >= r->vs)
        {
            r->ystep = 0;
            r->line0 = r->line1;
            if (++r->ypos < z->img_comp[k].y)
                r->line1 += z->img_comp[k].w2;
        }
    }
    if (n >= 3)
    {
        uint8* y = coutput[0];
        if (z->s->img_n == 3)
        {
#ifdef STBI_SIMD
            stbi_YCbCr_installed(out, y, coutput[1], coutput[2], z->s->img_x, n);
#else
            YCbCr_to_RGB_row(out, y, coutput[1], coutput[2], z->s->img_x, n);
#endif
        }
        else
        {
            for (i = 0; i < z->s->img_x; ++i)
            {
                out[0] = out[1] = out[2] = y[i];
                out[3] = 255; // not used if n==3
                out += n;
            }
        }
    }
    else
    {
        uint8* y = coutput[0];
        if (n == 1)
            for (i = 0; i < z->s->img_x; ++i)
                out[i] = y[i];
    }
}

```

```

        else
            for (i = 0; i < z->s->img_x; ++i)
                *out++ = y[i], *out++ = 255;
    }
}
cleanup_jpeg(z);
*out_x = z->s->img_x;
*out_y = z->s->img_y;
if (comp)
    *comp = z->s->img_n; // report original components, not output
return output;
}
}

static unsigned char* stbi_jpeg_load(stbi* s, int* x, int* y, int* comp,
                                     int req_comp)
{
    jpeg j;
    j.s = s;
    return load_jpeg_image(&j, x, y, comp, req_comp);
}

static int stbi_jpeg_test(stbi* s)
{
    int r;
    jpeg j;
    j.s = s;
    r = decode_jpeg_header(&j, SCAN_type);
    stbi_rewind(s);
    return r;
}

static int stbi_jpeg_info_raw(jpeg* j, int* x, int* y, int* comp)
{
    if (!decode_jpeg_header(j, SCAN_header))
    {
        stbi_rewind(j->s);
        return 0;
    }
    if (x)
        *x = j->s->img_x;
    if (y)
        *y = j->s->img_y;
    if (comp)
        *comp = j->s->img_n;
    return 1;
}

static int stbi_jpeg_info(stbi* s, int* x, int* y, int* comp)
{
    jpeg j;
    j.s = s;
    return stbi_jpeg_info_raw(&j, x, y, comp);
}

// public domain zlib decode    v0.2  Sean Barrett 2006-11-18
//   simple implementation
//   - all input must be provided in an upfront buffer
//   - all output is written to a single output buffer (can malloc/realloc)
//   performance
//   - fast huffman

// fast-way is faster to check than jpeg huffman, but slow way is slower
#define ZFAST_BITS 9 // accelerate all cases in default tables
#define ZFAST_MASK ((1 << ZFAST_BITS) - 1)

// zlib-style huffman encoding
// (jpegs packs from left, zlib from right, so can't share code)
typedef struct
{
    uint16 fast[1 << ZFAST_BITS];
    uint16 firstcode[16];
    int maxcode[17];
    uint16 firstsymbol[16];
    uint8 size[288];
    uint16 value[288];
} zhuffman;

```

```

stbi_inline static int bitreverse16(int n)
{
    n = ((n & 0xAAAA) >> 1) | ((n & 0x5555) << 1);
    n = ((n & 0xCCCC) >> 2) | ((n & 0x3333) << 2);
    n = ((n & 0xF0F0) >> 4) | ((n & 0x0F0F) << 4);
    n = ((n & 0xFF00) >> 8) | ((n & 0x00FF) << 8);
    return n;
}

stbi_inline static int bit_reverse(int v, int bits)
{
    assert(bits <= 16);
    // to bit reverse n bits, reverse 16 and shift
    // e.g. 11 bits, bit reverse and shift away 5
    return bitreverse16(v) >> (16 - bits);
}

static int zbuild_huffman(zhuffman* z, uint8* sizelist, int num)
{
    int i, k = 0;
    int code, next_code[16], sizes[17];

    // DEFLATE spec for generating codes
    memset(sizes, 0, sizeof(sizes));
    memset(z->fast, 255, sizeof(z->fast));
    for (i = 0; i < num; ++i)
        ++sizes[sizelist[i]];
    sizes[0] = 0;
    for (i = 1; i < 16; ++i)
        assert(sizes[i] <= (1 << i));
    code = 0;
    for (i = 1; i < 16; ++i)
    {
        next_code[i] = code;
        z->firstcode[i] = (uint16)code;
        z->firstsymbol[i] = (uint16)k;
        code = (code + sizes[i]);
        if (sizes[i])
            if (code - 1 >= (1 << i))
                return e("bad codelengths", "Corrupt JPEG");
        z->maxcode[i] = code << (16 - i); // preshift for inner loop
        code <<= 1;
        k += sizes[i];
    }
    z->maxcode[16] = 0x10000; // sentinel
    for (i = 0; i < num; ++i)
    {
        int s = sizelist[i];
        if (s)
        {
            int c = next_code[s] - z->firstcode[s] + z->firstsymbol[s];
            z->size[c] = (uint8)s;
            z->value[c] = (uint16)i;
            if (s <= ZFAST_BITS)
            {
                int k = bit_reverse(next_code[s], s);
                while (k < (1 << ZFAST_BITS))
                {
                    z->fast[k] = (uint16)c;
                    k += (1 << s);
                }
            }
            ++next_code[s];
        }
    }
    return 1;
}

// zlib-from-memory implementation for PNG reading
//   because PNG allows splitting the zlib stream arbitrarily,
//   and it's annoying structurally to have PNG call ZLIB call PNG,
//   we require PNG read all the IDATs and combine them into a single
//   memory buffer

```

```

typedef struct
{
    uint8 *zbuffer, *zbuffer_end;

```

```

int num_bits;
uint32 code_buffer;

char* zout;
char* zout_start;
char* zout_end;
int z_expandable;

zhuffman z_length, z_distance;
} zbuf;

stbi_inline static int zget8(zbuf* z)
{
    if (z->zbuffer >= z->zbuffer_end)
        return 0;
    return *z->zbuffer++;
}

static void fill_bits(zbuf* z)
{
    do
    {
        assert(z->code_buffer < (1U << z->num_bits));
        z->code_buffer |= zget8(z) << z->num_bits;
        z->num_bits += 8;
    } while (z->num_bits <= 24);
}

stbi_inline static unsigned int zreceive(zbuf* z, int n)
{
    unsigned int k;
    if (z->num_bits < n)
        fill_bits(z);
    k = z->code_buffer & ((1 << n) - 1);
    z->code_buffer >>= n;
    z->num_bits -= n;
    return k;
}

stbi_inline static int zhuffman_decode(zbuf* a, zhuffman* z)
{
    int b, s, k;
    if (a->num_bits < 16)
        fill_bits(a);
    b = z->fast[a->code_buffer & ZFAST_MASK];
    if (b < 0xffff)
    {
        s = z->size[b];
        a->code_buffer >>= s;
        a->num_bits -= s;
        return z->value[b];
    }

    // not resolved by fast table, so compute it the slow way
    // use jpeg approach, which requires MSbits at top
    k = bit_reverse(a->code_buffer, 16);
    for (s = ZFAST_BITS + 1; ++s)
        if (k < z->maxcode[s])
            break;
    if (s == 16)
        return -1; // invalid code!
    // code size is s, so:
    b = (k >> (16 - s)) - z->firstcode[s] + z->firstsymbol[s];
    assert(z->size[b] == s);
    a->code_buffer >>= s;
    a->num_bits -= s;
    return z->value[b];
}

static int expand(zbuf* z, int n) // need to make room for n bytes
{
    char* q;
    int cur, limit;
    if (!z->z_expandable)
        return e("output buffer limit", "Corrupt PNG");
    cur = (int)(z->zout - z->zout_start);
    limit = (int)(z->zout_end - z->zout_start);

```

```

while (cur + n > limit)
    limit *= 2;
q = (char*)realloc(z->zout_start, limit);
if (q == NULL)
    return e("outofmem", "Out of memory");
z->zout_start = q;
z->zout = q + cur;
z->zout_end = q + limit;
return 1;
}

static int length_base[31] = {3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 15,
                             17, 19, 23, 27, 31, 35, 43, 51, 59, 67, 83,
                             99, 115, 131, 163, 195, 227, 258, 0, 0};

static int length_extra[31] = {0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2,
                               3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 0, 0, 0};

static int dist_base[32] = {1, 2, 3, 4, 5, 7, 9, 13,
                            17, 25, 33, 49, 65, 97, 129, 193,
                            257, 385, 513, 769, 1025, 1537, 2049, 3073,
                            4097, 6145, 8193, 12289, 16385, 24577, 0, 0};

static int dist_extra[32] = {0, 0, 0, 0, 1, 1, 2, 2, 3, 3,
                             4, 4, 5, 5, 6, 6, 7, 7, 8, 8,
                             9, 9, 10, 10, 11, 11, 12, 12, 13, 13};

static int parse_huffman_block(zbuf* a)
{
    for (;;)
    {
        int z = zhuffman_decode(a, &a->z_length);
        if (z < 256)
        {
            if (z < 0)
                return e("bad huffman code", "Corrupt PNG"); // error in huffman codes
            if (a->zout >= a->zout_end)
                if (!expand(a, 1))
                    return 0;
            *a->zout++ = (char)z;
        }
        else
        {
            uint8* p;
            int len, dist;
            if (z == 256)
                return 1;
            z -= 257;
            len = length_base[z];
            if (length_extra[z])
                len += zreceive(a, length_extra[z]);
            z = zhuffman_decode(a, &a->z_distance);
            if (z < 0)
                return e("bad huffman code", "Corrupt PNG");
            dist = dist_base[z];
            if (dist_extra[z])
                dist += zreceive(a, dist_extra[z]);
            if (a->zout - a->zout_start < dist)
                return e("bad dist", "Corrupt PNG");
            if (a->zout + len > a->zout_end)
                if (!expand(a, len))
                    return 0;
            p = (uint8*)(a->zout - dist);
            while (len--)
                *a->zout++ = *p++;
        }
    }
}

static int compute_huffman_codes(zbuf* a)
{
    static uint8 length_dezigzag[19] = {16, 17, 18, 0, 8, 7, 9, 6, 10, 5,
                                         11, 4, 12, 3, 13, 2, 14, 1, 15};

    zhuffman z_codelength;
    uint8 lencodes[286 + 32 + 137]; // padding for maximum single op
    uint8 codelength_sizes[19];
    int i, n;

```

```

int hlit = zreceive(a, 5) + 257;
int hdist = zreceive(a, 5) + 1;
int hclen = zreceive(a, 4) + 4;

memset(codelength_sizes, 0, sizeof(codelength_sizes));
for (i = 0; i < hclen; ++i)
{
    int s = zreceive(a, 3);
    codelength_sizes[length_dezigzag[i]] = (uint8)s;
}
if (!zbuild_huffman(&z_codelength, codelength_sizes, 19))
    return 0;

n = 0;
while (n < hlit + hdist)
{
    int c = zhuffman_decode(a, &z_codelength);
    assert(c >= 0 && c < 19);
    if (c < 16)
        lencodes[n++] = (uint8)c;
    else if (c == 16)
    {
        c = zreceive(a, 2) + 3;
        memset(lencodes + n, lencodes[n - 1], c);
        n += c;
    }
    else if (c == 17)
    {
        c = zreceive(a, 3) + 3;
        memset(lencodes + n, 0, c);
        n += c;
    }
    else
    {
        assert(c == 18);
        c = zreceive(a, 7) + 11;
        memset(lencodes + n, 0, c);
        n += c;
    }
}
if (n != hlit + hdist)
    return e("bad codelengths", "Corrupt PNG");
if (!zbuild_huffman(&a->z_length, lencodes, hlit))
    return 0;
if (!zbuild_huffman(&a->z_distance, lencodes + hlit, hdist))
    return 0;
return 1;
}

static int parse_uncompressed_block(zbuf* a)
{
    uint8 header[4];
    int len, nlen, k;
    if (a->num_bits & 7)
        zreceive(a, a->num_bits & 7); // discard
    // drain the bit-packed data into header
    k = 0;
    while (a->num_bits > 0)
    {
        header[k++] = (uint8)(a->code_buffer & 255); // wtf this warns?
        a->code_buffer >>= 8;
        a->num_bits -= 8;
    }
    assert(a->num_bits == 0);
    // now fill header the normal way
    while (k < 4)
        header[k++] = (uint8)zget8(a);
    len = header[1] * 256 + header[0];
    nlen = header[3] * 256 + header[2];
    if (nlen != (len ^ 0xffff))
        return e("zlib corrupt", "Corrupt PNG");
    if (a->zbuffer + len > a->zbuffer_end)
        return e("read past buffer", "Corrupt PNG");
    if (a->zout + len > a->zout_end)
        if (!expand(a, len))
            return 0;
    memcpy(a->zout, a->zbuffer, len);

```

```

a->zbuffer += len;
a->zout += len;
return 1;
}

static int parse_zlib_header(zbuf* a)
{
    int cmf = zget8(a);
    int cm = cmf & 15;
    /* int cinfo = cmf >> 4; */
    int flg = zget8(a);
    if ((cmf * 256 + flg) % 31 != 0)
        return e("bad zlib header", "Corrupt PNG"); // zlib spec
    if (flg & 32)
        return e("no preset dict",
            "Corrupt PNG"); // preset dictionary not allowed in png
    if (cm != 8)
        return e("bad compression", "Corrupt PNG"); // DEFLATE required for png
    // window = 1 << (8 + cinfo)... but who cares, we fully buffer output
    return 1;
}

// @TODO: should statically initialize these for optimal thread safety
static uint8 default_length[288], default_distance[32];
static void init_defaults(void)
{
    int i; // use <= to match clearly with spec
    for (i = 0; i <= 143; ++i)
        default_length[i] = 8;
    for (; i <= 255; ++i)
        default_length[i] = 9;
    for (; i <= 279; ++i)
        default_length[i] = 7;
    for (; i <= 287; ++i)
        default_length[i] = 8;

    for (i = 0; i <= 31; ++i)
        default_distance[i] = 5;
}

int stbi_png_partial; // a quick hack to only allow decoding some of a PNG... I
// should implement real streaming support instead
static int parse_zlib(zbuf* a, int parse_header)
{
    int final, type;
    if (parse_header)
        if (!parse_zlib_header(a))
            return 0;
    a->num_bits = 0;
    a->code_buffer = 0;
    do
    {
        final = zreceive(a, 1);
        type = zreceive(a, 2);
        if (type == 0)
        {
            if (!parse_uncompressed_block(a))
                return 0;
        }
        else if (type == 3)
        {
            return 0;
        }
        else
        {
            if (type == 1)
            {
                // use fixed code lengths
                if (!default_distance[31])
                    init_defaults();
                if (!zbuild_huffman(&a->z_length, default_length, 288))
                    return 0;
                if (!zbuild_huffman(&a->z_distance, default_distance, 32))
                    return 0;
            }
            else
            {

```

```

        if (!compute_huffman_codes(a))
            return 0;
    }
    if (!parse_huffman_block(a))
        return 0;
}
if (stbi_png_partial && a->zout - a->zout_start > 65536)
    break;
} while (!final);
return 1;
}

static int do_zlib(zbuf* a, char* obuf, int olen, int exp, int parse_header)
{
    a->zout_start = obuf;
    a->zout = obuf;
    a->zout_end = obuf + olen;
    a->z_expandable = exp;

    return parse_zlib(a, parse_header);
}

char* stbi_zlib_decode_malloc_guesssize(const char* buffer, int len,
                                       int initial_size, int* outlen)
{
    zbuf a;
    char* p = (char*)malloc(initial_size);
    if (p == NULL)
        return NULL;
    a.zbuffer = (uint8*)buffer;
    a.zbuffer_end = (uint8*)buffer + len;
    if (do_zlib(&a, p, initial_size, 1, 1))
    {
        if (outlen)
            *outlen = (int)(a.zout - a.zout_start);
        return a.zout_start;
    }
    else
    {
        free(a.zout_start);
        return NULL;
    }
}

char* stbi_zlib_decode_malloc(char const* buffer, int len, int* outlen)
{
    return stbi_zlib_decode_malloc_guesssize(buffer, len, 16384, outlen);
}

char* stbi_zlib_decode_malloc_guesssize_headerflag(const char* buffer, int len,
                                                    int initial_size,
                                                    int* outlen,
                                                    int parse_header)
{
    zbuf a;
    char* p = (char*)malloc(initial_size);
    if (p == NULL)
        return NULL;
    a.zbuffer = (uint8*)buffer;
    a.zbuffer_end = (uint8*)buffer + len;
    if (do_zlib(&a, p, initial_size, 1, parse_header))
    {
        if (outlen)
            *outlen = (int)(a.zout - a.zout_start);
        return a.zout_start;
    }
    else
    {
        free(a.zout_start);
        return NULL;
    }
}

int stbi_zlib_decode_buffer(char* obuffer, int olen, char const* ibuffer,
                           int ilen)
{
    zbuf a;

```



```

    a.zbuffer = (uint8*)ibuffer;
    a.zbuffer_end = (uint8*)ibuffer + ilen;
    if (do_zlib(&a, obuffer, olen, 0, 1))
        return (int)(a.zout - a.zout_start);
    else
        return -1;
}

char* stbi_zlib_decode_noheader_malloc(char const* buffer, int len, int* outlen)
{
    zbuf a;
    char* p = (char*)malloc(16384);
    if (p == NULL)
        return NULL;
    a.zbuffer = (uint8*)buffer;
    a.zbuffer_end = (uint8*)buffer + len;
    if (do_zlib(&a, p, 16384, 1, 0))
    {
        if (outlen)
            *outlen = (int)(a.zout - a.zout_start);
        return a.zout_start;
    }
    else
    {
        free(a.zout_start);
        return NULL;
    }
}

int stbi_zlib_decode_noheader_buffer(char* obuffer, int olen,
                                     const char* ibuffer, int ilen)
{
    zbuf a;
    a.zbuffer = (uint8*)ibuffer;
    a.zbuffer_end = (uint8*)ibuffer + ilen;
    if (do_zlib(&a, obuffer, olen, 0, 0))
        return (int)(a.zout - a.zout_start);
    else
        return -1;
}

// public domain "baseline" PNG decoder    v0.10   Sean Barrett 2006-11-18
//     simple implementation
//     - only 8-bit samples
//     - no CRC checking
//     - allocates lots of intermediate memory
//     - avoids problem of streaming data between subsystems
//     - avoids explicit window management
//     performance
//     - uses stb_zlib, a PD zlib implementation with fast huffman decoding

typedef struct
{
    uint32 length;
    uint32 type;
} chunk;

#define PNG_TYPE(a, b, c, d) (((a) << 24) + ((b) << 16) + ((c) << 8) + (d))

static chunk get_chunk_header(stbi* s)
{
    chunk c;
    c.length = get32(s);
    c.type = get32(s);
    return c;
}

static int check_png_header(stbi* s)
{
    static uint8 png_sig[8] = {137, 80, 78, 71, 13, 10, 26, 10};
    int i;
    for (i = 0; i < 8; ++i)
        if (get8u(s) != png_sig[i])
            return e("bad png sig", "Not a PNG");
    return 1;
}

typedef struct

```

```

{
    stbi* s;
    uint8 *idata, *expanded, *out;
} png;

enum
{
    F_none = 0,
    F_sub = 1,
    F_up = 2,
    F_avg = 3,
    F_paeth = 4,
    F_avg_first,
    F_paeth_first
};

static uint8 first_row_filter[5] = {F_none, F_sub, F_none, F_avg_first,
                                    F_paeth_first};

static int paeth(int a, int b, int c)
{
    int p = a + b - c;
    int pa = abs(p - a);
    int pb = abs(p - b);
    int pc = abs(p - c);
    if (pa <= pb && pa <= pc)
        return a;
    if (pb <= pc)
        return b;
    return c;
}

// create the png data from post-deflated data
static int create_png_image_raw(png* a, uint8* raw, uint32 raw_len, int out_n,
                                uint32 x, uint32 y)
{
    stbi* s = a->s;
    uint32 i, j, stride = x * out_n;
    int k;
    int img_n = s->img_n; // copy it into a local for later
    assert(out_n == s->img_n || out_n == s->img_n + 1);
    if (stbi_png_partial)
        y = 1;
    a->out = (uint8*)malloc(x * y * out_n);
    if (!a->out)
        return e("outofmem", "Out of memory");
    if (!stbi_png_partial)
    {
        if (s->img_x == x && s->img_y == y)
        {
            if (raw_len != (img_n * x + 1) * y)
                return e("not enough pixels", "Corrupt PNG");
        }
        else
        {
            // interlaced:
            if (raw_len < (img_n * x + 1) * y)
                return e("not enough pixels", "Corrupt PNG");
        }
    }
    for (j = 0; j < y; ++j)
    {
        uint8* cur = a->out + stride * j;
        uint8* prior = cur - stride;
        int filter = *raw++;
        if (filter > 4)
            return e("invalid filter", "Corrupt PNG");
        // if first row, use special filter that doesn't sample previous row
        if (j == 0)
            filter = first_row_filter[filter];
        // handle first pixel explicitly
        for (k = 0; k < img_n; ++k)
        {
            switch (filter)
            {
            case F_none:
                cur[k] = raw[k];
                break;

```

```

    case F_sub:
        cur[k] = raw[k];
        break;
    case F_up:
        cur[k] = raw[k] + prior[k];
        break;
    case F_avg:
        cur[k] = raw[k] + (prior[k] >> 1);
        break;
    case F_paeth:
        cur[k] = (uint8)(raw[k] + paeth(0, prior[k], 0));
        break;
    case F_avg_first:
        cur[k] = raw[k];
        break;
    case F_paeth_first:
        cur[k] = raw[k];
        break;
}
}
if (img_n != out_n)
    cur[img_n] = 255;
raw += img_n;
cur += out_n;
prior += out_n;
// this is a little gross, so that we don't switch per-pixel or
// per-component
if (img_n == out_n)
{
#define CASE(f) \
case f: \
    for (i = x - 1; i >= 1; --i, raw += img_n, cur += img_n, prior += img_n) \
        for (k = 0; k < img_n; ++k) \
            switch (filter) \
            { \
                CASE(F_none) cur[k] = raw[k]; \
                break; \
                CASE(F_sub) cur[k] = raw[k] + cur[k - img_n]; \
                break; \
                CASE(F_up) cur[k] = raw[k] + prior[k]; \
                break; \
                CASE(F_avg) cur[k] = raw[k] + ((prior[k] + cur[k - img_n]) >> 1); \
                break; \
                CASE(F_paeth) \
                cur[k] = \
                    (uint8)(raw[k] + paeth(cur[k - img_n], prior[k], prior[k - img_n])); \
                break; \
                CASE(F_avg_first) cur[k] = raw[k] + (cur[k - img_n] >> 1); \
                break; \
                CASE(F_paeth_first) \
                cur[k] = (uint8)(raw[k] + paeth(cur[k - img_n], 0, 0)); \
                break; \
            } \
#undef CASE
}
else
{
    assert(img_n + 1 == out_n);
#define CASE(f) \
case f: \
    for (i = x - 1; i >= 1; \
        --i, cur[img_n] = 255, raw += img_n, cur += out_n, prior += out_n) \
        for (k = 0; k < img_n; ++k) \
            switch (filter) \
            { \
                CASE(F_none) cur[k] = raw[k]; \
                break; \
                CASE(F_sub) cur[k] = raw[k] + cur[k - out_n]; \
                break; \
                CASE(F_up) cur[k] = raw[k] + prior[k]; \
                break; \
                CASE(F_avg) cur[k] = raw[k] + ((prior[k] + cur[k - out_n]) >> 1); \
                break; \
                CASE(F_paeth) \
                cur[k] = \
                    (uint8)(raw[k] + paeth(cur[k - out_n], prior[k], prior[k - out_n])); \
                break; \
            }

```

```

        CASE(F_avg_first) cur[k] = raw[k] + (cur[k - out_n] >> 1);
        break;
        CASE(F_paeth_first)
        cur[k] = (uint8)(raw[k] + paeth(cur[k - out_n], 0, 0));
        break;
    }
#undef CASE
}
}
return 1;
}

static int create_png_image(png* a, uint8* raw, uint32 raw_len, int out_n,
                           int interlaced)
{
    uint8* final;
    int p;
    int save;
    if (!interlaced)
        return create_png_image_raw(a, raw, raw_len, out_n, a->s->img_x,
                                    a->s->img_y);

    save = stbi_png_partial;
    stbi_png_partial = 0;

    // de-interlacing
    final = (uint8*)malloc(a->s->img_x * a->s->img_y * out_n);
    for (p = 0; p < 7; ++p)
    {
        int xorig[] = {0, 4, 0, 2, 0, 1, 0};
        int yorig[] = {0, 0, 4, 0, 2, 0, 1};
        int xspc[] = {8, 8, 4, 4, 2, 2, 1};
        int yspc[] = {8, 8, 8, 4, 4, 2, 2};
        int i, j, x, y;
        // pass1_x[4] = 0, pass1_x[5] = 1, pass1_x[12] = 1
        x = (a->s->img_x - xorig[p] + xspc[p] - 1) / xspc[p];
        y = (a->s->img_y - yorig[p] + yspc[p] - 1) / yspc[p];
        if (x && y)
        {
            if (!create_png_image_raw(a, raw, raw_len, out_n, x, y))
            {
                free(final);
                return 0;
            }
            for (j = 0; j < y; ++j)
                for (i = 0; i < x; ++i)
                    memcpy(final + (j * yspc[p] + yorig[p]) * a->s->img_x * out_n +
                           (i * xspc[p] + xorig[p]) * out_n,
                           a->out + (j * x + i) * out_n, out_n);
            free(a->out);
            raw += (x * out_n + 1) * y;
            raw_len -= (x * out_n + 1) * y;
        }
    }
    a->out = final;

    stbi_png_partial = save;
    return 1;
}

static int compute_transparency(png* z, uint8 tc[3], int out_n)
{
    stbi* s = z->s;
    uint32 i, pixel_count = s->img_x * s->img_y;
    uint8* p = z->out;

    // compute color-based transparency, assuming we've
    // already got 255 as the alpha value in the output
    assert(out_n == 2 || out_n == 4);

    if (out_n == 2)
    {
        for (i = 0; i < pixel_count; ++i)
        {
            {
                p[1] = (p[0] == tc[0] ? 0 : 255);
                p += 2;
            }
        }
    }
}

```

```

else
{
    for (i = 0; i < pixel_count; ++i)
    {
        if (p[0] == tc[0] && p[1] == tc[1] && p[2] == tc[2])
            p[3] = 0;
        p += 4;
    }
}
return 1;
}

static int expand_palette(png* a, uint8* palette, int len, int pal_img_n)
{
    uint32 i, pixel_count = a->s->img_x * a->s->img_y;
    uint8 *p, *temp_out, *orig = a->out;

    p = (uint8*)malloc(pixel_count * pal_img_n);
    if (p == NULL)
        return e("outofmem", "Out of memory");

    // between here and free(out) below, exiting would leak
    temp_out = p;

    if (pal_img_n == 3)
    {
        for (i = 0; i < pixel_count; ++i)
        {
            int n = orig[i] * 4;
            p[0] = palette[n];
            p[1] = palette[n + 1];
            p[2] = palette[n + 2];
            p += 3;
        }
    }
    else
    {
        for (i = 0; i < pixel_count; ++i)
        {
            int n = orig[i] * 4;
            p[0] = palette[n];
            p[1] = palette[n + 1];
            p[2] = palette[n + 2];
            p[3] = palette[n + 3];
            p += 4;
        }
    }
    free(a->out);
    a->out = temp_out;

    STBI_NOTUSED(len);

    return 1;
}

static int stbi_unpremultiply_on_load = 0;
static int stbi_de_iphone_flag = 0;

void stbi_set_unpremultiply_on_load(int flag_true_if_should_unpremultiply)
{
    stbi_unpremultiply_on_load = flag_true_if_should_unpremultiply;
}

void stbi_convert_iphone_png_to_rgb(int flag_true_if_should_convert)
{
    stbi_de_iphone_flag = flag_true_if_should_convert;
}

static void stbi_de_iphone(png* z)
{
    stbi* s = z->s;
    uint32 i, pixel_count = s->img_x * s->img_y;
    uint8* p = z->out;

    if (s->img_out_n == 3)
    {
        // convert bgr to rgb
        for (i = 0; i < pixel_count; ++i)
        {

```

```

        uint8 t = p[0];
        p[0] = p[2];
        p[2] = t;
        p += 3;
    }
}
else
{
    assert(s->img_out_n == 4);
    if (stbi_unpremultiply_on_load)
    {
        // convert bgr to rgb and unpremultiply
        for (i = 0; i < pixel_count; ++i)
        {
            uint8 a = p[3];
            uint8 t = p[0];
            if (a)
            {
                p[0] = p[2] * 255 / a;
                p[1] = p[1] * 255 / a;
                p[2] = t * 255 / a;
            }
            else
            {
                p[0] = p[2];
                p[2] = t;
            }
            p += 4;
        }
    }
    else
    {
        // convert bgr to rgb
        for (i = 0; i < pixel_count; ++i)
        {
            uint8 t = p[0];
            p[0] = p[2];
            p[2] = t;
            p += 4;
        }
    }
}
}

static int parse_png_file(png* z, int scan, int req_comp)
{
    uint8 palette[1024], pal_img_n = 0;
    uint8 has_trans = 0, tc[3];
    uint32 ioff = 0, idata_limit = 0, i, pal_len = 0;
    int first = 1, k, interlace = 0, iphone = 0;
    stbi* s = z->s;

    z->expanded = NULL;
    z->idata = NULL;
    z->out = NULL;

    if (!check_png_header(s))
        return 0;

    if (scan == SCAN_type)
        return 1;

    for (;;)
    {
        chunk c = get_chunk_header(s);
        switch (c.type)
        {
            case PNG_TYPE('C', 'g', 'B', 'I'):
                iphone = stbi_de_iphone_flag;
                skip(s, c.length);
                break;
            case PNG_TYPE('I', 'H', 'D', 'R'):
            {
                int depth, color, comp, filter;
                if (!first)
                    return e("multiple IHDR", "Corrupt PNG");
                first = 0;
            }

```

```

if (c.length != 13)
    return e("bad IHDR len", "Corrupt PNG");
s->img_x = get32(s);
if (s->img_x > (1 << 24))
    return e("too large", "Very large image (corrupt?)");
s->img_y = get32(s);
if (s->img_y > (1 << 24))
    return e("too large", "Very large image (corrupt?)");
depth = get8(s);
if (depth != 8)
    return e("8bit only", "PNG not supported: 8-bit only");
color = get8(s);
if (color > 6)
    return e("bad ctype", "Corrupt PNG");
if (color == 3)
    pal_img_n = 3;
else if (color & 1)
    return e("bad ctype", "Corrupt PNG");
comp = get8(s);
if (comp)
    return e("bad comp method", "Corrupt PNG");
filter = get8(s);
if (filter)
    return e("bad filter method", "Corrupt PNG");
interlace = get8(s);
if (interlace > 1)
    return e("bad interlace method", "Corrupt PNG");
if (!s->img_x || !s->img_y)
    return e("0-pixel image", "Corrupt PNG");
if (!pal_img_n)
{
    s->img_n = (color & 2 ? 3 : 1) + (color & 4 ? 1 : 0);
    if ((1 << 30) / s->img_x / s->img_n < s->img_y)
        return e("too large", "Image too large to decode");
    if (scan == SCAN_header)
        return 1;
}
else
{
    // if paletted, then pal_n is our final components, and
    // img_n is # components to decompress/filter.
    s->img_n = 1;
    if ((1 << 30) / s->img_x / 4 < s->img_y)
        return e("too large", "Corrupt PNG");
    // if SCAN_header, have to scan to see if we have a tRNS
}
break;
}

case PNG_TYPE('P', 'L', 'T', 'E'):
{
    if (first)
        return e("first not IHDR", "Corrupt PNG");
    if (c.length > 256 * 3)
        return e("invalid PLTE", "Corrupt PNG");
    pal_len = c.length / 3;
    if (pal_len * 3 != c.length)
        return e("invalid PLTE", "Corrupt PNG");
    for (i = 0; i < pal_len; ++i)
    {
        palette[i * 4 + 0] = get8u(s);
        palette[i * 4 + 1] = get8u(s);
        palette[i * 4 + 2] = get8u(s);
        palette[i * 4 + 3] = 255;
    }
    break;
}

case PNG_TYPE('t', 'R', 'N', 'S'):
{
    if (first)
        return e("first not IHDR", "Corrupt PNG");
    if (z->idata)
        return e("tRNS after IDAT", "Corrupt PNG");
    if (pal_img_n)
    {
        if (scan == SCAN_header)

```

```

{
    s->img_n = 4;
    return 1;
}
if (pal_len == 0)
    return e("tRNS before PLTE", "Corrupt PNG");
if (c.length > pal_len)
    return e("bad tRNS len", "Corrupt PNG");
pal_img_n = 4;
for (i = 0; i < c.length; ++i)
    palette[i * 4 + 3] = get8u(s);
}
else
{
    if (!(s->img_n & 1))
        return e("tRNS with alpha", "Corrupt PNG");
    if (c.length != (uint32)s->img_n * 2)
        return e("bad tRNS len", "Corrupt PNG");
    has_trans = 1;
    for (k = 0; k < s->img_n; ++k)
        tc[k] = (uint8)get16(s); // non 8-bit images will be larger
}
break;
}

case PNG_TYPE('I', 'D', 'A', 'T'):
{
    if (first)
        return e("first not IHDR", "Corrupt PNG");
    if (pal_img_n && !pal_len)
        return e("no PLTE", "Corrupt PNG");
    if (scan == SCAN_header)
    {
        s->img_n = pal_img_n;
        return 1;
    }
    if (ioff + c.length > idata_limit)
    {
        uint8* p;
        if (idata_limit == 0)
            idata_limit = c.length > 4096 ? c.length : 4096;
        while (ioff + c.length > idata_limit)
            idata_limit *= 2;
        p = (uint8*)realloc(z->idata, idata_limit);
        if (p == NULL)
            return e("outofmem", "Out of memory");
        z->idata = p;
    }
    if (!getn(s, z->idata + ioff, c.length))
        return e("outofdata", "Corrupt PNG");
    ioff += c.length;
    break;
}

case PNG_TYPE('I', 'E', 'N', 'D'):
{
    uint32 raw_len;
    if (first)
        return e("first not IHDR", "Corrupt PNG");
    if (scan != SCAN_load)
        return 1;
    if (z->idata == NULL)
        return e("no IDAT", "Corrupt PNG");
    z->expanded = (uint8*)stbi_zlib_decode_malloc_guesssize_headerflag(
        (char*)z->idata, ioff, 16384, (int*)&raw_len, !iphone);
    if (z->expanded == NULL)
        return 0; // zlib should set error
    free(z->idata);
    z->idata = NULL;
    if ((req_comp == s->img_n + 1 && req_comp != 3 && !pal_img_n) ||
        has_trans)
        s->img_out_n = s->img_n + 1;
    else
        s->img_out_n = s->img_n;
    if (!create_png_image(z, z->expanded, raw_len, s->img_out_n, interlace))
        return 0;
    if (has_trans)

```



```

        if (!compute_transparency(z, tc, s->img_out_n))
            return 0;
    if (iphone && s->img_out_n > 2)
        stbi_de_iphone(z);
    if (pal_img_n)
    {
        // pal_img_n == 3 or 4
        s->img_n = pal_img_n; // record the actual colors we had
        s->img_out_n = pal_img_n;
        if (req_comp >= 3)
            s->img_out_n = req_comp;
        if (!expand_palette(z, palette, pal_len, s->img_out_n))
            return 0;
    }
    free(z->expanded);
    z->expanded = NULL;
    return 1;
}

default:
    // if critical, fail
    if (first)
        return e("first not IHDR", "Corrupt PNG");
    if ((c.type & (1 << 29)) == 0)
    {
#ifdef STBI_NO_FAILURE_STRINGS
        // not threadsafe
        static char invalid_chunk[] = "XXXX chunk not known";
        invalid_chunk[0] = (uint8)(c.type >> 24);
        invalid_chunk[1] = (uint8)(c.type >> 16);
        invalid_chunk[2] = (uint8)(c.type >> 8);
        invalid_chunk[3] = (uint8)(c.type >> 0);
#else
        return e(invalid_chunk, "PNG not supported: unknown chunk type");
#endif
    }
    skip(s, c.length);
    break;
}
// end of chunk, read and skip CRC
get32(s);
}
}

static unsigned char* do_png(png* p, int* x, int* y, int* n, int req_comp)
{
    unsigned char* result = NULL;
    if (req_comp < 0 || req_comp > 4)
        return epuc("bad req_comp", "Internal error");
    if (parse_png_file(p, SCAN_load, req_comp))
    {
        result = p->out;
        p->out = NULL;
        if (req_comp && req_comp != p->s->img_out_n)
        {
            result = convert_format(result, p->s->img_out_n, req_comp, p->s->img_x,
                                   p->s->img_y);
            p->s->img_out_n = req_comp;
            if (result == NULL)
                return result;
        }
        *x = p->s->img_x;
        *y = p->s->img_y;
        if (n)
            *n = p->s->img_n;
    }
    free(p->out);
    p->out = NULL;
    free(p->expanded);
    p->expanded = NULL;
    free(p->idata);
    p->idata = NULL;

    return result;
}

static unsigned char* stbi_png_load(stbi* s, int* x, int* y, int* comp,
                                   int req_comp)

```

```

{
    png p;
    p.s = s;
    return do_png(&p, x, y, comp, req_comp);
}

static int stbi_png_test(stbi* s)
{
    int r;
    r = check_png_header(s);
    stbi_rewind(s);
    return r;
}

static int stbi_png_info_raw(png* p, int* x, int* y, int* comp)
{
    if (!parse_png_file(p, SCAN_header, 0))
    {
        stbi_rewind(p->s);
        return 0;
    }
    if (x)
        *x = p->s->img_x;
    if (y)
        *y = p->s->img_y;
    if (comp)
        *comp = p->s->img_n;
    return 1;
}

static int stbi_png_info(stbi* s, int* x, int* y, int* comp)
{
    png p;
    p.s = s;
    return stbi_png_info_raw(&p, x, y, comp);
}

// Microsoft/Windows BMP image

static int bmp_test(stbi* s)
{
    int sz;
    if (get8(s) != 'B')
        return 0;
    if (get8(s) != 'M')
        return 0;
    get32le(s); // discard filesize
    get16le(s); // discard reserved
    get16le(s); // discard reserved
    get32le(s); // discard data offset
    sz = get32le(s);
    if (sz == 12 || sz == 40 || sz == 56 || sz == 108)
        return 1;
    return 0;
}

static int stbi_bmp_test(stbi* s)
{
    int r = bmp_test(s);
    stbi_rewind(s);
    return r;
}

// returns 0..31 for the highest set bit
static int high_bit(unsigned int z)
{
    int n = 0;
    if (z == 0)
        return -1;
    if (z >= 0x10000)
        n += 16, z >>= 16;
    if (z >= 0x00100)
        n += 8, z >>= 8;
    if (z >= 0x00010)
        n += 4, z >>= 4;
    if (z >= 0x00004)
        n += 2, z >>= 2;

```

```

    if (z >= 0x000002)
        n += 1, z >>= 1;
    return n;
}

static int bitcount(unsigned int a)
{
    a = (a & 0x55555555) + ((a >> 1) & 0x55555555); // max 2
    a = (a & 0x33333333) + ((a >> 2) & 0x33333333); // max 4
    a = (a + (a >> 4)) & 0x0f0f0f0f; // max 8 per 4, now 8 bits
    a = (a + (a >> 8)); // max 16 per 8 bits
    a = (a + (a >> 16)); // max 32 per 8 bits
    return a & 0xff;
}

static int shiftsigned(int v, int shift, int bits)
{
    int result;
    int z = 0;

    if (shift < 0)
        v <<= -shift;
    else
        v >>= shift;
    result = v;

    z = bits;
    while (z < 8)
    {
        result += v >> z;
        z += bits;
    }
    return result;
}

static stbi_uc* bmp_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    uint8* out;
    unsigned int mr = 0, mg = 0, mb = 0, ma = 0, fake_a = 0;
    stbi_uc pal[256][4];
    int psize = 0, i, j, compress = 0, width;
    int bpp, flip_vertically, pad, target, offset, hsz;
    if (get8(s) != 'B' || get8(s) != 'M')
        return epuc("not BMP", "Corrupt BMP");
    get32le(s); // discard filesize
    get16le(s); // discard reserved
    get16le(s); // discard reserved
    offset = get32le(s);
    hsz = get32le(s);
    if (hsz != 12 && hsz != 40 && hsz != 56 && hsz != 108)
        return epuc("unknown BMP", "BMP type not supported: unknown");
    if (hsz == 12)
    {
        s->img_x = get16le(s);
        s->img_y = get16le(s);
    }
    else
    {
        s->img_x = get32le(s);
        s->img_y = get32le(s);
    }
    if (get16le(s) != 1)
        return epuc("bad BMP", "bad BMP");
    bpp = get16le(s);
    if (bpp == 1)
        return epuc("monochrome", "BMP type not supported: 1-bit");
    flip_vertically = ((int)s->img_y) > 0;
    s->img_y = abs((int)s->img_y);
    if (hsz == 12)
    {
        if (bpp < 24)
            psize = (offset - 14 - 24) / 3;
    }
    else
    {
        compress = get32le(s);
        if (compress == 1 || compress == 2)

```

```

    return epuc("BMP RLE", "BMP type not supported: RLE");
get32le(s); // discard sizeof
get32le(s); // discard hres
get32le(s); // discard vres
get32le(s); // discard colorsused
get32le(s); // discard max important
if (hsz == 40 || hsz == 56)
{
    if (hsz == 56)
    {
        get32le(s);
        get32le(s);
        get32le(s);
        get32le(s);
    }
    if (bpp == 16 || bpp == 32)
    {
        mr = mg = mb = 0;
        if (compress == 0)
        {
            if (bpp == 32)
            {
                mr = 0xffu << 16;
                mg = 0xffu << 8;
                mb = 0xffu << 0;
                ma = 0xffu << 24;
                fake_a = 1; // @TODO: check for cases like alpha value is all 0 and
                           // switch it to 255
            }
            else
            {
                mr = 31u << 10;
                mg = 31u << 5;
                mb = 31u << 0;
            }
        }
        else if (compress == 3)
        {
            mr = get32le(s);
            mg = get32le(s);
            mb = get32le(s);
            // not documented, but generated by photoshop and handled by mspaint
            if (mr == mg && mg == mb)
            {
                // ?!?!?
                return epuc("bad BMP", "bad BMP");
            }
        }
        else
            return epuc("bad BMP", "bad BMP");
    }
}
else
{
    assert(hsz == 108);
    mr = get32le(s);
    mg = get32le(s);
    mb = get32le(s);
    ma = get32le(s);
    get32le(s); // discard color space
    for (i = 0; i < 12; ++i)
        get32le(s); // discard color space parameters
}
if (bpp < 16)
    psize = (offset - 14 - hsz) >> 2;
}
s->img_n = ma ? 4 : 3;
if (req_comp && req_comp >= 3) // we can directly decode 3 or 4
    target = req_comp;
else
    target = s->img_n; // if they want monochrome, we'll post-convert
out = (stbi_uc*)malloc(target * s->img_x * s->img_y);
if (!out)
    return epuc("outofmem", "Out of memory");
if (bpp < 16)
{
    int z = 0;

```

```

if (psize == 0 || psize > 256)
{
    free(out);
    return epuc("invalid", "Corrupt BMP");
}
for (i = 0; i < psize; ++i)
{
    pal[i][2] = get8u(s);
    pal[i][1] = get8u(s);
    pal[i][0] = get8u(s);
    if (hsz != 12)
        get8(s);
    pal[i][3] = 255;
}
skip(s, offset - 14 - hsz - psize * (hsz == 12 ? 3 : 4));
if (bpp == 4)
    width = (s->img_x + 1) >> 1;
else if (bpp == 8)
    width = s->img_x;
else
{
    free(out);
    return epuc("bad bpp", "Corrupt BMP");
}
pad = (-width) & 3;
for (j = 0; j < (int)s->img_y; ++j)
{
    for (i = 0; i < (int)s->img_x; i += 2)
    {
        int v = get8(s), v2 = 0;
        if (bpp == 4)
        {
            v2 = v & 15;
            v >>= 4;
        }
        out[z++] = pal[v][0];
        out[z++] = pal[v][1];
        out[z++] = pal[v][2];
        if (target == 4)
            out[z++] = 255;
        if (i + 1 == (int)s->img_x)
            break;
        v = (bpp == 8) ? get8(s) : v2;
        out[z++] = pal[v][0];
        out[z++] = pal[v][1];
        out[z++] = pal[v][2];
        if (target == 4)
            out[z++] = 255;
    }
    skip(s, pad);
}
}
else
{
    int rshift = 0, gshift = 0, bshift = 0, ashift = 0, rcount = 0, gcount = 0,
        bcount = 0, acount = 0;
    int z = 0;
    int easy = 0;
    skip(s, offset - 14 - hsz);
    if (bpp == 24)
        width = 3 * s->img_x;
    else if (bpp == 16)
        width = 2 * s->img_x;
    else /* bpp = 32 and pad = 0 */
        width = 0;
    pad = (-width) & 3;
    if (bpp == 24)
    {
        easy = 1;
    }
    else if (bpp == 32)
    {
        if (mb == 0xff && mg == 0xff00 && mr == 0x00ff0000 && ma == 0xff000000)
            easy = 2;
    }
    if (!easy)
    {

```

```

    if (!mr || !mg || !mb)
    {
        free(out);
        return epuc("bad masks", "Corrupt BMP");
    }
    // right shift amt to put high bit in position #7
    rshift = high_bit(mr) - 7;
    rcount = bitcount(mr);
    gshift = high_bit(mg) - 7;
    gcount = bitcount(mg);
    bshift = high_bit(mb) - 7;
    bcount = bitcount(mb);
    ashift = high_bit(ma) - 7;
    acount = bitcount(ma);
}
for (j = 0; j < (int)s->img_y; ++j)
{
    if (easy)
    {
        for (i = 0; i < (int)s->img_x; ++i)
        {
            int a;
            out[z + 2] = get8u(s);
            out[z + 1] = get8u(s);
            out[z + 0] = get8u(s);
            z += 3;
            a = (easy == 2 ? get8(s) : 255);
            if (target == 4)
                out[z++] = (uint8)a;
        }
    }
    else
    {
        for (i = 0; i < (int)s->img_x; ++i)
        {
            uint32 v = (bpp == 16 ? get16le(s) : get32le(s));
            int a;
            out[z++] = (uint8)shiftsigned(v & mr, rshift, rcount);
            out[z++] = (uint8)shiftsigned(v & mg, gshift, gcount);
            out[z++] = (uint8)shiftsigned(v & mb, bshift, bcount);
            a = (ma ? shiftsigned(v & ma, ashift, acount) : 255);
            if (target == 4)
                out[z++] = (uint8)a;
        }
    }
    skip(s, pad);
}
}
if (flip_vertically)
{
    stbi_uc t;
    for (j = 0; j < (int)s->img_y >> 1; ++j)
    {
        stbi_uc* p1 = out + j * s->img_x * target;
        stbi_uc* p2 = out + (s->img_y - 1 - j) * s->img_x * target;
        for (i = 0; i < (int)s->img_x * target; ++i)
        {
            t = p1[i], p1[i] = p2[i], p2[i] = t;
        }
    }
}

if (req_comp && req_comp != target)
{
    out = convert_format(out, target, req_comp, s->img_x, s->img_y);
    if (out == NULL)
        return out; // convert_format frees input on failure
}

*x = s->img_x;
*y = s->img_y;
if (comp)
    *comp = s->img_n;
return out;
}

static stbi_uc* stbi_bmp_load(stbi* s, int* x, int* y, int* comp, int req_comp)

```

```

{
    return bmp_load(s, x, y, comp, req_comp);
}

// Targa Truevision - TGA
// by Jonathan Dummer

static int tga_info(stbi* s, int* x, int* y, int* comp)
{
    int tga_w, tga_h, tga_comp;
    int sz;
    get8u(s); // discard Offset
    sz = get8u(s); // color type
    if (sz > 1)
    {
        stbi_rewind(s);
        return 0; // only RGB or indexed allowed
    }
    sz = get8u(s); // image type
    // only RGB or grey allowed, +/- RLE
    if ((sz != 1) && (sz != 2) && (sz != 3) && (sz != 9) && (sz != 10) &&
        (sz != 11))
        return 0;
    skip(s, 9);
    tga_w = get16le(s);
    if (tga_w < 1)
    {
        stbi_rewind(s);
        return 0; // test width
    }
    tga_h = get16le(s);
    if (tga_h < 1)
    {
        stbi_rewind(s);
        return 0; // test height
    }
    sz = get8(s); // bits per pixel
    // only RGB or RGBA or grey allowed
    if ((sz != 8) && (sz != 16) && (sz != 24) && (sz != 32))
    {
        stbi_rewind(s);
        return 0;
    }
    tga_comp = sz;
    if (x)
        *x = tga_w;
    if (y)
        *y = tga_h;
    if (comp)
        *comp = tga_comp / 8;
    return 1; // seems to have passed everything
}

int stbi_tga_info(stbi* s, int* x, int* y, int* comp)
{
    return tga_info(s, x, y, comp);
}

static int tga_test(stbi* s)
{
    int sz;
    get8u(s); // discard Offset
    sz = get8u(s); // color type
    if (sz > 1)
        return 0; // only RGB or indexed allowed
    sz = get8u(s); // image type
    if ((sz != 1) && (sz != 2) && (sz != 3) && (sz != 9) && (sz != 10) &&
        (sz != 11))
        return 0; // only RGB or grey allowed, +/- RLE
    get16(s); // discard palette start
    get16(s); // discard palette length
    get8(s); // discard bits per palette color entry
    get16(s); // discard x origin
    get16(s); // discard y origin
    if (get16(s) < 1)
        return 0; // test width
    if (get16(s) < 1)

```

```

    return 0; // test height
    sz = get8(s); // bits per pixel
    if ((sz != 8) && (sz != 16) && (sz != 24) && (sz != 32))
        return 0; // only RGB or RGBA or grey allowed
    return 1; // seems to have passed everything
}

static int stbi_tga_test(stbi* s)
{
    int res = tga_test(s);
    stbi_rewind(s);
    return res;
}

static stbi_uc* tga_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    // read in the TGA header stuff
    int tga_offset = get8u(s);
    int tga_indexed = get8u(s);
    int tga_image_type = get8u(s);
    int tga_is_RLE = 0;
    int tga_palette_start = get16le(s);
    int tga_palette_len = get16le(s);
    int tga_palette_bits = get8u(s);
    int tga_x_origin = get16le(s);
    int tga_y_origin = get16le(s);
    int tga_width = get16le(s);
    int tga_height = get16le(s);
    int tga_bits_per_pixel = get8u(s);
    int tga_inverted = get8u(s);
    // image data
    unsigned char* tga_data;
    unsigned char* tga_palette = NULL;
    int i, j;
    unsigned char raw_data[4];
    unsigned char trans_data[4];
    int RLE_count = 0;
    int RLE_repeating = 0;
    int read_next_pixel = 1;

    // do a tiny bit of preprocessing
    if (tga_image_type >= 8)
    {
        tga_image_type -= 8;
        tga_is_RLE = 1;
    }
    /* int tga_alpha_bits = tga_inverted & 15; */
    tga_inverted = 1 - ((tga_inverted >> 5) & 1);

    // error check
    if ( (//(tga_indexed) ||
        (tga_width < 1) || (tga_height < 1) || (tga_image_type < 1) ||
        (tga_image_type > 3) ||
        ((tga_bits_per_pixel != 8) && (tga_bits_per_pixel != 16) &&
        (tga_bits_per_pixel != 24) && (tga_bits_per_pixel != 32)))
    {
        return NULL; // we don't report this as a bad TGA because we don't even know
        // if it's TGA
    }

    // If I'm paletted, then I'll use the number of bits from the palette
    if (tga_indexed)
    {
        tga_bits_per_pixel = tga_palette_bits;
    }

    // tga info
    *x = tga_width;
    *y = tga_height;
    if ((req_comp < 1) || (req_comp > 4))
    {
        // just use whatever the file was
        req_comp = tga_bits_per_pixel / 8;
        *comp = req_comp;
    }
    else
    {

```



```

    // force a new number of components
    *comp = tga_bits_per_pixel / 8;
}
tga_data = (unsigned char*)malloc(tga_width * tga_height * req_comp);
if (!tga_data)
    return epuc("outofmem", "Out of memory");

// skip to the data's starting position (offset usually = 0)
skip(s, tga_offset);
// do I need to load a palette?
if (tga_indexed)
{
    // any data to skip? (offset usually = 0)
    skip(s, tga_palette_start);
    // load the palette
    tga_palette =
        (unsigned char*)malloc(tga_palette_len * tga_palette_bits / 8);
    if (!tga_palette)
        return epuc("outofmem", "Out of memory");
    if (!getn(s, tga_palette, tga_palette_len * tga_palette_bits / 8))
    {
        free(tga_data);
        free(tga_palette);
        return epuc("bad palette", "Corrupt TGA");
    }
}
// load the data
trans_data[0] = trans_data[1] = trans_data[2] = trans_data[3] = 0;
for (i = 0; i < tga_width * tga_height; ++i)
{
    // if I'm in RLE mode, do I need to get a RLE chunk?
    if (tga_is_RLE)
    {
        if (RLE_count == 0)
        {
            // yep, get the next byte as a RLE command
            int RLE_cmd = get8u(s);
            RLE_count = 1 + (RLE_cmd & 127);
            RLE_repeating = RLE_cmd >> 7;
            read_next_pixel = 1;
        }
        else if (!RLE_repeating)
        {
            read_next_pixel = 1;
        }
    }
    else
    {
        read_next_pixel = 1;
    }
    // OK, if I need to read a pixel, do it now
    if (read_next_pixel)
    {
        // load however much data we did have
        if (tga_indexed)
        {
            // read in 1 byte, then perform the lookup
            int pal_idx = get8u(s);
            if (pal_idx >= tga_palette_len)
            {
                // invalid index
                pal_idx = 0;
            }
            pal_idx *= tga_bits_per_pixel / 8;
            for (j = 0; j * 8 < tga_bits_per_pixel; ++j)
            {
                raw_data[j] = tga_palette[pal_idx + j];
            }
        }
        else
        {
            // read in the data raw
            for (j = 0; j * 8 < tga_bits_per_pixel; ++j)
            {
                raw_data[j] = get8u(s);
            }
        }
    }
}

```

```

// convert raw to the intermediate format
switch (tga_bits_per_pixel)
{
case 8:
    // Luminous => RGBA
    trans_data[0] = raw_data[0];
    trans_data[1] = raw_data[0];
    trans_data[2] = raw_data[0];
    trans_data[3] = 255;
    break;
case 16:
    // Luminous,Alpha => RGBA
    trans_data[0] = raw_data[0];
    trans_data[1] = raw_data[0];
    trans_data[2] = raw_data[0];
    trans_data[3] = raw_data[1];
    break;
case 24:
    // BGR => RGBA
    trans_data[0] = raw_data[2];
    trans_data[1] = raw_data[1];
    trans_data[2] = raw_data[0];
    trans_data[3] = 255;
    break;
case 32:
    // BGRA => RGBA
    trans_data[0] = raw_data[2];
    trans_data[1] = raw_data[1];
    trans_data[2] = raw_data[0];
    trans_data[3] = raw_data[3];
    break;
}
// clear the reading flag for the next pixel
read_next_pixel = 0;
} // end of reading a pixel
// convert to final format
switch (req_comp)
{
case 1:
    // RGBA => Luminance
    tga_data[i * req_comp + 0] =
        compute_y(trans_data[0], trans_data[1], trans_data[2]);
    break;
case 2:
    // RGBA => Luminance,Alpha
    tga_data[i * req_comp + 0] =
        compute_y(trans_data[0], trans_data[1], trans_data[2]);
    tga_data[i * req_comp + 1] = trans_data[3];
    break;
case 3:
    // RGBA => RGB
    tga_data[i * req_comp + 0] = trans_data[0];
    tga_data[i * req_comp + 1] = trans_data[1];
    tga_data[i * req_comp + 2] = trans_data[2];
    break;
case 4:
    // RGBA => RGBA
    tga_data[i * req_comp + 0] = trans_data[0];
    tga_data[i * req_comp + 1] = trans_data[1];
    tga_data[i * req_comp + 2] = trans_data[2];
    tga_data[i * req_comp + 3] = trans_data[3];
    break;
}
// in case we're in RLE mode, keep counting down
--RLE_count;
}
// do I need to invert the image?
if (tga_inverted)
{
    for (j = 0; j * 2 < tga_height; ++j)
    {
        int index1 = j * tga_width * req_comp;
        int index2 = (tga_height - 1 - j) * tga_width * req_comp;
        for (i = tga_width * req_comp; i > 0; --i)
        {
            unsigned char temp = tga_data[index1];
            tga_data[index1] = tga_data[index2];

```

```

        tga_data[index2] = temp;
        ++index1;
        ++index2;
    }
}
// clear my palette, if I had one
if (tga_palette != NULL)
{
    free(tga_palette);
}
// the things I do to get rid of an error message, and yet keep
// Microsoft's C compilers happy... [8^(
tga_palette_start = tga_palette_len = tga_palette_bits = tga_x_origin =
    tga_y_origin = 0;
// OK, done
return tga_data;
}

static stbi_uc* stbi_tga_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    return tga_load(s, x, y, comp, req_comp);
}

// *****
// Photoshop PSD loader -- PD by Thatcher Ulrich, integration by Nicolas Schulz,
// tweaked by STB

static int psd_test(stbi* s)
{
    if (get32(s) != 0x38425053)
        return 0; // "8BPS"
    else
        return 1;
}

static int stbi_psd_test(stbi* s)
{
    int r = psd_test(s);
    stbi_rewind(s);
    return r;
}

static stbi_uc* psd_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    int pixelCount;
    int channelCount, compression;
    int channel, i, count, len;
    int w, h;
    uint8* out;

    // Check identifier
    if (get32(s) != 0x38425053) // "8BPS"
        return epuc("not PSD", "Corrupt PSD image");

    // Check file type version.
    if (get16(s) != 1)
        return epuc("wrong version", "Unsupported version of PSD image");

    // Skip 6 reserved bytes.
    skip(s, 6);

    // Read the number of channels (R, G, B, A, etc).
    channelCount = get16(s);
    if (channelCount < 0 || channelCount > 16)
        return epuc("wrong channel count",
            "Unsupported number of channels in PSD image");

    // Read the rows and columns of the image.
    h = get32(s);
    w = get32(s);

    // Make sure the depth is 8 bits.
    if (get16(s) != 8)
        return epuc("unsupported bit depth", "PSD bit depth is not 8 bit");

    // Make sure the color mode is RGB.

```

```

// Valid options are:
// 0: Bitmap
// 1: Grayscale
// 2: Indexed color
// 3: RGB color
// 4: CMYK color
// 7: Multichannel
// 8: Duotone
// 9: Lab color
if (get16(s) != 3)
    return epuc("wrong color format", "PSD is not in RGB color format");

// Skip the Mode Data. (It's the palette for indexed color; other info for
// other modes.)
skip(s, get32(s));

// Skip the image resources. (resolution, pen tool paths, etc)
skip(s, get32(s));

// Skip the reserved data.
skip(s, get32(s));

// Find out if the data is compressed.
// Known values:
// 0: no compression
// 1: RLE compressed
compression = get16(s);
if (compression > 1)
    return epuc("bad compression", "PSD has an unknown compression format");

// Create the destination image.
out = (stbi_uc*)malloc(4 * w * h);
if (!out)
    return epuc("outofmem", "Out of memory");
pixelCount = w * h;

// Initialize the data to zero.
// memset( out, 0, pixelCount * 4 );

// Finally, the image data.
if (compression)
{
    // RLE as used by .PSD and .TIFF
    // Loop until you get the number of unpacked bytes you are expecting:
    //   Read the next source byte into n.
    //   If n is between 0 and 127 inclusive, copy the next n+1 bytes
    //   literally. Else if n is between -127 and -1 inclusive, copy the next
    //   byte -n+1 times. Else if n is 128, noop.
    // Endloop

    // The RLE-compressed data is preceeded by a 2-byte data count for each row
    // in the data, which we're going to just skip.
    skip(s, h * channelCount * 2);

    // Read the RLE data by channel.
    for (channel = 0; channel < 4; channel++)
    {
        uint8* p;

        p = out + channel;
        if (channel >= channelCount)
        {
            // Fill this channel with default data.
            for (i = 0; i < pixelCount; i++)
                *p = (channel == 3 ? 255 : 0), p += 4;
        }
        else
        {
            // Read the RLE data.
            count = 0;
            while (count < pixelCount)
            {
                len = get8(s);
                if (len == 128)
                {
                    // No-op.
                }
            }
        }
    }
}

```

```

        else if (len < 128)
        {
            // Copy next len+1 bytes literally.
            len++;
            count += len;
            while (len)
            {
                *p = get8u(s);
                p += 4;
                len--;
            }
        }
        else if (len > 128)
        {
            uint8 val;
            // Next -len+1 bytes in the dest are replicated from next source
            // byte. (Interpret len as a negative 8-bit int.)
            len ^= 0xFF;
            len += 2;
            val = get8u(s);
            count += len;
            while (len)
            {
                *p = val;
                p += 4;
                len--;
            }
        }
    }
}
}
else
{
    // We're at the raw image data. It's each channel in order (Red, Green,
    // Blue, Alpha, ...) where each channel consists of an 8-bit value for each
    // pixel in the image.

    // Read the data by channel.
    for (channel = 0; channel < 4; channel++)
    {
        uint8* p;

        p = out + channel;
        if (channel > channelCount)
        {
            // Fill this channel with default data.
            for (i = 0; i < pixelCount; i++)
                *p = channel == 3 ? 255 : 0, p += 4;
        }
        else
        {
            // Read the data.
            for (i = 0; i < pixelCount; i++)
                *p = get8u(s), p += 4;
        }
    }
}

if (req_comp && req_comp != 4)
{
    out = convert_format(out, 4, req_comp, w, h);
    if (out == NULL)
        return out; // convert_format frees input on failure
}

if (comp)
    *comp = channelCount;
*y = h;
*x = w;

return out;
}

static stbi_uc* stbi_psd_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    return psd_load(s, x, y, comp, req_comp);
}

```

```

}

// *****
// Softimage PIC loader
// by Tom Seddon
//
// See http://softimage.wiki.softimage.com/index.php/INFO:\_PIC\_file\_format
// See http://ozviz.wasp.uwa.edu.au/~pbourke/dataformats/softimagepic/

static int pic_is4(stbi* s, const char* str)
{
    int i;
    for (i = 0; i < 4; ++i)
        if (get8(s) != (stbi_uc)str[i])
            return 0;

    return 1;
}

static int pic_test(stbi* s)
{
    int i;

    if (!pic_is4(s, "\x53\x80\xF6\x34"))
        return 0;

    for (i = 0; i < 84; ++i)
        get8(s);

    if (!pic_is4(s, "PICT"))
        return 0;

    return 1;
}

typedef struct
{
    stbi_uc size, type, channel;
} pic_packet_t;

static stbi_uc* pic_readval(stbi* s, int channel, stbi_uc* dest)
{
    int mask = 0x80, i;

    for (i = 0; i < 4; ++i, mask >>= 1)
    {
        if (channel & mask)
        {
            if (at_eof(s))
                return epuc("bad file", "PIC file too short");
            dest[i] = get8u(s);
        }
    }

    return dest;
}

static void pic_copyval(int channel, stbi_uc* dest, const stbi_uc* src)
{
    int mask = 0x80, i;

    for (i = 0; i < 4; ++i, mask >>= 1)
        if (channel & mask)
            dest[i] = src[i];
}

static stbi_uc* pic_load2(stbi* s, int width, int height, int* comp,
                          stbi_uc* result)
{
    int act_comp = 0, num_packets = 0, y, chained;
    pic_packet_t packets[10];

    // this will (should...) cater for even some bizarre stuff like having data
    // for the same channel in multiple packets.
    do
    {
        pic_packet_t* packet;

```

```

if (num_packets == sizeof(packets) / sizeof(packets[0]))
    return epuc("bad format", "too many packets");

packet = &packets[num_packets++];

chained = get8(s);
packet->size = get8u(s);
packet->type = get8u(s);
packet->channel = get8u(s);

act_comp |= packet->channel;

if (at_eof(s))
    return epuc("bad file", "file too short (reading packets)");
if (packet->size != 8)
    return epuc("bad format", "packet isn't 8bpp");
} while (chained);

*comp = (act_comp & 0x10 ? 4 : 3); // has alpha channel?

for (y = 0; y < height; ++y)
{
    int packet_idx;

    for (packet_idx = 0; packet_idx < num_packets; ++packet_idx)
    {
        pic_packet_t* packet = &packets[packet_idx];
        stbi_uc* dest = result + y * width * 4;

        switch (packet->type)
        {
        default:
            return epuc("bad format", "packet has bad compression type");

        case 0:
            { // uncompressed
                int x;

                for (x = 0; x < width; ++x, dest += 4)
                    if (!pic_readval(s, packet->channel, dest))
                        return 0;
                break;
            }

        case 1: // Pure RLE
            {
                int left = width, i;

                while (left > 0)
                {
                    stbi_uc count, value[4];

                    count = get8u(s);
                    if (at_eof(s))
                        return epuc("bad file", "file too short (pure read count)");

                    if (count > left)
                        count = (uint8)left;

                    if (!pic_readval(s, packet->channel, value))
                        return 0;

                    for (i = 0; i < count; ++i, dest += 4)
                        pic_copyval(packet->channel, dest, value);
                    left -= count;
                }
            }
        }
        break;

        case 2:
            { // Mixed RLE
                int left = width;
                while (left > 0)
                {
                    int count = get8(s), i;
                    if (at_eof(s))
                        return epuc("bad file", "file too short (mixed read count)");

```

```

        if (count >= 128)
        { // Repeated
            stbi_uc value[4];
            int i;

            if (count == 128)
                count = get16(s);
            else
                count -= 127;
            if (count > left)
                return epuc("bad file", "scanline overrun");

            if (!pic_readval(s, packet->channel, value))
                return 0;

            for (i = 0; i < count; ++i, dest += 4)
                pic_copyval(packet->channel, dest, value);
        }
        else
        { // Raw
            ++count;
            if (count > left)
                return epuc("bad file", "scanline overrun");

            for (i = 0; i < count; ++i, dest += 4)
                if (!pic_readval(s, packet->channel, dest))
                    return 0;
        }
        left -= count;
    }
    break;
}
}
}

return result;
}

static stbi_uc* pic_load(stbi* s, int* px, int* py, int* comp, int req_comp)
{
    stbi_uc* result;
    int i, x, y;

    for (i = 0; i < 92; ++i)
        get8(s);

    x = get16(s);
    y = get16(s);
    if (at_eof(s))
        return epuc("bad file", "file too short (pic header)");
    if ((1 < 28) / x < y)
        return epuc("too large", "Image too large to decode");

    get32(s); // skip `ratio'
    get16(s); // skip `fields'
    get16(s); // skip `pad'

    // intermediate buffer is RGBA
    result = (stbi_uc*)malloc(x * y * 4);
    memset(result, 0xff, x * y * 4);

    if (!pic_load2(s, x, y, comp, result))
    {
        free(result);
        result = 0;
    }
    *px = x;
    *py = y;
    if (req_comp == 0)
        req_comp = *comp;
    result = convert_format(result, 4, req_comp, x, y);

    return result;
}

static int stbi_pic_test(stbi* s)

```



```

{
    int r = pic_test(s);
    stbi_rewind(s);
    return r;
}

static stbi_uc* stbi_pic_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    return pic_load(s, x, y, comp, req_comp);
}

// *****
// GIF loader -- public domain by Jean-Marc Lienher -- simplified/shrunk by stb
typedef struct stbi_gif_lzw_struct
{
    int16 prefix;
    uint8 first;
    uint8 suffix;
} stbi_gif_lzw;

typedef struct stbi_gif_struct
{
    int w, h;
    stbi_uc* out; // output buffer (always 4 components)
    int flags, bindex, ratio, transparent, eflags;
    uint8 pal[256][4];
    uint8 lpal[256][4];
    stbi_gif_lzw codes[4096];
    uint8* color_table;
    int parse, step;
    int lflags;
    int start_x, start_y;
    int max_x, max_y;
    int cur_x, cur_y;
    int line_size;
} stbi_gif;

static int gif_test(stbi* s)
{
    int sz;
    if (get8(s) != 'G' || get8(s) != 'I' || get8(s) != 'F' || get8(s) != '8')
        return 0;
    sz = get8(s);
    if (sz != '9' && sz != '7')
        return 0;
    if (get8(s) != 'a')
        return 0;
    return 1;
}

static int stbi_gif_test(stbi* s)
{
    int r = gif_test(s);
    stbi_rewind(s);
    return r;
}

static void stbi_gif_parse_colortable(stbi* s, uint8 pal[256][4],
                                     int num_entries, int transp)
{
    int i;
    for (i = 0; i < num_entries; ++i)
    {
        pal[i][2] = get8u(s);
        pal[i][1] = get8u(s);
        pal[i][0] = get8u(s);
        pal[i][3] = transp ? 0 : 255;
    }
}

static int stbi_gif_header(stbi* s, stbi_gif* g, int* comp, int is_info)
{
    uint8 version;
    if (get8(s) != 'G' || get8(s) != 'I' || get8(s) != 'F' || get8(s) != '8')
        return e("not GIF", "Corrupt GIF");

    version = get8u(s);

```

```

if (version != '7' && version != '9')
    return e("not GIF", "Corrupt GIF");
if (get8(s) != 'a')
    return e("not GIF", "Corrupt GIF");

failure_reason = "";
g->w = get16le(s);
g->h = get16le(s);
g->flags = get8(s);
g->bgindex = get8(s);
g->ratio = get8(s);
g->transparent = -1;

if (comp != 0)
    *comp = 4; // can't actually tell whether it's 3 or 4 until we parse the
               // comments

if (is_info)
    return 1;

if (g->flags & 0x80)
    stbi_gif_parse_colortable(s, g->pal, 2 << (g->flags & 7), -1);

return 1;
}

static int stbi_gif_info_raw(stbi_gif* s, int* x, int* y, int* comp)
{
    stbi_gif g;
    if (!stbi_gif_header(s, &g, comp, 1))
    {
        stbi_rewind(s);
        return 0;
    }
    if (x)
        *x = g.w;
    if (y)
        *y = g.h;
    return 1;
}

static void stbi_out_gif_code(stbi_gif* g, uint16 code)
{
    uint8 *p, *c;

    // recurse to decode the prefixes, since the linked-list is backwards,
    // and working backwards through an interleaved image would be nasty
    if (g->codes[code].prefix >= 0)
        stbi_out_gif_code(g, g->codes[code].prefix);

    if (g->cur_y >= g->max_y)
        return;

    p = &g->out[g->cur_x + g->cur_y];
    c = &g->color_table[g->codes[code].suffix * 4];

    if (c[3] >= 128)
    {
        p[0] = c[2];
        p[1] = c[1];
        p[2] = c[0];
        p[3] = c[3];
    }
    g->cur_x += 4;

    if (g->cur_x >= g->max_x)
    {
        g->cur_x = g->start_x;
        g->cur_y += g->step;

        while (g->cur_y >= g->max_y && g->parse > 0)
        {
            g->step = (1 << g->parse) * g->line_size;
            g->cur_y = g->start_y + (g->step >> 1);
            --g->parse;
        }
    }
}

```

```

}

static uint8* stbi_process_gif_raster(stbi* s, stbi_gif* g)
{
    uint8 lzw_cs;
    int32 len, code;
    uint32 first;
    int32 codesize, codemask, avail, oldcode, bits, valid_bits, clear;
    stbi_gif_lzw* p;

    lzw_cs = get8u(s);
    clear = 1 << lzw_cs;
    first = 1;
    codesize = lzw_cs + 1;
    codemask = (1 << codesize) - 1;
    bits = 0;
    valid_bits = 0;
    for (code = 0; code < clear; code++)
    {
        g->codes[code].prefix = -1;
        g->codes[code].first = (uint8)code;
        g->codes[code].suffix = (uint8)code;
    }

    // support no starting clear code
    avail = clear + 2;
    oldcode = -1;

    len = 0;
    for (;;)
    {
        if (valid_bits < codesize)
        {
            if (len == 0)
            {
                len = get8(s); // start new block
                if (len == 0)
                    return g->out;
            }
            --len;
            bits |= (int32)get8(s) << valid_bits;
            valid_bits += 8;
        }
        else
        {
            int32 code = bits & codemask;
            bits >>= codesize;
            valid_bits -= codesize;
            // @OPTIMIZE: is there some way we can accelerate the non-clear path?
            if (code == clear)
            {
                // clear code
                codesize = lzw_cs + 1;
                codemask = (1 << codesize) - 1;
                avail = clear + 2;
                oldcode = -1;
                first = 0;
            }
            else if (code == clear + 1)
            {
                // end of stream code
                skip(s, len);
                while ((len = get8(s)) > 0)
                    skip(s, len);
                return g->out;
            }
            else if (code <= avail)
            {
                if (first)
                    return epuc("no clear code", "Corrupt GIF");

                if (oldcode >= 0)
                {
                    p = &g->codes[avail++];
                    if (avail > 4096)
                        return epuc("too many codes", "Corrupt GIF");
                    p->prefix = (int16)oldcode;
                    p->first = g->codes[oldcode].first;
                    p->suffix = (code == avail) ? p->first : g->codes[code].first;
                }
            }
        }
    }
}

```

```

    }
    else if (code == avail)
        return epuc("illegal code in raster", "Corrupt GIF");

    stbi_out_gif_code(g, (uint16)code);

    if ((avail & codemask) == 0 && avail <= 0x0FFF)
    {
        codesize++;
        codemask = (1 << codesize) - 1;
    }

    oldcode = code;
}
else
{
    return epuc("illegal code in raster", "Corrupt GIF");
}
}
}

static void stbi_fill_gif_background(stbi_gif* g)
{
    int i;
    uint8* c = g->pal[g->bgindex];
    // @OPTIMIZE: write a dword at a time
    for (i = 0; i < g->w * g->h * 4; i += 4)
    {
        uint8* p = &g->out[i];
        p[0] = c[2];
        p[1] = c[1];
        p[2] = c[0];
        p[3] = c[3];
    }
}

// this function is designed to support animated gifs, although stb_image
// doesn't support it
static uint8* stbi_gif_load_next(stbi* s, stbi_gif* g, int* comp, int req_comp)
{
    int i;
    uint8* old_out = 0;

    if (g->out == 0)
    {
        if (!stbi_gif_header(s, g, comp, 0))
            return 0; // failure_reason set by stbi_gif_header
        g->out = (uint8*)malloc(4 * g->w * g->h);
        if (g->out == 0)
            return epuc("outofmem", "Out of memory");
        stbi_fill_gif_background(g);
    }
    else
    {
        // animated-gif-only path
        if (((g->eflags & 0x1C) >> 2) == 3)
        {
            old_out = g->out;
            g->out = (uint8*)malloc(4 * g->w * g->h);
            if (g->out == 0)
                return epuc("outofmem", "Out of memory");
            memcpy(g->out, old_out, g->w * g->h * 4);
        }
    }
}

for (;;)
{
    switch (get8(s))
    {
        case 0x2C: /* Image Descriptor */
        {
            int32 x, y, w, h;
            uint8* o;

            x = get16le(s);
            y = get16le(s);

```

```

w = get16le(s);
h = get16le(s);
if (((x + w) > (g->w)) || ((y + h) > (g->h)))
    return epuc("bad Image Descriptor", "Corrupt GIF");

g->line_size = g->w * 4;
g->start_x = x * 4;
g->start_y = y * g->line_size;
g->max_x = g->start_x + w * 4;
g->max_y = g->start_y + h * g->line_size;
g->cur_x = g->start_x;
g->cur_y = g->start_y;

g->lflags = get8(s);

if (g->lflags & 0x40)
{
    g->step = 8 * g->line_size; // first interlaced spacing
    g->parse = 3;
}
else
{
    g->step = g->line_size;
    g->parse = 0;
}

if (g->lflags & 0x80)
{
    stbi_gif_parse_colortable(s, g->lpal, 2 << (g->lflags & 7),
                             g->eflags & 0x01 ? g->transparent : -1);
    g->color_table = (uint8*)g->lpal;
}
else if (g->flags & 0x80)
{
    for (i = 0; i < 256;
         ++i) // @OPTIMIZE: reset only the previous transparent
        g->pal[i][3] = 255;
    if (g->transparent >= 0 && (g->eflags & 0x01))
        g->pal[g->transparent][3] = 0;
    g->color_table = (uint8*)g->pal;
}
else
    return epuc("missing color table", "Corrupt GIF");

o = stbi_process_gif_raster(s, g);
if (o == NULL)
    return NULL;

if (req_comp && req_comp != 4)
    o = convert_format(o, 4, req_comp, g->w, g->h);
return o;
}

case 0x21: // Comment Extension.
{
    int len;
    if (get8(s) == 0xF9)
    { // Graphic Control Extension.
        len = get8(s);
        if (len == 4)
        {
            g->eflags = get8(s);
            get16le(s); // delay
            g->transparent = get8(s);
        }
        else
        {
            skip(s, len);
            break;
        }
    }
    while ((len = get8(s)) != 0)
        skip(s, len);
    break;
}

case 0x3B: // gif stream termination code

```

```

        return (uint8*)1;

    default:
        return epuc("unknown code", "Corrupt GIF");
    }
}

static stbi_uc* stbi_gif_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    uint8* u = 0;
    stbi_gif g = {0};

    u = stbi_gif_load_next(s, &g, comp, req_comp);
    if (u == (void*)1)
        u = 0; // end of animated gif marker
    if (u)
    {
        *x = g.w;
        *y = g.h;
    }

    return u;
}

static int stbi_gif_info(stbi* s, int* x, int* y, int* comp)
{
    return stbi_gif_info_raw(s, x, y, comp);
}

// *****
// Radiance RGBE HDR loader
// originally by Nicolas Schulz
#ifndef STBI_NO_HDR
static int hdr_test(stbi* s)
{
    const char* signature = "#?RADIANCE\n";
    int i;
    for (i = 0; signature[i]; ++i)
        if (get8(s) != signature[i])
            return 0;
    return 1;
}

static int stbi_hdr_test(stbi* s)
{
    int r = hdr_test(s);
    stbi_rewind(s);
    return r;
}

#define HDR_BUFLEN 1024
static char* hdr_gettoken(stbi* z, char* buffer)
{
    int len = 0;
    char c = '\0';

    c = (char)get8(z);

    while (!at_eof(z) && c != '\n')
    {
        buffer[len++] = c;
        if (len == HDR_BUFLEN - 1)
        {
            // flush to end of line
            while (!at_eof(z) && get8(z) != '\n')
                ;
            break;
        }
        c = (char)get8(z);
    }

    buffer[len] = 0;
    return buffer;
}

static void hdr_convert(float* output, stbi_uc* input, int req_comp)

```

```

{
    if (input[3] != 0)
    {
        float f1;
        // Exponent
        f1 = (float)ldexp(1.0f, input[3] - (int)(128 + 8));
        if (req_comp <= 2)
            output[0] = (input[0] + input[1] + input[2]) * f1 / 3;
        else
        {
            output[0] = input[0] * f1;
            output[1] = input[1] * f1;
            output[2] = input[2] * f1;
        }
        if (req_comp == 2)
            output[1] = 1;
        if (req_comp == 4)
            output[3] = 1;
    }
    else
    {
        switch (req_comp)
        {
            case 4:
                output[3] = 1; /* fallthrough */
            case 3:
                output[0] = output[1] = output[2] = 0;
                break;
            case 2:
                output[1] = 1; /* fallthrough */
            case 1:
                output[0] = 0;
                break;
        }
    }
}

static float* hdr_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    char buffer[HDR_BUFLEN];
    char* token;
    int valid = 0;
    int width, height;
    stbi_uc* scanline;
    float* hdr_data;
    int len;
    unsigned char count, value;
    int i, j, k, c1, c2, z;

    // Check identifier
    if (strcmp(hdr_gettoken(s, buffer), "#?RADIANCE") != 0)
        return epf("not HDR", "Corrupt HDR image");

    // Parse header
    for (;;)
    {
        token = hdr_gettoken(s, buffer);
        if (token[0] == 0)
            break;
        if (strcmp(token, "FORMAT=32-bit_rle_rgbe") == 0)
            valid = 1;
    }

    if (!valid)
        return epf("unsupported format", "Unsupported HDR format");

    // Parse width and height
    // can't use sscanf() if we're not using stdio!
    token = hdr_gettoken(s, buffer);
    if (strncmp(token, "-Y ", 3))
        return epf("unsupported data layout", "Unsupported HDR format");
    token += 3;
    height = strtol(token, &token, 10);
    while (*token == ' ')
        ++token;
    if (strncmp(token, "+X ", 3))
        return epf("unsupported data layout", "Unsupported HDR format");

```

```

token += 3;
width = strtol(token, NULL, 10);

*x = width;
*y = height;

*comp = 3;
if (req_comp == 0)
    req_comp = 3;

// Read data
hdr_data = (float*)malloc(height * width * req_comp * sizeof(float));

// Load image data
// image data is stored as some number of sca
if (width < 8 || width >= 32768)
{
    // Read flat data
    for (j = 0; j < height; ++j)
    {
        for (i = 0; i < width; ++i)
        {
            stbi_uc rgbe[4];
main_decode_loop:
            getn(s, rgbe, 4);
            hdr_convert(hdr_data + j * width * req_comp + i * req_comp, rgbe,
                        req_comp);
        }
    }
}
else
{
    // Read RLE-encoded data
    scanline = NULL;

    for (j = 0; j < height; ++j)
    {
        c1 = get8(s);
        c2 = get8(s);
        len = get8(s);
        if (c1 != 2 || c2 != 2 || (len & 0x80))
        {
            // not run-length encoded, so we have to actually use THIS data as a
            // decoded pixel (note this can't be a valid pixel--one of RGB must be
            // >= 128)
            uint8 rgbe[4];
            rgbe[0] = (uint8)c1;
            rgbe[1] = (uint8)c2;
            rgbe[2] = (uint8)len;
            rgbe[3] = (uint8)get8u(s);
            hdr_convert(hdr_data, rgbe, req_comp);
            i = 1;
            j = 0;
            free(scanline);
            goto main_decode_loop; // yes, this makes no sense
        }
        len <= 8;
        len |= get8(s);
        if (len != width)
        {
            free(hdr_data);
            free(scanline);
            return epf("invalid decoded scanline length", "corrupt HDR");
        }
    }
    if (scanline == NULL)
        scanline = (stbi_uc*)malloc(width * 4);

    for (k = 0; k < 4; ++k)
    {
        i = 0;
        while (i < width)
        {
            count = get8u(s);
            if (count > 128)
            {
                // Run
                value = get8u(s);

```



```

        count -= 128;
        for (z = 0; z < count; ++z)
            scanline[i++ * 4 + k] = value;
    }
    else
    {
        // Dump
        for (z = 0; z < count; ++z)
            scanline[i++ * 4 + k] = get8u(s);
    }
}
}
for (i = 0; i < width; ++i)
    hdr_convert(hdr_data + (j * width + i) * req_comp, scanline + i * 4,
                req_comp);
}
free(scanline);
}

return hdr_data;
}

static float* stbi_hdr_load(stbi* s, int* x, int* y, int* comp, int req_comp)
{
    return hdr_load(s, x, y, comp, req_comp);
}

static int stbi_hdr_info(stbi* s, int* x, int* y, int* comp)
{
    char buffer[HDR_BUFLen];
    char* token;
    int valid = 0;

    if (strcmp(hdr_gettoken(s, buffer), "#?RADIANCE") != 0)
    {
        stbi_rewind(s);
        return 0;
    }

    for (;;)
    {
        token = hdr_gettoken(s, buffer);
        if (token[0] == 0)
            break;
        if (strcmp(token, "FORMAT=32-bit_rle_rgbe") == 0)
            valid = 1;
    }

    if (!valid)
    {
        stbi_rewind(s);
        return 0;
    }
    token = hdr_gettoken(s, buffer);
    if (strncmp(token, "-Y ", 3))
    {
        stbi_rewind(s);
        return 0;
    }
    token += 3;
    *y = strtol(token, &token, 10);
    while (*token == ' ')
        ++token;
    if (strncmp(token, "+X ", 3))
    {
        stbi_rewind(s);
        return 0;
    }
    token += 3;
    *x = strtol(token, NULL, 10);
    *comp = 3;
    return 1;
}
#endif // STBI_NO_HDR

static int stbi_bmp_info(stbi* s, int* x, int* y, int* comp)
{

```

```

int hsz;
if (get8(s) != 'B' || get8(s) != 'M')
{
    stbi_rewind(s);
    return 0;
}
skip(s, 12);
hsz = get32le(s);
if (hsz != 12 && hsz != 40 && hsz != 56 && hsz != 108)
{
    stbi_rewind(s);
    return 0;
}
if (hsz == 12)
{
    *x = get16le(s);
    *y = get16le(s);
}
else
{
    *x = get32le(s);
    *y = get32le(s);
}
if (get16le(s) != 1)
{
    stbi_rewind(s);
    return 0;
}
*comp = get16le(s) / 8;
return 1;
}

static int stbi_psd_info(stbi* s, int* x, int* y, int* comp)
{
    int channelCount;
    if (get32(s) != 0x38425053)
    {
        stbi_rewind(s);
        return 0;
    }
    if (get16(s) != 1)
    {
        stbi_rewind(s);
        return 0;
    }
    skip(s, 6);
    channelCount = get16(s);
    if (channelCount < 0 || channelCount > 16)
    {
        stbi_rewind(s);
        return 0;
    }
    *y = get32(s);
    *x = get32(s);
    if (get16(s) != 8)
    {
        stbi_rewind(s);
        return 0;
    }
    if (get16(s) != 3)
    {
        stbi_rewind(s);
        return 0;
    }
    *comp = 4;
    return 1;
}

static int stbi_pic_info(stbi* s, int* x, int* y, int* comp)
{
    int act_comp = 0, num_packets = 0, chained;
    pic_packet_t packets[10];

    skip(s, 92);

    *x = get16(s);
    *y = get16(s);

```

```

    if (at_eof(s))
        return 0;
    if ((*x) != 0 && (1 << 28) / (*x) < (*y))
    {
        stbi_rewind(s);
        return 0;
    }

    skip(s, 8);

    do
    {
        pic_packet_t* packet;

        if (num_packets == sizeof(packets) / sizeof(packets[0]))
            return 0;

        packet = &packets[num_packets++];
        chained = get8(s);
        packet->size = get8u(s);
        packet->type = get8u(s);
        packet->channel = get8u(s);
        act_comp |= packet->channel;

        if (at_eof(s))
        {
            stbi_rewind(s);
            return 0;
        }
        if (packet->size != 8)
        {
            stbi_rewind(s);
            return 0;
        }
    } while (chained);

    *comp = (act_comp & 0x10 ? 4 : 3);

    return 1;
}

static int stbi_info_main(stbi* s, int* x, int* y, int* comp)
{
    if (stbi_jpeg_info(s, x, y, comp))
        return 1;
    if (stbi_png_info(s, x, y, comp))
        return 1;
    if (stbi_gif_info(s, x, y, comp))
        return 1;
    if (stbi_bmp_info(s, x, y, comp))
        return 1;
    if (stbi_psd_info(s, x, y, comp))
        return 1;
    if (stbi_pic_info(s, x, y, comp))
        return 1;
#ifdef STBI_NO_HDR
    if (stbi_hdr_info(s, x, y, comp))
        return 1;
#endif
    // test tga last because it's a crappy test!
    if (stbi_tga_info(s, x, y, comp))
        return 1;
    return e("unknown image type", "Image not of any known type, or corrupt");
}

#ifdef STBI_NO_STDIO
int stbi_info(char const* filename, int* x, int* y, int* comp)
{
    FILE* f = fopen(filename, "rb");
    int result;
    if (!f)
        return e("can't fopen", "Unable to open file");
    result = stbi_info_from_file(f, x, y, comp);
    fclose(f);
    return result;
}
int stbi_info_from_file(FILE* f, int* x, int* y, int* comp)

```

```

{
    int r;
    stbi s;
    long pos = ftell(f);
    start_file(&s, f);
    r = stbi_info_main(&s, x, y, comp);
    fseek(f, pos, SEEK_SET);
    return r;
}
#endif // !STBI_NO_STDIO

int stbi_info_from_memory(stbi_uc const* buffer, int len, int* x, int* y,
                          int* comp)
{
    stbi s;
    start_mem(&s, buffer, len);
    return stbi_info_main(&s, x, y, comp);
}

int stbi_info_from_callbacks(stbi_io_callbacks const* c, void* user, int* x,
                             int* y, int* comp)
{
    stbi s;
    start_callbacks(&s, (stbi_io_callbacks*)c, user);
    return stbi_info_main(&s, x, y, comp);
}

#endif // STBI_HEADER_FILE_ONLY

/*
revision history:
1.33 (2011-07-14)
    make stbi_is_hdr work in STBI_NO_HDR (as specified), minor
    compiler-friendly improvements 1.32 (2011-07-13) support for "info" function
    for all supported filetypes (SpartanJ) 1.31 (2011-06-20) a few more leak
    fixes, bug in PNG handling (SpartanJ) 1.30 (2011-06-11) added ability to load
    files via callbacks to accomodate custom input streams (Ben Wenger) removed
    deprecated format-specific test/load functions removed support for
    installable file formats (stbi_loader) -- would have been broken for IO
    callbacks anyway error cases in bmp and tga give messages and don't leak
    (Raymond Barbiero, grisha) fix inefficiency in decoding 32-bit BMP (David
    Woo) 1.29 (2010-08-16) various warning fixes from Aurelien Pocheville 1.28
    (2010-08-01) fix bug in GIF palette transparency (SpartanJ) 1.27 (2010-08-01)
    cast-to-uint8 to fix warnings
1.26 (2010-07-24)
    fix bug in file buffering for PNG reported by SpartanJ
1.25 (2010-07-17)
    refix trans_data warning (Won Chun)
1.24 (2010-07-12)
    perf improvements reading from files on platforms with lock-heavy
    fgetc() minor perf improvements for jpeg deprecated type-specific functions
    so we'll get feedback if they're needed attempt to fix trans_data warning
    (Won Chun) 1.23 fixed bug in iPhone support 1.22 (2010-07-10) removed image
    *writing* support stbi_info support from Jetro Lauha GIF support from
    Jean-Marc Lienher iPhone PNG-extensions from James Brown warning-fixes from
    Nicolas Schulz and Janez Zemva (i.e. Janez (U+017D)emva) 1.21 fix use of
    'uint8' in header (reported by jon blow) 1.20 added support for Softimage
    PIC, by Tom Seddon 1.19 bug in interlaced PNG corruption check (found by
    ryg) 1.18 2008-08-02 fix a threading bug (local mutable static) 1.17 support
    interlaced PNG 1.16 major bugfix - convert_format converted one too many
    pixels 1.15 initialize some fields for thread safety 1.14 fix threadsafe
    conversion bug header-file-only version (#define STBI_HEADER_FILE_ONLY before
    including) 1.13 threadsafe 1.12 const qualifiers in the API 1.11 Support
    installable IDCT, colorspace conversion routines 1.10 Fixes for 64-bit
    (don't use "unsigned long") optimized upsampling by Fabian "ryg" Giesen 1.09
    Fix format-conversion for PSD code (bad global variables!) 1.08 Thatcher
    Ulrich's PSD code integrated by Nicolas Schulz 1.07 attempt to fix C++
    warning/errors again 1.06 attempt to fix C++ warning/errors again 1.05 fix
    TGA loading to return correct *comp and use good luminance calc 1.04 default
    float alpha is 1, not 255; use 'void *' for stbi_image_free 1.03 bugfixes
    to STBI_NO_STDIO, STBI_NO_HDR 1.02 support for (subset of) HDR files, float
    interface for preferred access to them 1.01 fix bug: possible bug in
    handling right-side up bmps... not sure fix bug: the stbi_bmp_load() and
    stbi_tga_load() functions didn't work at all 1.00 interface to zlib that
    skips zlib header 0.99 correct handling of alpha in palette 0.98 TGA
    loader by lonesock; dynamically add loaders (untested) 0.97 jpeg errors on
    too large a file; also catch another malloc failure 0.96 fix detection of

```

invalid v value - particleman@mollyrocket forum 0.95 during header scan,
 seek to markers in case of padding 0.94 STBI_NO_STDIO to disable stdio
 usage; rename all #defines the same 0.93 handle jpegtran output; verbose
 errors 0.92 read 4,8,16,24,32-bit BMP files of several formats 0.91 output
 24-bit Windows 3.0 BMP files 0.90 fix a few more warnings; bump version
 number to approach 1.0 0.61 bugfixes due to Marc LeBlanc, Christopher Lloyd
 0.60 fix compiling as c++
 0.59 fix warnings: merge Dave Moore's -Wall fixes
 0.58 fix bug: zlib uncompressed mode len/nlen was wrong endian
 0.57 fix bug: jpg last huffman symbol before marker was >9 bits but less
 than 16 available 0.56 fix bug: zlib uncompressed mode len vs. nlen 0.55
 fix bug: restart_interval not initialized to 0 0.54 allow NULL for 'int
 *comp' 0.53 fix bug in png 3->4; speedup png decoding 0.52 png handles
 req_comp=3,4 directly; minor cleanup; jpeg comments 0.51 obey req_comp
 requests, 1-component jpegs return as 1-component, on 'test' only check type,
 not whether we support this variant 0.50 first released version

*/

Sources/texture.cpp

```
#include "stb_image.h"
#include "texture.h"
#include <iostream>

Texture* Texture::m_lastBind = 0;

Texture::Texture(const std::string& fileName, GLenum textureTarget,
                 GLfloat filter)
{
    int x, y, numComponents;
    unsigned char* data =
        stbi_load((std::string(RESOURCE_PATH) + "/texture/" + fileName).c_str(),
                 &x, &y, &numComponents, 4);
    if (data == NULL)
    {
        std::cerr << "Unable to load texture: " << fileName << std::endl;
    }
    initTexture(x, y, data, textureTarget, filter);

    stbi_image_free(data);
}

Texture::Texture(int width, int height, unsigned char* data,
                 GLenum textureTarget, GLfloat filter)
{
    initTexture(width, height, data, textureTarget, filter);
}

void Texture::initTexture(int width, int height, unsigned char* data,
                          GLenum textureTarget, GLfloat filter)
{
    m_textureTarget = textureTarget;
    m_freeTexture = true;

    if (width > 0 && height > 0 && data != 0)
    {
        glGenTextures(1, &m_textureID);
        glBindTexture(textureTarget, m_textureID);
        glTexParameterf(textureTarget, GL_TEXTURE_MIN_FILTER, filter);
        glTexParameterf(textureTarget, GL_TEXTURE_MAG_FILTER, filter);
        glTexParameterf(textureTarget, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameterf(textureTarget, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexImage2D(textureTarget, 0, GL_RGBA, width, height, 0, GL_RGBA,
                     GL_UNSIGNED_BYTE, data);
    }
    else
    {
        m_textureID = 0;
    }
}

Texture::~Texture()
{
    if (m_freeTexture)
        glDeleteTextures(1, &m_textureID);
}

Texture::Texture(Texture& texture)
{
    m_textureTarget = texture.m_textureTarget;
    m_textureID = texture.m_textureID;
    m_freeTexture = true;
    texture.m_freeTexture = false;
}

Texture& Texture::operator=(Texture&& other)
{
    m_textureTarget = other.m_textureTarget;
    m_textureID = other.m_textureID;
    m_freeTexture = true;
    other.m_freeTexture = false;

    return *this;
}
```

```
void Texture::operator=(Texture& texture)
{
    m_textureTarget = texture.m_textureTarget;
    m_textureID = texture.m_textureID;
    m_freeTexture = true;
    texture.m_freeTexture = false;
}

void Texture::bind(GLenum textureUnit)
{
    if (m_lastBind != this)
    {
        glActiveTexture(textureUnit);
        glBindTexture(m_textureTarget, m_textureID);
        m_lastBind = this;
    }
}

GLuint Texture::getID()
{
    return m_textureID;
}
```

Sources/transform.cpp

```
#include "transform.h"

static Camera cameraInstance;

Camera* Transform::m_camera = &cameraInstance;
float Transform::m_zFar = 0;
float Transform::m_zNear = 0;
float Transform::m_fov = 0;
float Transform::m_width = 0;
float Transform::m_height = 0;

Transform::Transform(Vector3f pos, Vector3f rot, Vector3f scale)
{
    m_pos = pos;
    m_rot = rot;
    m_scale = scale;
}

Transform::~Transform()
{
}

Matrix4f Transform::getTransformation()
{
    Matrix4f translationMatrix;
    Matrix4f rotationMatrix;
    Matrix4f scaleMatrix;

    translationMatrix.initTranslationTransform(m_pos.x, m_pos.y, m_pos.z);
    rotationMatrix.initRotateTransform(m_rot.x, m_rot.y, m_rot.z);
    scaleMatrix.initScaleTransform(m_scale.x, m_scale.y, m_scale.z);

    return translationMatrix * rotationMatrix * scaleMatrix;
}

Matrix4f Transform::getProjectedTransformation()
{
    Matrix4f transformationMatrix = getTransformation();
    Matrix4f projectionMatrix;
    Matrix4f cameraRotation;
    Matrix4f cameraTranslation;

    projectionMatrix.initPersProjTransform(m_fov, m_width, m_height, m_zNear,
                                           m_zFar);
    cameraRotation.initCameraTransform(m_camera->getForward(), m_camera->getUp());
    cameraTranslation.initTranslationTransform(
        -m_camera->getPos().x, -m_camera->getPos().y, -m_camera->getPos().z);

    return projectionMatrix * cameraRotation * cameraTranslation *
           transformationMatrix;
}

const Vector3f& Transform::getPos() const
{
    return m_pos;
}

const Vector3f& Transform::getRot() const
{
    return m_rot;
}

const Vector3f& Transform::getScale() const
{
    return m_scale;
}

Vector3f& Transform::getPos()
{
    return m_pos;
}

Vector3f& Transform::getRot()
{
}
```



```

    return m_rot;
}

Vector3f& Transform::getScale()
{
    return m_scale;
}

void Transform::setPos(const Vector3f& pos)
{
    this->m_pos = pos;
}

void Transform::setRot(const Vector3f& rot)
{
    this->m_rot = rot;
}

void Transform::setScale(const Vector3f& scale)
{
    this->m_scale = scale;
}

void Transform::setProjection(float fov, float width, float height, float zNear,
                             float zFar)
{
    Transform::m_fov = fov;
    Transform::m_width = width;
    Transform::m_height = height;
    Transform::m_zNear = zNear;
    Transform::m_zFar = zFar;
}

Camera& Transform::getCamera()
{
    return *m_camera;
}

void Transform::setCamera(Camera& camera)
{
    Transform::m_camera = &camera;
}

```

Sources/vector2f.cpp

```
#include "util.h"
#include "vector2f.h"
#include <math.h>

Vector2f& Vector2f::operator+=(const Vector2f& r)
{
    x += r.x;
    y += r.y;

    return *this;
}

Vector2f& Vector2f::operator-=(const Vector2f& r)
{
    x -= r.x;
    y -= r.y;

    return *this;
}

Vector2f& Vector2f::operator*=(float f)
{
    x *= f;
    y *= f;

    return *this;
}

Vector2f& Vector2f::operator/=(float f)
{
    x /= f;
    y /= f;

    return *this;
}

Vector2f Vector2f::operator+(const Vector2f& r) const
{
    Vector2f ret(x + r.x, y + r.y);

    return ret;
}

Vector2f Vector2f::operator-(const Vector2f& r) const
{
    Vector2f ret(x - r.x, y - r.y);

    return ret;
}

Vector2f Vector2f::operator*(float f) const
{
    Vector2f ret(x * f, y * f);

    return ret;
}

Vector2f Vector2f::operator/(float f) const
{
    Vector2f ret(x / f, y / f);

    return ret;
}

Vector2f Vector2f::rotate(float angle)
{
    double rad = ToRadian(angle);
    double cos = cosf(rad);
    double sin = sinf(rad);

    Vector2f ret =
        Vector2f((float)(x * cos - y * sin), (float)(x * sin + y * cos));
    return ret;
}

bool Vector2f::operator==(const Vector2f& r) const
{
}
```

```

    return x == r.x && y == r.y;
}

bool Vector2f::operator!=(const Vector2f& r) const
{
    return !operator==(r);
}

Vector2f Vector2f::mult(const Vector2f& other)
{
    Vector2f ret = Vector2f(x * other.x, y * other.y);
    return ret;
}

float Vector2f::getX() const
{
    return this->x;
}

float Vector2f::getY() const
{
    return this->y;
}

void Vector2f::setX(float value)
{
    this->x = value;
}

void Vector2f::setY(float value)
{
    this->y = value;
}

float Vector2f::length() const
{
    return sqrtf(x * x + y * y);
}

Vector2f& Vector2f::normalize()
{
    const float vectorLength = length();

    x /= vectorLength;
    y /= vectorLength;

    return *this;
}

```

Sources/vector3f.cpp

```
#include "quaternion.h"
#include "util.h"
#include "vector3f.h"
#include <cmath>
#include <math.h>

const Vector3f Vector3f::UP(0, 1, 0);
const Vector3f Vector3f::DOWN(0, -1, 0);
const Vector3f Vector3f::LEFT(-1, 0, 0);
const Vector3f Vector3f::RIGHT(1, 0, 0);
const Vector3f Vector3f::FORWARD(0, 0, 1);
const Vector3f Vector3f::BACK(0, 0, -1);
const Vector3f Vector3f::ONE(1, 1, 1);
const Vector3f Vector3f::ZERO(0, 0, 0);

Vector3f::Vector3f(float x, float y, float z)
{
    this->x = x;
    this->y = y;
    this->z = z;
}

float Vector3f::length() const
{
    return sqrtf(x * x + y * y + z * z);
}

float Vector3f::dot(const Vector3f& v) const
{
    return x * v.x + y * v.y + z * v.z;
}

Vector3f Vector3f::cross(const Vector3f& v) const
{
    const float _x = y * v.z - z * v.y;
    const float _y = z * v.x - x * v.z;
    const float _z = x * v.y - y * v.x;

    return Vector3f(_x, _y, _z);
}

Vector3f& Vector3f::normalize()
{
    const float vectorLength = length();

    x /= vectorLength;
    y /= vectorLength;
    z /= vectorLength;

    return *this;
}

Vector3f Vector3f::rotate(float angle, const Vector3f& axe) const
{
    const float sinHalfAngle = sinf(ToRadian(angle / 2));
    const float cosHalfAngle = cosf(ToRadian(angle / 2));

    const float Rx = axe.x * sinHalfAngle;
    const float Ry = axe.y * sinHalfAngle;
    const float Rz = axe.z * sinHalfAngle;
    const float Rw = cosHalfAngle;

    Quaternion rotationQ(Rx, Ry, Rz, Rw);

    Quaternion conjugateQ = rotationQ.conjugate();
    // ConjugateQ.Normalize();
    Quaternion w = rotationQ * (*this) * conjugateQ;

    Vector3f ret(w.x, w.y, w.z);

    return ret;
}

Vector3f& Vector3f::operator+=(const Vector3f& r)
```

```

{
    x += r.x;
    y += r.y;
    z += r.z;

    return *this;
}

Vector3f& Vector3f::operator-=(const Vector3f& r)
{
    x -= r.x;
    y -= r.y;
    z -= r.z;

    return *this;
}

Vector3f& Vector3f::operator*=(float f)
{
    x *= f;
    y *= f;
    z *= f;

    return *this;
}

Vector3f& Vector3f::operator/=(float f)
{
    x /= f;
    y /= f;
    z /= f;

    return *this;
}

Vector3f Vector3f::operator+(const Vector3f& r) const
{
    Vector3f ret(x + r.x, y + r.y, z + r.z);

    return ret;
}

Vector3f Vector3f::operator-(const Vector3f& r) const
{
    Vector3f ret(x - r.x, y - r.y, z - r.z);
    return ret;
}

Vector3f Vector3f::operator*(float f) const
{
    Vector3f ret(x * f, y * f, z * f);
    return ret;
}

Vector3f Vector3f::operator/(float f) const
{
    Vector3f ret(x / f, y / f, z / f);
    return ret;
}

inline bool Vector3f::operator==(const Vector3f& r) const
{
    return x == r.x && y == r.y && z == r.z;
}

inline bool Vector3f::operator!=(const Vector3f& r) const
{
    return !operator==(r);
}

void Vector3f::setX(float value)
{
    this->x = value;
}

void Vector3f::setY(float value)
{

```

```
    this->y = value;
}

void Vector3f::setZ(float value)
{
    this->z = value;
}

float Vector3f::getX() const
{
    return this->x;
}

float Vector3f::getY() const
{
    return this->y;
}

float Vector3f::getZ() const
{
    return this->z;
}

Vector3f Vector3f::mult(const Vector3f& r)
{
    Vector3f ret = Vector3f(x * r.getX(), y * r.getY(), z * r.getZ());
    return ret;
}
```

Sources/window.cpp

```
#include "sdl_requests.h"
#include "util.h"
#include "window.h"

#include <GL/glew.h>
#include <SDL2/SDL.h>
#include <SDL2/SDL_mixer.h>

#include <iostream>

int Window::s_width = 0;
int Window::s_height = 0;
std::string Window::s_title = "Title";

void Window::create(int width, int height, const std::string& title)
{
    Window::s_width = width;
    Window::s_height = height;
    Window::s_title = title;

    SDL_Init(SDL_INIT_VIDEO);

    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 4);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 1);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_CORE);

    SDL_GL_SetAttribute(SDL_GL_RED_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_GREEN_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_BLUE_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_ALPHA_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_BUFFER_SIZE, 32);
    SDL_GL_SetAttribute(SDL_GL_DEPTH_SIZE, 16);
    SDL_GL_SetAttribute(SDL_GL_DOUBLEBUFFER, 1);

    SDL_CreateWindow(title.c_str(), SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
        width, height, false);

    GLenum res = glewInit();
    if (res != GLEW_OK)
    {
        fprintf(stderr, "Error: '%s'\n", glewGetErrorString(res));
    }

    if (SDL_Init(SDL_INIT_AUDIO) < 0)
    {
        std::cerr << "Failed to initialize SDL: " << SDL_GetError() << std::endl;
    }

    if (Mix_OpenAudio(44100, MIX_DEFAULT_FORMAT, 2, 2048) < 0)
    {
        std::cerr << "Failed to initialize SDL_mixer: " << Mix_GetError()
            << std::endl;
        SDL_Quit();
    }
    Mix_AllocateChannels(16);
}

void Window::render()
{
    SDL_SwapBuffers();
}

void Window::dispose()
{
    Mix_Quit();
    SDL_DestroyWindow();
    SDL_Quit();
}

bool Window::isCloseRequested()
{
    return SDL_GetIsCloseRequested();
}

void Window::setFullScreen(bool value)
```

```

{
    SDLSetWindowFullscreen(value);
}

int Window::getWidth()
{
    return s_width;
}

int Window::getHeight()
{
    return s_height;
}

const std::string& Window::getTitle()
{
    return s_title;
}

void Window::playSound(const std::string& fileName, int channel, float volume)
{
    std::string filePath = std::string(RESOURCE_PATH) + "/audio/" + fileName;
    Mix_Chunk* sound = Mix_LoadWAV(filePath.c_str());

    if (!sound)
    {
        std::cerr << "Failed to load sound: " << Mix_GetError() << std::endl;
        Mix_CloseAudio();
        return;
    }
    Mix_MasterVolume(volume * MIX_MAX_VOLUME);
    if ((channel == PLAYER_FIRE_CHANNEL || channel == MONSTER_FIRE_CHANNEL))
    {
        Mix_PlayChannel(channel, sound, 0);
    }
    else if (Mix_Playing(channel) > 0)
    {
        return;
    }
    if (Mix_PlayChannel(channel, sound, 0) == -1)
    {
        std::cerr << "Failed to play sound: " << Mix_GetError() << std::endl;
        Mix_FreeChunk(sound);
        return;
    }
}

```


main.cpp

```
#include "main.h"
#include <stdio.h>

const static int WIDTH = 800;
const static int HEIGHT = 600;
const static char* TITLE = "Wolfenstein 3D";
const static double FRAME_CAP = 100.0;
const static bool IGNORE_FRAME_CAP = true;

Main::Main()
{
    RenderUtil::initGraphics();
    isRunning = false;
}

Main::~Main()
{
    Window::dispose();
}

void Main::start()
{
    if (isRunning)
        return;

    run();
}

void Main::stop()
{
    if (!isRunning)
        return;

    isRunning = false;
}

void Main::run()
{
    isRunning = true;

    const double frameTime = 1.0 / FRAME_CAP;

    double lastTime = Time::getTime();
    double unprocessedTime = 0;
    double frameCounter = 0;
    int frames = 0;

    while (isRunning)
    {
        bool render = false;

        double startTime = Time::getTime();
        double passedTime = startTime - lastTime;
        lastTime = startTime;

        unprocessedTime += passedTime;
        frameCounter += passedTime;

        if (frameCounter >= 1.0)
        {
            // printf("%i\n", frames);
            frames = 0;
            frameCounter = 0;
        }

        while (unprocessedTime > frameTime)
        {
            render = true;

            if (Window::isCloseRequested())
                isRunning = false;

            Time::setDelta(frameTime);
            Input::update();
        }
    }
}
```

```

        game.input();
        game.update();

        unprocessedTime -= frameTime;
    }

    if (render || IGNORE_FRAME_CAP)
    {
        RenderUtil::clearScreen();
        game.render();
        window::render();
        frames++;
    }
    else
    {
        Util::sleep(1);
    }
}

int main()
{
    Window::create(WIDTH, HEIGHT, (char*)TITLE);

    Main main;

    main.start();

    return 0;
}

```